ABC 802 inifrån

I samma familj:

ABC80 inifrån
Band 1: Maskinen och BASIC:en
Band 2: Utbyggnaderna

ABC 802 inifrån

Maskinen, ROM:en och programvaran

Set Lonnert

Arbetsutkast

ABC 802 inifrån

© Set Lonnert 2026

Arbetsutkast. Utgiven som PDF.

Boken är skriven av Set Lonnert tillsammans med Claude, en AI-modell från Anthropic, använd i verktyget Claude Code. Claude har disassemblerat ROM:arna, följt programflödet, föreslagit namn på rutiner och variabler, skrivit kommentarerna i listningen, skrivit emulatorn, bearbetat och formulerat om författarens underlag till löpande text, ritat skisserna och kontrollerat sidhänvisningarna mot källorna. Set Lonnert har lagt upp och styrt arbetet, valt ämnen och källor, skrivit underlaget till texterna, tagit fotografierna, prövat tolkningarna och läst, rättat och godkänt texten. Ansvaret för innehållet är författarens. Listningen går att assemblera på nytt till samma byte som kretsarna.

Fotografierna är författarens där inget annat anges. Bilder ur andra verk har källan i bildtexten.

Boken återger omslag till manualer och tidskrifter och programkod ur datorns kretsar. De ursprungliga rättighetsinnehavarna har eftersökts, men flera av förlagen, företagen och upphovspersonerna finns inte längre eller går inte att identifiera. Materialet återges i dokumentärt, teknikhistoriskt och kulturhistoriskt syfte. Om du har rätten till något av materialet (fotografier, omslag, illustrationer eller programkod) och anser att den har förbisetts, skriv till `setlonnert@gmail.com`. Vi kan då komma överens om namnpublicering eller borttagning i kommande utgåvor.

Innehåll

Förord **1**

1 ABC 802 i sin tid **3**
1.1 Våren 1983 . . . 3
1.2 Vem den var till för . . . 3
1.3 Omvärlden . . . 4
1.4 Samma kretsar på andra håll . . . 4
1.5 Hur det gick . . . 5

I Maskinen **6**

2 Maskinen **7**
2.1 Datorn i bildskärmen . . . 7
2.2 Datorkortet . . . 8
2.3 Kommunikationsmodulen . . . 12
2.4 Klockorna . . . 13
2.5 Minneskartan . . . 13
2.6 Portkartan . . . 13
2.7 Avbrott, NMI och återställning . . . 14
2.8 Anslutningarna . . . 14
2.9 Jämfört med ABC80 . . . 15

3 Bildskärmen **17**
3.1 Bilden . . . 17
3.2 Bildkretsen 6845 . . . 18
3.3 Bildminnet . . . 19
3.4 Att rulla . . . 20
3.5 25 rader . . . 21
3.6 Teckengeneratorn . . . 23
3.7 40 och 80 tecken . . . 24
3.8 Attributen och grafiken . . . 25
3.9 Markören och skärmsläckaren . . . 27

4 Tangentbordet **28**
4.1 Ett bord för sig . . . 28
4.2 Fyra tangentbord . . . 28
4.3 Processorn . . . 29
4.4 Sladden . . . 30
4.5 Koderna . . . 31
4.6 Funktionstangenterna . . . 31
4.7 Att styra tangentbordet . . . 31
4.8 Upp och ned . . . 32
4.9 Två ljud . . . 32
4.10 Datorns sida . . . 33
4.11 Att skriva en rad . . . 33

5 Minnet **35**

5.1 Tre minnen på samma adresser . . . 35
5.2 ROM:en . . . 36
5.3 Det dynamiska RAM:et . . . 36
5.4 Hämtning eller data . . . 37
5.5 LDIR och RET på `$7FFD` . . . 37
5.6 64 kilobyte RAM . . . 38
5.7 Externt minne . . . 39
5.8 MEM: . . . 39

II Programvaran 40

6 BASIC II i ABC 802 41
6.1 Samma tolk, ny översättning . . . 41
6.2 Det som syns . . . 41
6.3 `WIDTH` . . . 42
6.4 Det låga området . . . 42
6.5 Tillägget på `$7000` . . . 43
6.6 Adresser . . . 43

7 DOS och skivminnet 45
7.1 Samma DOS som i ABC 800 M . . . 45
7.2 Skivminnet på ABC-bussen . . . 45
7.3 Enheterna . . . 46
7.4 Hårddisken . . . 46
7.5 Starten och minnet . . . 48
7.6 ABC NET . . . 48

8 ABC NET och LUXNET 49
8.1 Nätet . . . 49
8.2 Nätets DOS-PROM . . . 49
8.3 NETHK frågar nätet . . . 50
8.4 Nodkortet . . . 51
8.5 Enheterna och satserna . . . 52
8.6 LIB och biblioteket . . . 53
8.7 Nätprogrammen . . . 53
8.8 Det som inte är känt . . . 54

9 CP/M på ABC 802 55
9.1 RAM från adress 0 . . . 55
9.2 MyAB:s CP/M Plus . . . 55
9.3 Laddaren i BASICINI.SYS . . . 57
9.4 BIOS:en . . . 58
9.5 Skivformatet . . . 58
9.6 Programmen . . . 59

10 Pascal på ABC 802 61
10.1 Ett språk från tiden . . . 61
10.2 Delmängden . . . 61
10.3 P-koden . . . 62
10.4 Tolken . . . 63
10.5 Flyttalen . . . 64
10.6 Kompilatorn . . . 64
10.7 Att kompilera sig själv . . . 65
10.8 Mot 64 kilobyte . . . 66

10.9 Pascal 800 66

III Kontoret **69**

11 Ord 800 och Kalkyl 800 **70**
11.1 Vad Luxor sålde 70
11.2 Ord 800 70
11.3 Kalkyl 800 71
11.4 Filerna emellan 72
11.5 Andra än Luxor 72

12 ORD och KALKYL **74**
12.1 Programskivan 74
12.2 ORD 74
12.3 Luckan på RAM-skivan 75
12.4 Skärmen 75
12.5 KALKYL 76
12.6 Formeln blir Forth 76
12.7 Ordningen 77
12.8 ROM:ens tal 77
12.9 Omgångarna 77
12.10 Minnet och överläggen 78
12.11 Filerna 78

13 VERK **80**
13.1 Brevet 80
13.2 Fälten i texten 81
13.3 Filen 81
13.4 Minnet 81
13.5 Varifrån VERK kommer 82
13.6 Vad som saknas 83

A Källor **84**

B Namnregister **93**

Sakregister **95**

Förord

Första gången jag mötte en ABC 802 var på kontoret där Olle Wolkert arbetade. Han var chef för OWOCO AB, som var mest känt för Smartaid till ABC80. Han hade en ABC 802 på ett skrivbord och en Apple Lisa på ett annat. Det var omkring 1985, och de två datorerna stod där som två olika föreställningar om framtidens arbete. Hos utvecklarna, i ett annat rum, stod en Macintosh.

Kanske var det ungefär så ABC 802 var tänkt att användas. Chefen skulle kunna följa börskurserna, koppla upp sig mot en stordator och se vad företaget sysslade med. Men vid den här tiden var det sällan chefen själv som skrev texterna eller gick igenom de ekonomiska detaljerna. Det arbetet gjorde oftast sekreterare och andra medarbetare. För chefen kunde datorn i stället vara något att visa upp: ett tecken på att kontoret var modernt och att företaget låg i framkant, som biltelefonen (NMT).

ABC 802 stod i hörnet av rummet, redo för de uppkopplade tjänster och upplysningar som chefen kunde tänkas behöva. Den behövde inte användas hela tiden. Det viktiga kunde lika gärna vara att den fanns där, synlig, avancerad och imponerande.

Tekniskt var ABC 802 ett tydligt steg upp från ABC80. Den hade en mer utvecklad arkitektur och var bättre lämpad för seriösa arbetsuppgifter. Men den saknade något av ABC80:s charm. ABC80 hade en direkthet och en lekfullhet som gjorde det lätt att se vad datorn kunde bli, också utanför arbetet. ABC 802 pekade snarare mot kontoret: mot ordbehandling, kalkyler, databaser och andra nyttiga program.

Det var också den sortens användning som skolorna väntades satsa på. Datorn skulle ge nytta och produktivitet, och eleverna skulle lära sig ekonomiska beräkningar, ordbehandling, databaser och kalkyl. Men det var inte alltid det som lockade dem mest. Många ville spela, göra musik eller utforska grafik. Skolans bild av datorn som arbetsredskap var en sak, elevernas lust att experimentera och ha roligt med den en annan.

ABC80 var på många sätt före sin tid. När den kom 1978 öppnade den dörren för en ny sorts datoranvändning i Sverige: tillräckligt enkel och tillgänglig för att väcka nyfikenhet, men samtidigt kapabel nog att kännas som ett riktigt arbetsredskap. Den lade också en grund att bygga vidare på: användare, erfarenheter och en svensk marknad som hade börjat vänja sig vid datorn.

Fem år senare kom ABC 802. I Sverige kunde den kännas som en naturlig fortsättning på något som redan hade fått fäste. Den passade väl in på dåtidens kontor, och priset gjorde att skolorna åtminstone kunde överväga den. Men fem år var lång tid i datorvärlden, och utanför Sverige hade utvecklingen gått snabbt. Utländska tillverkare kom med nya modeller, och både tekniken och förväntningarna förändrades. ABC 802 hade en historia och en publik hemma, men den mötte en omvärld som redan hade tagit nya steg. Den var modern nog för många svenska arbetsplatser och skolor, men mätt mot omvärlden kunde den inte framstå som lika självklar eller nyskapande som ABC80 en gång hade gjort.

ABC 802 var på många sätt en bättre dator än ABC80, men det var inte självklart att den skulle få samma plats i människors föreställningar. Den kom som ett kompetent arbetsredskap till ett landskap som redan höll på att ritas om. Kanske är det därför den känns så speciell i efterhand: en dator från en övergångstid, när datorn ännu kunde vara både en symbol för modernitet och ett verktyg vars framtida användning ingen riktigt hade bestämt.

Den här boken beskriver ABC 802 inifrån, på samma sätt som *ABC80 inifrån* beskriver ABC80: maskinen efter Luxors servicemanual och ritningarna, och programvaran efter koden i ROM:en. ABC 802 har i stort sett samma BASIC II som ABC 800, och den tolken finns redan beskriven i andra bandet av *ABC80 inifrån*. Här står i stället det som är ABC 802:s eget: bildskärmen med 40

och 80 tecken, tangentbordet med sin egen processor, minnet med RAM-skivan, DOS:en, nätet och CP/M. Boken går att läsa utan banden om ABC80.

Själv har jag nu en ABC 802, köpt begagnad långt efter sitt utgångsdatum. Fotografierna i boken är tagna av den, och tidsproven är körda på den.

Som för ABC80 har ROM:en disassemblerats och kommenterats i samarbete med Anthropics AI-modell Claude, i varianten Claude Code. Den här gången kunde mycket flyttas över. Ungefär två tredjedelar av ABC 802:s 32 kilobyte ROM finns också i BASIC II för ABC 800, om än på andra adresser, och namnen och kommentarerna därifrån kunde följa med. Det som är nytt är skärmen, tangentbordet, RAM-skivan, DOS:en och nätet. Modellen har också läst servicemanualen och ritningarna och jämfört dem med koden. Jag har styrt arbetet, ställt frågorna och prövat svaren.

Boken bygger på det som har gått att hitta, mest på nätet: Luxors handböcker och prislistor i arkivet på abc80.net, ritningarna på devili.iki.fi, inskannade tidskrifter och avbildningar av ROM:arna. Ett varmt tack till alla som har bevarat, skannat och delat med sig. Servicemanualen och ritningen stämmer inte alltid med varandra, och där de skiljer sig står båda.

Fel kan ändå finnas kvar. Boken är helt öppen för korrigeringar, tillägg och förbättringar. Skicka gärna dina förslag till `setlonnert@gmail.com`.

Det som hör till boken och går att köra eller söka i finns i ett kodförråd på webben: den kommenterade ROM-listningen, tidsproven och emulatorn:

`https://github.com/Feyerabend/ABC80/tree/master/abc802`

Uppsala, 2026

Set Lonnert

ABC 802 i sin tid

ABC80 kom 1978, när persondatorerna var nya. ABC 802 kom fem år senare till en helt annan marknad. Där fanns färgdatorer och hemdatorer med ljud för några tusen kronor, sexton bitars processorer och framför allt IBM PC. Kapitlet handlar mer om den omvärlden än om maskinen själv: när ABC 802 kom, vem den var gjord för och vad den mötte.

1.1 Våren 1983

Luxor pressvisade ABC 802 och ABC 806 i februari 1983 (Sjöström 1996, s. 123). Visningen hölls på Sheraton i Stockholm, och samtidigt höll de fyrtio största återförsäljarna egna presskonferenser på sina orter (s. 135). ABC-klubbens krönika förlägger de två maskinerna till våren 1983 (Kullmar 1984, s. 12).

Båda var vidareutvecklingar av ABC 800 och kallades ”lilla och stora ABC 800”. Det var Dataindustrier (DIAB) som gjorde arbetet, som de tidigare hade gjort ABC 800 av ABC80. Kontraktet för ABC 802 skrevs i juli 1982, kontraktet för ABC 806 först den 26 maj 1983, efter visningen (Sjöström 1996, s. 126). Avsikten var lätta uppdateringar av en lönsam serie (s. 125). Luxors produktblad kallar ABC 802 ”Lilla ABC 800”, och på ABC-klubbens årsmöte den 12 februari 1983 nämndes ”den nya 802 som är den som kallas lilla ABC-800” (ABC-bladet 1983 nr 2, s. 13).

Den största förändringen var lådorna. I ABC80 och ABC 800 satt datorkortet i tangentbordet. I ABC 802 satt det i bildskärmen, och tangentbordet blev ett lågt bord för sig, enligt Sjöström 30 millimeter tunt och med 77 tangenter (s. 126). De 30 millimetrarna finns i servicemanualen, men där är det avståndet från bordet till toppen av tangenterna i raden ASDFGHJKLÖÄ, enligt Svensk Standard SS 662241 (servicemanualen s. 5). Produktbladets tangentbord, ABC 55, hade 55 tangenter. 77 stämmer bara med ABC 77, som också har funktionstangenterna och sifferdelen (se s. 28), så det är det Sjöström beskriver. Processorn var densamma som 1978, en Z80 med åtta bitar (s. 126).

1.2 Vem den var till för

ABC 802 riktade sig till kontoret. Produktbladet kallar den ”perfekt som beslutsfattardator och terminaldator, där man behöver ett kraftfullt hjälpmedel som tar liten plats”. I datorn fanns rutiner för teledata och för att vara terminal mot andra datorer, och hårdvaran för nätet ABC NET ingick, medan programmet för nätet var ett tillval (Luxor, produktblad ABC 802).

I Luxors cirkaprislista nr 1-83 kostade ABC 802 utan tangentbord 7 600 kronor och tangentbordet ABC 55 2 150 kronor, tillsammans 9 750 kronor utan moms. Det stämmer med ”ett pris strax under 10 000 kr” i Mikrodatorns test (Falke 1983, s. 127). ABC 806 kostade 10 700 kronor utan tangentbord, och ABC 800 M 11 600 kronor. En NRZI-modul för ABC 802 kostade 840 kronor (Luxor, cirkaprislista 1-83).

Bengt Falke i Mikrodatorn provade ett av de första exemplaren. Han berömde tangentbordet och lådan men tyckte att skärpan var ojämn och att tecknen blev små med 80 tecken på en tiotumsskärm. Det saknades färg, ljudet var begränsat och grafiken ”inte alltför god”, men det tilltalade

nog ”andra kundkretsar” än dem ABC 802 riktade sig mot. Han såg den som en nästan idealisk skoldator och som terminal mot stordatorer, i skolorna och på arbetsplatserna (Falke 1983, s. 126–127). Luxor gav journalisterna tillgång till de första exemplaren och ”färdiga artiklar”, och Sjöström citerar Falkes sammanfattning (Sjöström 1996, s. 135–136).

Programmen följde med. ABC 802 hade i stort sett samma BASIC II som ABC 800, och BASIC-program från ABC80 gick för det mesta att köra om de var sparade i BAS-format (Falke 1983, s. 127). Gunnar Tidner gav samma råd i ABC-bladets ledare till den som tänkte byta en ABC80 mot till exempel en ABC 802: spara programmen i BAS-format medan tid är (ABC-bladet 1983 nr 2, s. 2). Han trodde att ”den lilla ABC 802” hade stora förutsättningar att bli populär.

1.3 Omvärlden

Samma år kom IBM PC och Ericsson PC ut på den svenska marknaden (Sjöström 1996, s. 123). Den 18 januari annonserade IBM att PC:n släpptes för försäljning i Sverige och de flesta andra länderna i Europa. Båda var ”16-bitars” persondatorer och kostade omkring 30 000 kronor, och en hel arbetsplats med skrivare kostade under 50 000 kronor (s. 129). IBM PC hade en Intel 8088 på 4,77 MHz, 40 kilobyte ROM med BASIC och 16 till 64 kilobyte RAM på kortet, och färggrafikkortet gav 320 × 200 bildpunkter i fyra färger (Williams 1982, s. 36–45). Enligt Sjöström hade ”i stort sett alla konkurrenter” gått över till sexton bitar, medan ABC 802 och ABC 806 hade kvar åtta (Sjöström 1996, s. 126).

IBM ändrade hur marknaden såg på ABC-datorerna. Intresset för ABC 802 och ABC 806 var inte översvallande hos återförsäljare, press och köpare. Luxor blev över en natt andra märke hos kedjan TEAM 100, och det gick ett rykte att Luxor skulle få svårt att överleva, bland annat för att ABC-datorerna inte var IBM-kompatibla (s. 123). Tester visade att ABC 802 och ABC 806 var lika snabba som de nyare maskinerna vid ordbehandling men mycket långsammare vid kalkylering (Sjöström 1996, band 1, s. 137).

Skolan fick en egen dator. Compis presenterades den 22 september 1983. Den byggde på Intels iAPX 186, en processor med ett femtontal kringkretsar i samma krets, och priset i grundutförande blev omkring 9 000 kronor i stället för de planerade 6 000. Teli tillverkade och Esselte sålde (Sjöström 1996, s. 124). Jet 80 från CAT, med Z80 och CP/M 3.0, riktade sig också till skolorna och räknade med konkurrens ”främst” från ABC 802 (s. 132).

Hemdatorerna var en marknad för sig. Redan 1982 kostade en VIC-20 med svartvit skärm och bandspelare omkring 4 000 kronor i Sverige (s. 103). Commodore 64 hade 64 kilobyte minne, bildkretsen VIC-II, som såg 16 kilobyte åt gången (Commodore 1982, s. 102), och en ljudkrets med tre röster, ADSR-kurvor, filter och brus (s. 184). Herbert Söderström skrev i ABC-bladet att entusiasterna på ZX 80 och 81, Spectrum, Atari och VIC-20 hade hans respekt, men att de maskinerna inte räckte till för att lära ut ordbehandling och kalkyl i en kontorsmiljö. Ungefär hälften av alla som arbetade professionellt med mikrodatorer gjorde det på ABC80 eller ABC 800 (ABC-bladet 1984 nr 1, s. 5).

1.4 Samma kretsar på andra håll

ABC 802 var gammal i processorn men inte i kretsarna runt den. Bildkretsen 6845 (se s. 18) satt också på IBM PC:s färggrafikkort, där den nåddes på portarna `$3D4` och `$3D5` (Williams 1982, s. 39–40), och i BBC-datorn (Bray, Dickens och Holmes 1983, s. 359). Där ABC80 hade en PIO och läste kassetten bit för bit i programmet, hade ABC 802 Zilogs egna kretsar för serieportarna och räknarna, SIO, DART och CTC, och en processor i tangentbordet (se s. 15). Minnet räckte till 64 kilobyte, men bara 32 av dem var arbetsminne för BASIC; resten var en RAM-skiva, MEM:, eller plats för CP/M (se s. 35).

1.5 Hur det gick

Försäljningen tog fart. I maj och juni 1983 sålde Luxor omkring 800 ABC 802 och ABC 806 i månaden, nästan tre gånger så många som de omkring 350 ABC 800 i månaden tidigare (Sjöström 1996, s. 135). Men maskinerna blev dyra att tillverka. Inköpsavdelningen köpte kretsar för ett år i taget, och under tiden föll priset på dynamiska RAM från 60 till 8 kronor styck, vilket kostade mellan 5 och 10 miljoner kronor i vinst (s. 125).

I skolan höll ABC-datorerna sin ställning. Skolöverstyrelsen godkände nio leverantörer för statsbidrag till högstadiet, och Apple och IBM föll bort; Luxor, Jet, Compis och Ericssons EPC kom med. Luxor hade omkring 70 procent av skolmarknaden, och för att klara Skolöverstyrelsens framtida krav skulle ABC 802 uppdateras (s. 162). År 1984 sålde Luxor 2 500 ABC80 och 8 500 ABC 800, ABC 802 och ABC 806 inräknade, drygt hälften av den svenska marknaden för sådana datorer enligt Luxors egna undersökningar (s. 166). Samma år köpte Nokia Luxor, och IBM blev marknadsledare (s. 165).

Del I

Maskinen

Maskinen

Där inget annat anges kommer uppgifterna i det här kapitlet från Luxors servicemanual för ABC 802 (ABC 802 Servicemanual), och sidnumren inom parentes avser den. Bildskärmen, tangentbordet och minnet har egna kapitel; här beskrivs maskinen som helhet.

2.1 Datorn i bildskärmen

Luxor kallade ABC 802 den ”minsta” datorn i ABC 800-serien. Den består av två enheter: en bildskärm som innehåller själva datorn och ett lågt tangentbord med en egen mikroprocessor, som avsöker tangenterna och sköter förbindelsen med datorn. Förbindelsen går åt båda hållen, så att programmen kan styra vissa saker i tangentbordet. Raderna kan ha från 1 till 80 tecken. Med högst 40 tecken per rad går bilden över till förstorade tecken och blir som i ABC80 och ABC 800 C, efter text-tv:s modell. I blockschemat är ABC 802 byggd som en ABC 800 med svartvit skärm. Skivminnena styrs av ett DOS av typen UFD, med flera biblioteksnivåer, och maskinvaran för nätet ABC NET är inbyggd (s. 4).

Konstruktörerna lade vikt vid låg temperatur i lådan. Bildminnet och dess buffertar är snabba CMOS-kretsar som drar lite ström, RAM:et är dynamiskt och de övriga kretsarna är TTL av typerna LS och ALS. Nätdelen sitter baktill i bildskärmen. Luften går in framtill på undersidan, passerar över och under datorkortet och runt nätdelen och går ut upptill på baksidan. Aluminiumkåpan leder bort värmen och skärmar av strålningen från datorn. Tangentbordet följer svensk standard SS 662241, och från bordet till ovansidan av tangenterna i mittraden är det 30 millimeter. Bildskärmen kan vridas och lutas mot tangentbordet och är behandlad mot reflexer (s. 4–5).

Luxors tekniska data sammanfattar maskinen (s. 6–8):

Processor	Z80A, 3 MHz
ROM	24 KB BASIC II, 4 KB DOS, 4 KB skrivar- och terminalrutiner
RAM	64 KB: 32 KB arbetsminne, 32 KB RAM-skiva; 2 KB bildminne
Bildskärm	24 cm (10 tum), gul fosfor (L5C), 24 rader med 80 eller 40 tecken
Tecken	cell 10x6 punkter, matris 9x5, 96 ISO-tecken, 64 grafiktecken som i text-tv
Kassett	700 eller 2400 baud
CH A	asynkron, 75-19200 baud, för skrivare
CH B	synkron eller asynkron, 75-19200 baud, för kommunikation
Ljud	programstyrd summer
Nätdel	primärswitchad, 230 V (115 V med bygel), högst 60 W
Vikt	8 kg

Datorn kan också ställas om så att hela RAM:et laddas med program, för andra operativsystem som CP/M (s. 6). Hur det går till står i kapitlet Minnet (se s. 35).



ABC 802 med tangentbordet ABC 77. Bredvid ligger BASIC II-boken för ABC 802 av Jan Lundgren och Sören Thornell.

2.2 Datorkortet

Elektroniken sitter på två kort. Datorkortet (Luxor 55 20892-01) bär processorn, minnet, bildkretsarna och de seriella kretsarna. Ovanpå det, i ena hörnet, sitter kommunikationsmodulen (Luxor 55 20893-01), ett mindre kort med kontakterna CH A och CH B och summern. Modulen beskrivs i avsnittet Kommunikationsmodulen (s. 12).

Datorkortet ligger i bildskärmen och dras ut bakåt, med modulen på. Komponentlistan tar upp en styrskena och ett undre bakstycke till kortet (s. 30).

Kortet mäter 200 × 233 millimeter och har fyra lager (s. 6, 9). Kretsarnas lägen anges med en bokstav och ett tal, och komponentlistan ger de viktigaste (s. 29):

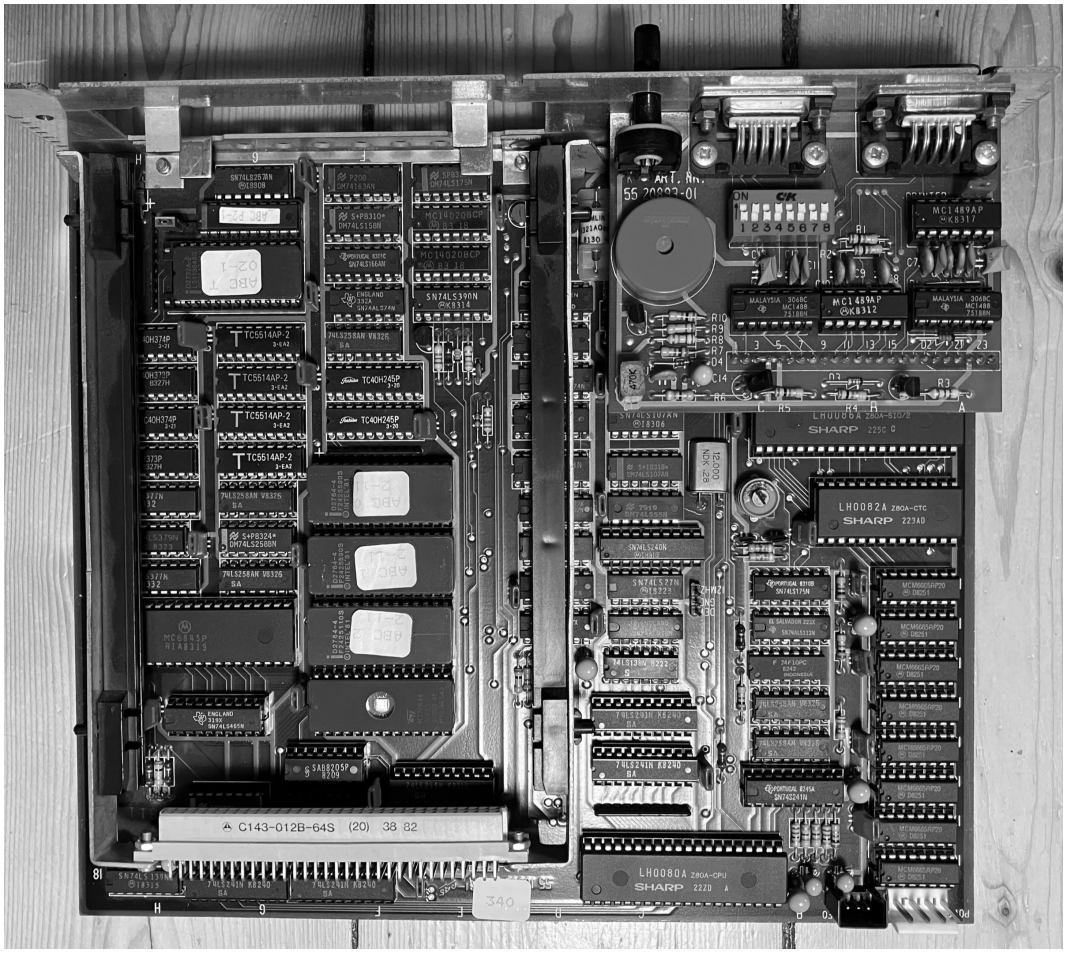
Läge	Krets	Uppgift
18C	Z80A	processorn
9A	Z80A CTC	klockorna till de seriella kretsarna och realtidsklockan
7A	Z80A SIO/2	kanal A till kassetten, kanal B till CH B
5A	Z80A DART	kanal A till CH A, kanal B till tangentbordet
12G	MC6845P	bildkretsen
9F, 11F, 12F, 14F	ABC02-1, ABC12-1, ABC22-1, ABC32-1	ROM:en, fyra kretsar om 8 KB
11A–18A	8 × HM 4864-3	RAM:et, 64 KB dynamiskt
5G–8G	4 × TC5514AP-2	bildminnet, 2 KB statiskt
3G	ABCT2-1	teckengeneratorn
2G	ABCP2-1	PAL-krets för bilden
13C	74LS138	väljer intern I/O-krets
15F, 16F	2 × 8205	strobarna till ABC-bussen
1F	74LS163	delar ned systemklockan



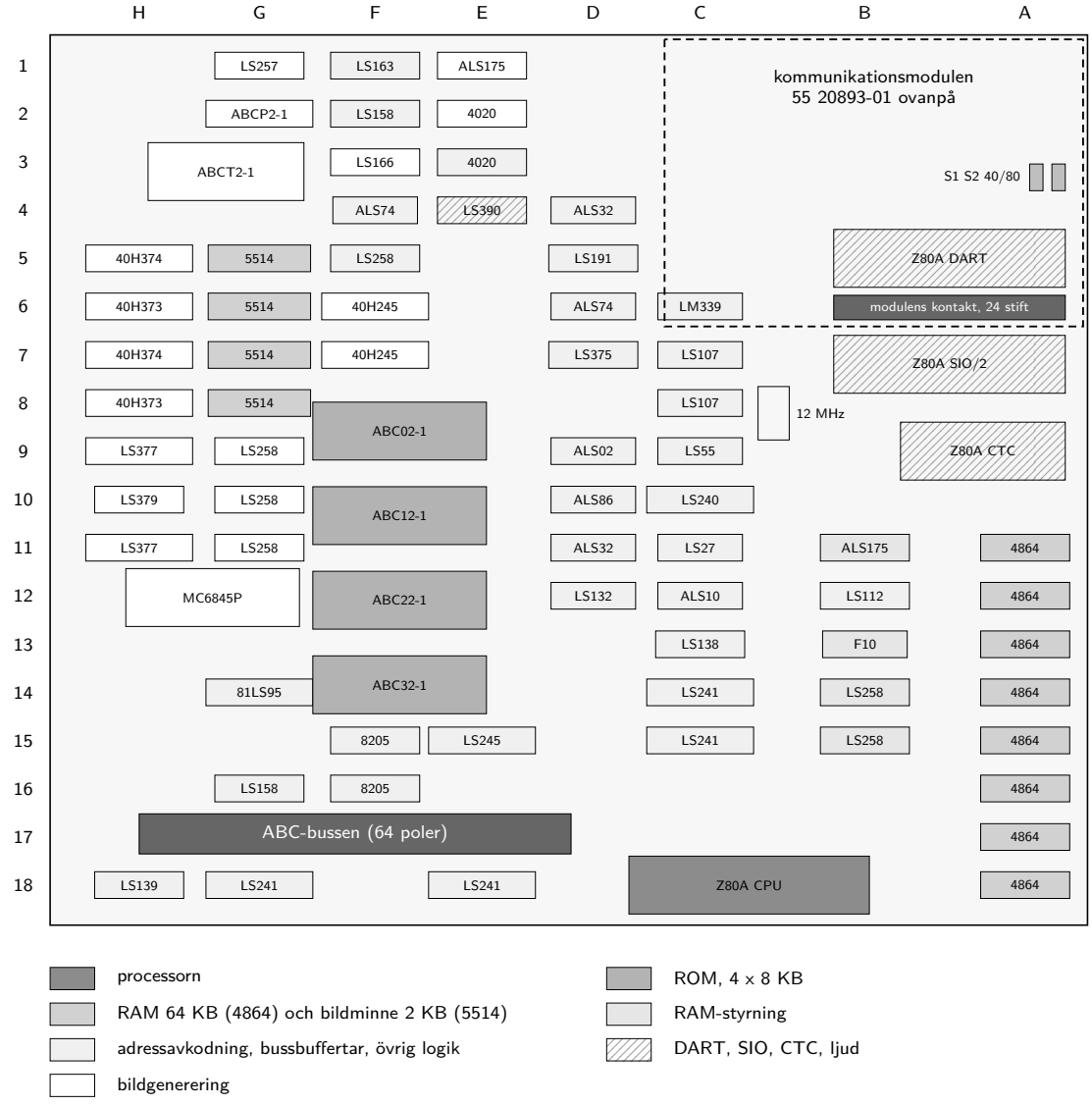
Datorkortet en bit utdraget ur bildskärmens baksida. Plåten med öppningen märkt ABC-BUS följer med kortet, och längst ned till höger skymtar kortets kant. Ovanför sitter typskylten (ABC 802, Part No. 230 8088-15).

Processorn styr databussen genom adress- och styrbussen. Adressbussen går genom en buffert ut till ABC-bussens kontakt och genom en annan till RAM:et, ROM:en och bildkretsen. Databussen når kontakten genom en buffert som kan vända riktning. ROM:en läses genom en egen buffert, medan RAM:et sitter direkt på databussen (s. 9).

Tangentbordet är en dator för sig, med en egen processor och ett eget program. Det beskrivs i kapitlet Tangentbordet (se s. 28).



Datorkortet uppifrån. Kommunikationsmodulen är det mindre kortet uppe till höger, med de två 9-poliga kontakterna, summern och DIP-omkopplaren; allt annat sitter på datorkortet. Kolumnernas bokstäver H–A står längs kortets kanter, H till vänster, och raderna räknas uppifrån, med 18 nere till vänster. RAM-kretsarna längst till höger är på det här kortet Motorolas MCM6665AP20, inte HM4864-3 som i komponentlistan.



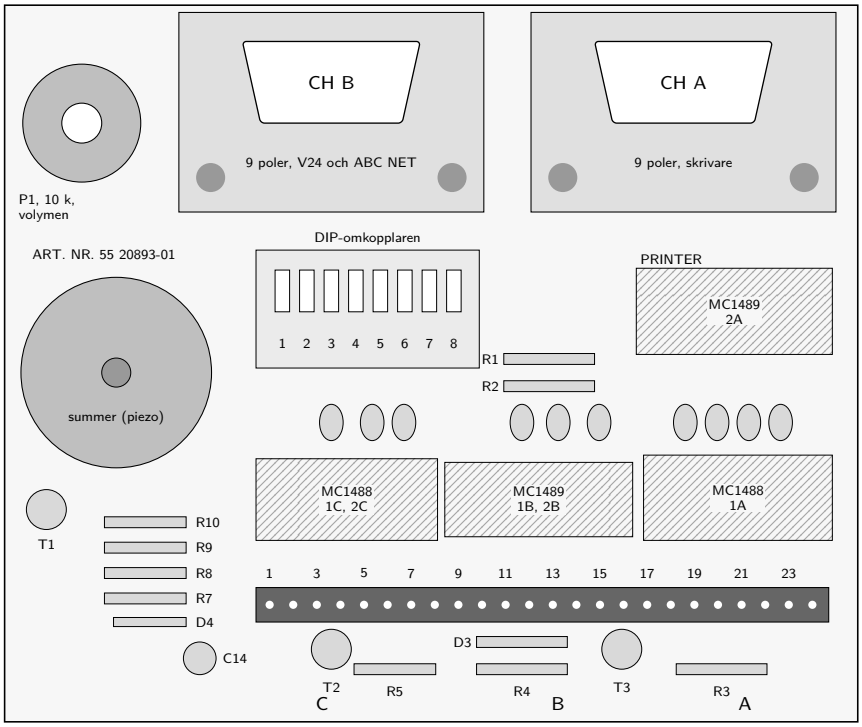
Datorkortet (Luxor 55 20892-01), skiss efter komponentlistan i servicemanualen (s. 29) och foto. Kommunikationsmodulen är bara utritad som en streckad ruta där den ligger; den har en egen skiss i avsnittet Kommunikationsmodulen. Under den är datorkortets kretsar kontrollerade genom inspektion. Där sitter också kontakten med 24 stift som modulen sticks i och byglarna S1 och S2 vid märkningen 40/80. DART:en på 5A är skymd. Gråtonen visar vad kretsen gör, efter kapitlen Minnet och Bildskärmen; kretsar som inte beskrivs där står som övrig logik. Manualens lägen 13B, 14B, 15B och 18H stämmer med kortet, där ritningen har 3B, 4B, 5B och 8H.

2.3 Kommunikationsmodulen

Kommunikationsmodulen (Luxor 55 20893-01) är ett eget litet kort, som sitter ovanpå datorkortet och sticks i en kontakt med 24 stift på det. Datorkortet har kretsarna som sänder och tar emot, DART:en, SIO:n och CTC:n; modulen har det som behövs mellan dem och omvärlden. Där sitter de två 9-poliga kontakterna CH A och CH B, två drivsteg MC1488 och två mottagare MC1489, som omvandlar mellan kretsarnas signaler och kontakternas V24-nivåer, DIP-omkopplaren för CH B och summern med potentiometern P1 för styrkan (s. 30–31, 99).

Omkopplaren har åtta lägen. I schemat är grundläget, för asynkron överföring, att 2, 4, 6 och 8 är till och de övriga från (s. 99).

Summern drivs av en transistor, och styrkan ställs med P1 på 10 kiloohm, vars axel är förlängd ut till ratten VOLUME på baksidan (s. 20, 30–31, 99). Summern slås om när port 5 läses, som i ABC 800 (se s. 32).



Kommunikationsmodulen (Luxor 55 20893-01), skiss efter fotot av datorkortet och modulens schema och komponentlista i servicemanualen (s. 30–31, 99). Bokstäverna C, B och A står på modulen. Schemat delar grindarna på sex platser, 1A–1C och 2A–2C, men på modulen sitter fyra kretsar. Grindarnas ben visar hur de hör ihop. Drivsteget 1A använder alla fyra grindarna, för CTC 1 och kanal B:s TXD, DTR och RTS. 1C har grind 1 och 4 och 2C grind 2 och 3, så de är en och samma 1488, med CTC 0 och kanal A:s TXD, DTR och RTS. Bland mottagarna har 1B grind 2 och 3 och 2B grind 1 och 4, och 2A har grind 2–4. Den enda grind som blir över, grind 1 för kanal A:s RXD, är märkt 1A, och den hör alltså till 2A; märket 1A vid en 1489 är troligen ett tryckfel. I kolumn A sitter därmed 1488:an 1A och 1489:an 2A, i C en 1488 och i B en 1489 (s. 99).

2.4 Klockorna

All takt kommer från en kristall på 12 megahertz. En räknare (1F) räknar 13, 14, 15, 0, 1 och 2 och börjar sedan om, så att den delar med sex. Räknarens första utgång ger 6 megahertz, som delas med två till processorns 3 megahertz, och en annan ger 2 megahertz till bildkretsen, en takt per tecken. Under senare delen av varje sådan period läggs adressen ut till bildminnet, teckengeneratorn läses och skiftregistret laddas (s. 76). Ett tecken är sex punkter brett (s. 7), så punkterna skiftas ut i kristallens 12 megahertz.

CTC:ns kanaler 0–2 räknar på en klocka om 1,5 megahertz, eller på processorns 3 megahertz, och ger sändnings- och mottagningsklockorna till SIO:n och DART:en. Kanal 3 får samma klocka som SIO:ns kanal A. Vid start ställs kanal 3 in så att den ger ett avbrott var 10,6:e millisekund, och avbrottsrutinen räknar upp sekunder, minuter, timmar och datum till `TIME$` (s. 54).

2.5 Minneskartan

ROM:en fyller de nedre 32 kilobyten, `$0000`–`$7FFF`, i fyra block om 8 kilobyte (s. 9). BASIC II tar de tre första. Det fjärde har DOS:en på `$6000`, skrivar- och terminalrutinerna på `$7000` och kassettrutinerna och MEM: på `$7800`. Parallellt med de sista 2 kilobyten ligger bildminnet. RAM:et är två block om 32 kilobyte. Det övre, `$8000`–`$FFFF`, är arbetsminnet, och högst upp ligger kassettbuffertarna och BASIC:ens enkla variabler. Det nedre ligger parallellt med ROM:en och används som RAM-skiva. Områdena går inte in i varandra, eftersom maskinen går över i ett särskilt läge när kassettrutinerna och RAM-skivan arbetar. För CP/M används hela RAM:et (s. 18, 46). Hur växlingen går till står i kapitlet Minnet (se s. 35).

Adress	Vanligt läge	Parallellt
`$FF00`–`$FFFF`	enkla variabler	
`$FD00`–`$FEFF`	kassettbuffertarna CASBUF 1 och 2	
`$8000`–`$FCFF`	RAM, arbetsminnet	
`$7800`–`$7FFF`	bildminnet, 2 KB	ROM: kassetten, MEM:; RAM-skivan
`$7000`–`$77FF`	ROM: skrivaren och terminalen	RAM-skivan
`$6000`–`$6FFF`	ROM: DOS	RAM-skivan
`$0000`–`$5FFF`	ROM: BASIC II, 24 KB	RAM-skivan

Servicemanualens karta har ett fel: `$7000` står som 28678, men det är 28672. Den som vill reservera plats för egna rutiner i maskinkod ändrar pekarna BOTTOM på adress 65292 och TOP på 65294 (s. 18).

2.6 Portkartan

En I/O-instruktion lägger ut en adress mellan 0 och 255 på adressbitarna A0–A7. Adresserna 0–31 och 128–255 ger strobar ut på ABC-bussen, medan 32–127 är reserverade för de inbyggda kretsarna och inte syns på bussen. När A7 är 0 väljer A4–A6 en av utgångarna på 13C: utgång 2 ger DART:en, 3 bildkretsen, 4 SIO:n och 6 CTC:n. Utgång 0 släpper fram de två 8205-kretsarna, som av de tre lägsta adressbitarna gör ABC-bussens strobar `OUT` 0–7 och `INP` 0–2. `OUT` 0–5 och `INP` 0–1 är anpassade till korten i Databoard 4680-systemet. `INP` 7 strobar inte bussen utan återställer den (s. 42–43).

Port	Krets
`$00`–`$07`	ABC-bussen: `OUT` 0–5, `INP` 0–2; `INP` 7 återställer bussen
`$08`–`$1F`	ABC-bussen, bara de allmänna strobarna
`$20`–`$2F`	DART
`$30`–`$3F`	bildkretsen (6845)
`$40`–`$4F`	SIO/2
`$60`–`$63`	CTC, kanal 0–3
`$80`–`$FF`	ABC-bussen, bara de allmänna strobarna

CTC:ns kanaler ligger på portarna 96–99 (s. 54). ROM:en använder bara de första portarna i de andra områdena: `$20`–`$23` för DART:ens data- och styrregister i kanal A och B, `$40`–`$43` på samma sätt för SIO:n och `$38`, `$39` och `$31` för bildkretsen.

2.7 Avbrott, NMI och återställning

CTC:n, SIO:n och DART:en ger avbrott, och korten på ABC-bussen kan också göra det. Kretsarna sitter i en prioritetskedja: CTC:n först, sedan SIO:n, DART:en och sist korten på bussen. NMI kan bara komma från bussen. Då hämtar processorn nästa instruktion på `$0066`, och där står ett hopp till RAM:et, som leder till att systemet återställs (s. 45). I ROM:en är hoppet `JP $FF8A`.

Återställningen kommer vid tillslag, från bussens ledning RES IN och från knappen på bildskärmens baksida eller tangentbordets. Signalen går till processorn, CTC:n, SIO:n, DART:en och bussen. En räknare håller den kvar i minst åtta pulser på en långsam klocka med omkring 16 millisekunder mellan pulserna, och ut på bussen varar den minst 5,28 mikrosekunder längre. Ett program kan återställa korten på bussen för sig med `INP` 7; då blir pulsen omkring 5,9 mikrosekunder (s. 44–45).

2.8 Anslutningarna

Alla kontakter sitter på baksidan. På framsidan finns bara en lysdiod som visar att strömmen är på (s. 20).

ABC-bussen100mm

Kontakt	Typ	Stift
CH A	9-polig D-sub	1 DTR, 2 TxD, 3 RxD, 4 RTS, 5 CTS, 6 DSR (+12 V), 7 jord, 8 DCD, 9 −12 V
CH B	9-polig D-sub	som CH A; stift 6 och 9 efter omkopplaren på kommunikationsmodulen
tangentbordet	7-polig DIN	1 TxD, 2 jord, 3 RxD, 4 TRxC (klocka), 5 Key Down*, 6 +12 V, 7 Reset*
kassetten	5-polig DIN	1 TxD, 2 jord, 3 RxD, 4–5 motorstyrningen
video	6-polig DIN	1 kopplingsspänning, 2 kompositvideo, 3 jord, 4 och 6 ljud, 5 oanvänd
ABC-bussen	64-polig Europakontakt	en ledig kortplats

CH A är avsedd för en skrivare. CH B är asynkron men kan kopplas om för synkron överföring, och den är också avsedd för ABC NET. Omkopplaren på kommunikationsmodulen bestämmer hur kanalen används. Videokontakten ger kompositvideo och ljud till en vanlig tv-mottagare (s. 7, 20–21). Den tekniska beskrivningen säger på ett ställe att tangentbordet ansluts med en sexpolig DIN-kontakt (s. 10), men tekniska data och kontaktbeskrivningen säger sjupolig (s. 7, 21).



Baksidan. Upptill ratten för ljusstyrkan, strömbrytaren, nätkontakten (230 V, 50–60 Hz, 60 W) och typskylten (ABC 802, Part No. 230 8088-15). Nedtill CH. A och CH. B, ratten VOLUME, knappen RESET, kontakterna KEYBOARD, VIDEO och TAPE och den lediga kortplatsen på ABC-bussen.

2.9 Jämfört med ABC80

ABC80 kom 1978, fem år före ABC 802, och båda bygger på en Z80A på 3 megahertz och ABC-bussen. Uppgifterna om ABC80 i det här avsnittet kommer från Gunnar Markesjös *Mikrodatorns ABC* (Markesjö 1978), som beskriver ABC80:s elektronik krets för krets, och sidnumren efter ”M.” avser den. Uppgifterna om ABC 802 kommer från servicemanualen och de andra kapitlen i den här boken. Mer om ABC80 står i *ABC80 inifrån*, band 1, kapitlet ABC80 i standardutförande.

	ABC80	**ABC 802**
låda	datordelen med tangentbordet, ström från bildskärmen	datorn i bildskärmen, eget tangentbord
kristall	11,9808 MHz (M. 145–146)	12 MHz (s. 76)
ROM	16 KB, 4 kretsar om 4 KB (M. 99)	32 KB, 4 kretsar 2764
RAM	16 KB, 8 × 4116 (M. 100–101)	64 KB, 8 × HM4864-3
uppfriskning	Z80:ns R-register (M. 104)	Z80:ns R-register
bildminne	1 KB statiskt, 2 × 1024 × 4 bitar, `$7C00` (M. 100–101)	2 KB statiskt CMOS, 4 × TC5514AP-2, `$7800`
avkodning	bipolära PROM 7611 (M. 101, 105, 107)	TTL-kretsar (74LS139, 74LS55)
bilden	egen räknarkedja, 40 × 24 tecken (M. 143, 145–146)	6845, 40 eller 80 × 24 tecken
markören	bit 7 i tecknet, blinkar inverterat (M. 154)	bildkretsens markör, ett streck under tecknet
tangentbordet	Key Tronic, kapacitivt, 7 bitar och strobe parallellt till PIO A (M. 135–141)	egen processor 8035/8048, seriellt med 600 bit/s till DART:en
ljudet	SN76477 på utport 6 (M. 105–107, 112–114)	en summer som slår om vid varje läsning av port 5

	ABC80	**ABC 802**
kassetten	PIO B, bit för bit i programmet (M. 160–167); 700 baud enligt Luxor	SIO/2 kanal A, 700 eller 2400 baud
V24	fem bitar i PIO B, bit för bit i programmet (M. 93, 122–123)	två kanaler, CH A (DART) och CH B (SIO/2), 75–19 200 baud
klockan	NMI från bildräknaren, 50 gånger i sekunden; ROM:ens rutin på `$0066` räknar	CTC:ns kanal 3, avbrott var 10,6:e ms (s. 54)

Det största steget är att ABC 802 har gett varje uppgift en egen krets. I ABC80 sköter programmet kassetten och V24 bit för bit genom PIO:ns port B, och tiden räknas i slingor (M. 122–123, 160–167). I ABC 802 tar SIO:n och DART:en emot och sänder hela tecken, och CTC:n ger dem takten, så processorn behöver bara ta hand om en byte i taget. Samma sak gäller bilden: ABC80:s skärm är en räknarkedja av TTL-kretsar byggd för text-tv:s 40 × 24, medan ABC 802 har en bildkrets, 6845:an, vars register ROM:en ställer in för 40 eller 80 tecken.

Tangentbordet har tagit samma väg. I ABC80 lämnar tangentbordets kodare en färdig teckenkod parallellt, och strobebiten ger ett avbrott (M. 122, 141). ABC 802:s tangentbord har en egen processor och en seriell förbindelse åt båda hållen, så att datorn kan skicka kommandon tillbaka till tangentbordet (se kapitlet Tangentbordet, s. 28).

Ljudet är det enda som har blivit enklare. ABC80:s SN76477 kan ge sirener, brus och avklingande toner med ett enda `OUT` (M. 112–114). I ABC 802 finns bara en summer, och tonen görs av programmet, som läser port 5 i en slinga.

Bildminnet delas på liknande sätt i båda. I ABC80 håller låskretsar kvar den byte som bilden just har läst medan processorn går in i bildminnet, och processorn får ingen väntecykel (M. 152, 158). I ABC 802 får processorn också bildminnet genast, utan väntesignal, medan låskretsarna 6H och 8H håller de senaste tecknen (se kapitlet Bildskärmen, s. 17). Om det ändå kan bli fel tecken i bilden står inte i källorna; frågan är öppen och tas upp i kapitlet Bildskärmen.

Minnet har vuxit från 16 till 64 kilobyte RAM, men adressrymden är fortfarande 64 kilobyte. ABC80 hade plats för ROM, bildminne och RAM bredvid varandra. I ABC 802 delar RAM:et den nedre halvan med ROM:en och bildminnet, och vilket som syns styrs av vad processorn gör (se kapitlet Minnet, s. 35). Båda låter Z80:ns R-register sköta uppfriskningen av det dynamiska minnet (M. 104).

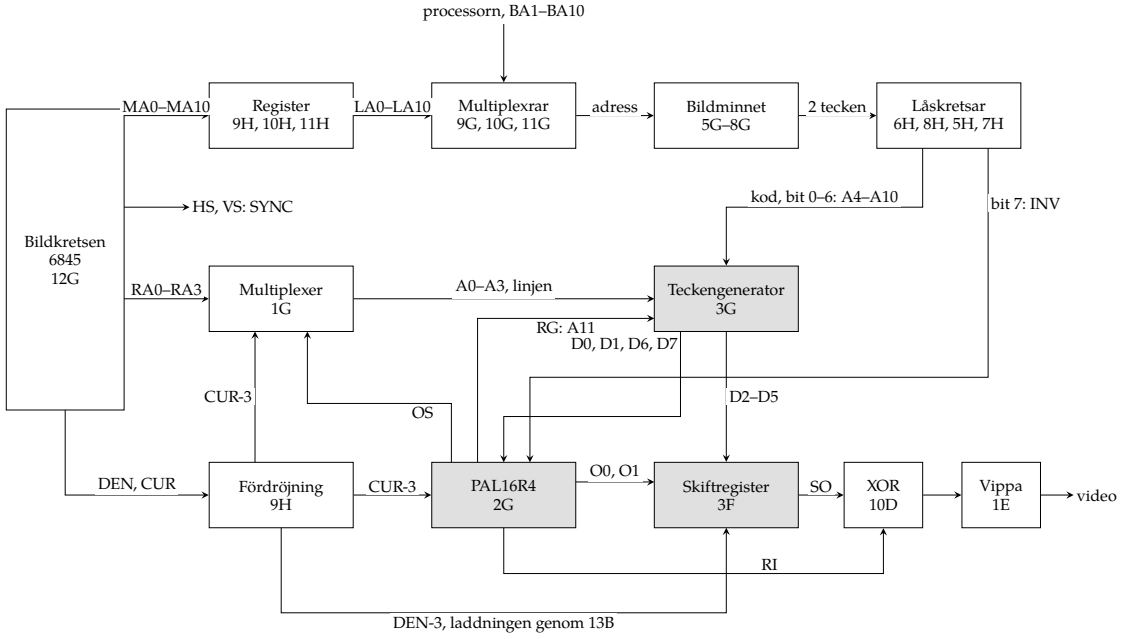
Bildskärmen

Där inget annat anges kommer uppgifterna i det här kapitlet från Luxors servicemanual för ABC 802 (ABC 802 Servicemanual), och sidnumren inom parentes avser den. Det som sägs om rutinerna är läst ur ROM:en.

3.1 Bilden

Bilden har 24 rader med 80 eller 40 tecken, men den kan få en rad till (avsnittet 25 rader, s. 21). Varje tecken står i en cell om 6 × 10 punkter, så att hela bilden blir 480 × 240 punkter i 80 teckens läge. Skärmen har gul fosfor, och bilden går till den som standardvideo: synkpulserna läggs ihop i en grind och två transistorer gör av dem och punkterna en kompositsignal på 1 volt, som också finns på stift 2 i den sexpoliga videokontakten på baksidan (s. 66).

Fem delar gör bilden. Bildkretsen 6845 räknar fram adresserna, bildminnet håller tecknen, teckengeneratorn ger punkterna, en PAL-krets sköter attributen och ett skiftregister lägger ut punkterna i takt med en klocka på 12 MHz. Var och en har ett avsnitt här, och figuren visar hur de hänger ihop.



Vägen från bildkretsen till videosignalen, med kretsarnas lägen. Adressen går genom register och multiplexrar till bildminnet, koden till teckengeneratorn och bit 7 till PAL-kretsen. Punkterna går genom skiftregistret, grinden som inverterar och en vippa till videon. Ur ritningen, blad 2 och 5, och servicemanualen s. 76–80.

3.2 Bildkretsen 6845

Bildkretsen är en Motorola MC6845 på läge 12G (ritningen, blad 2). Enligt Motorolas datablad är den en uppsättning räknare som styrs av 18 register. Den räknar teckentider längs en linje, linjer i en rad och rader i en bild, ger den horisontella och den vertikala synkpulsen, HS och VS, och lägger ut två adresser: MA0–MA13, platsen i bildminnet, och RA0–RA4, linjen i tecknet. Den ger också DEN, som är hög när bilden är synlig, och CUR, som är hög där markören ska vara. Kretsen klarar en teckenklocka på högst 2,5 MHz (MC6845, Motorola). I ABC 802 används MA0–MA10 och RA0–RA3; resten är inte inkopplade (ritningen, blad 2).

Kretsen har två portar: på port 56 (`$38`) skrivs numret på ett register och på port 57 (`$39`) värdet; port 49 (`$31`) läser tillbaka. Adressbiten A3 avgör genom en grind om kretsen läses eller skrivs (s. 66).

Klockorna

Alla klockor kommer från en kristall på 12 MHz. En räknare, 74LS161 på 1F, laddas med 13 och räknar 13, 14, 15, 0, 1 och 2, sex steg, innan den laddas igen. Utgången QA växlar i varje steg och ger 6 MHz, som JK-vippan på 12B delar till processorns klocka. QC är hög i tre steg och låg i tre och ger 2 MHz, teckenklockan CCLK till bildkretsen (s. 76). En teckentid är alltså en halv mikrosekund, sex punkter på 12 MHz. Mot slutet av varje teckentid är signalerna RC och RC* aktiva, och då läggs adressen ut till bildminnet, teckengeneratorn läses och skiftregistret laddas (s. 76).

Registren

BASIC:en ställer registren vid start från tabellen `CRTTAB` på `$0463`. Med 50 Hz blir värdena:

Register	Värde	Betydelse
R0	127	128 teckentider per linje, 64 µs
R1	80	80 synliga tecken per rad
R2	99	horisontell synk efter tecken 99
R3	9	horisontella synkpulsens längd, 9 teckentider
R4	30	31 rader per bild
R5	4	4 linjer extra, 314 linjer i allt
R6	24	24 synliga rader
R7	27	vertikal synk efter rad 27
R8	0	utan radsprång
R9	9	10 linjer per rad
R10	$29	markören av, från linje 9
R11	9	markören till linje 9
R12–R13	$7800	bildens början i bildminnet

En bild tar 314 linjer om 64 mikrosekunder, och det ger en bildtakt på 49,76 Hz. För 60 Hz finns tabellen `CRTT60` på `$0471`, där R4–R7 är 25, 2, 24 och 25: 262 linjer och 59,6 Hz. Vilken tabell som används bestäms av en bygel för 50 eller 60 Hz, som läses på ingången CTS i DART:ens kanal B. De två sista registren, R14 och R15, är markörens adress och sätts av utskriftsrutinerna. R16 och R17 håller en adress från en ljuspenna; ingången LP, ben 3, har på ritningen ett motstånd om 10 kilohm, R45, till jord och går till mittbenet på en bygel med tre ben, där ett av de andra benen är +5 volt och det tredje är märkt PR (ritningen, blad 2). Vad bygeln är till för står varken i manualen eller på ritningen, och ingen annan källa har hittats. Den förblir öppen.

Markören och synkpulsen

R10 har två uppgifter. Bit 0–4 är linjen där markören börjar, och bit 5–6 väljer läge: 00 är fast markör, 01 ingen markör, 10 blinkande med en sextondel av bildtakten och 11 med en trettiotvåondel. $29 är alltså ingen markör och $09 fast markör, båda från linje 9. R11 är linjen där markören slutar. CUR är hög bara på linjerna från R10 till R11 (MC6845, Motorola), och med värdena från `CRTTAB` är det bara linje 9, den tomma linjen under tecknet.

Den vertikala synkpulsen har i Motorolas krets en fast längd på 16 linjer; R3 anger bara den horisontella, i fyra bitar (MC6845, Motorola). Pulserna läggs ihop i två exklusiva eller-grindar, 74LS86 på 10D. Den ena har sin andra ingång på plus och vänder VS till VS*, och den andra ger SYNC som HS XOR VS*. Utanför den vertikala pulsen är SYNC alltså den omvända horisontella pulsen, och under den vänds de horisontella pulserna. Flankerna kommer alltså på samma ställen genom hela bilden, bara med omvänd riktning, och bildskärmen har något att hålla linjetakten på även under den vertikala pulsen, ungefär som de inskurna pulserna i TV-normens vertikala synk. SYNC går till transistorerna T1 och T2, BC547, som tillsammans med punkterna gör kompositsignalen (ritningen, blad 2).

Processorn och bilden

Databladet beskriver fyra sätt att dela minnet mellan processorn och bildkretsen: processorn får alltid gå före, processorn går före men använder minnet bara under återgången, processorn får vänta med en väntesignal, eller också delas varje teckentid mellan de två. ABC 802 har ingen väntesignal, och processorn får minnet så fort den vill, medan bilden får det som låskretsarna höll sist (avsnittet Bildminnet). Det är det första sättet, med den skillnaden att bilden får de senast lästa tecknen i stället för det som råkar ligga på bussen.

3.3 Bildminnet

Bildminnet är 2 kilobyte på `$7800`–`$7FFF`, i fyra kretsar TC5514AP-2 på lägena 5G–8G (s. 77; ritningen, blad 5). Det är Toshibas statiska CMOS-minne med 1024 ord om 4 bitar, med samma ben som NMOS-minnet 2114 och med högst 200 nanosekunders åtkomsttid (Toshiba MOS Memory Products 1983, s. 131). Två kretsar bredvid varandra ger ett block om 1024 byte, och bildminnet har två sådana block. Det ena har de jämna adresserna och det andra de udda.

Här skiljer sig manualens två sidor åt. Bufferten 6F, som hör till 5G och 6G, är öppen när processorns lägsta adressbit BA0 är 0, och 7F, som hör till 7G och 8G, när den är 1. Då ligger de jämna adresserna i 5G och 6G (s. 82). Sidan efter säger tvärtom, att de jämna ligger i 7G och 8G (s. 83). Ordningen när bilden läses talar för den första: blocket i 5G och 6G läggs fram först (s. 77). För ett program spelar det ingen roll, eftersom processorn och bildkretsen delar upp adresserna på samma sätt.

Bildens väg till teckengeneratorn

Bildkretsen har 14 adressbitar, MA0–MA13, men bara MA0–MA10 går vidare. De klockas in i register (74LS377 på 9H och 11H, 74LS379 på 10H) med 12 MHz när signalen RC säger det, och kommer ut som LA0–LA10. MA11–MA13 är inte inkopplade (ritningen, blad 2). Det är därför bildminnet blir en ring (avsnittet Att rulla, s. 20).

LA1–LA10 adresserar båda blocken på en gång, så varje läsning ger två tecken. De går först genom var sin låskrets, TC40H373 på 6H och 8H, och klockas sedan in i var sitt register, TC40H374 på 5H och 7H. LA0 väljer vilket av de två registren som lägger sitt tecken på teckengeneratorns adress, det ena under den ena teckentiden och det andra under nästa (s. 77). TC40H är Toshibas snabba CMOS-serie, som enligt Toshiba är kompatibel med TTL-serien 54/74, och kretsarna har

samma ben som 74LS373, 374 och 245 (Toshiba C^2MOS 1985). Med 80 tecken går en teckentid på en halv mikrosekund, och då behöver bildkretsen bara läsa minnet en gång per mikrosekund. Varför Luxor delade minnet så säger manualen inte, men klockorna visar vad det ger. Räknaren på 1F delar 12 megahertz med sex till teckenklockan på 2 megahertz och ger också de 6 megahertz som JK-vippan på 12B delar till processorns 3 megahertz. På en mikrosekund går det alltså precis två teckentider och tre klockcykler för processorn, alltid i samma fasläge. Signalen I3*, en inverterad kopia av processorklockan, används enligt manualen för att fasa in processorn och bildkretsen mot varandra (s. 76). Bildkretsen läser minnet en gång i den mikrosekunden, och resten av tiden är minnet ledigt för processorn.

Adressen till minneskretsarna kommer genom tre multiplexrar, 74LS258A på 9G, 10G och 11G, och chip select till de två blocken genom 5F och 9G (ritningen, blad 5). Signalen MUX väljer: normalt får minnet LA1–LA10 från bildkretsen, och när processorn läser eller skriver i området får det BA1–BA10 från processorn (s. 77, 83). Processorns lägsta bit BA0 klockas in i två vippor, 74ALS74 på 4F, som väljer block och buffert. Bussbuffertarna 6F och 7F, TC40H245, kopplar blockens data till processorns databuss, och signalen EWR, en något fördröjd RD*, ger riktningen och bestämmer när chip select kommer (s. 82; ritningen, blad 4).

Processorns åtkomst

Processorns åtkomst börjar i adressavkodningen. På `$7800`–`$7FFF` blir signalen VR* aktiv, men bara om den senaste instruktionen hämtades utanför området; annars läses ROM:en där (s. 48–49). Det går alltså inte att köra kod i bildminnet. VR* släpper JK-vippan på 8C, så länge varken LXM eller CASDIS* hindrar det. Vid nästa klockpuls på I3* går vippans utgång CPU* låg. CPU* och BMRQ* bildar tillsammans MUX i NOR-grinden på 9D (s. 82; ritningen, blad 2).

Medan CPU* är låg står låskretsarna 6H och 8H stilla och håller de tecken som bildkretsen läste senast (s. 82). Bildens väg till teckengeneratorn får alltså inte det som processorn läser eller skriver. Någon väntesignal finns inte: Z80:ns WAIT kommer bara från bussens RDY (ritningen, blad 3). Processorn får därför bildminnet lika fort som det vanliga minnet. Manualen säger att låskretsarna lämnar de data som fanns på ingångarna när CPU* gick låg, och att ändringar på ingångarna därefter inte påverkar dem (s. 82). När processorn har minnet just när bildkretsen skulle ha läst visar bilden alltså det tecken som låskretsarna redan höll, inte det rätta. Felet sitter bara på de platserna och bara i den bilden, för bildminnet är oförändrat och nästa bild läser rätt. Processorns data hamnar aldrig i bilden. Hur mycket det syns på skärmen säger varken manualen eller ritningen.

BASIC:en lägger rad n på adressen $\$7800 + 80n$, så att de 24 raderna fyller 1920 byte. `PRINT CHR$(12)`, som tömmer skärmen, skriver mellanslag i just de 1920 byten; de sista 128, `$7F80`–`$7FFF`, rör BASIC:en inte. Hur skärmen rullar står i nästa avsnitt.

3.4 Att rulla

Bildkretsen har en räknare om 14 bitar för adressen. I början av varje bild laddas den från R12–R13, och sedan räknar den upp ett steg för varje tecken. Till bildminnet går bara de 11 lägsta bitarna: bit 0 väljer block och bit 1–10 adressen i blocken. Bildminnet blir därför en ring. Räknas bildens början S från `$7800`, ligger rad r på skärmen på adressen $\$7800 + (S + 80r) \bmod 2048$, och en rad som går förbi `$7FFF` fortsätter på `$7800`. Kretsen ser bara 14 bitar av $7800, det vill säga $3800, och $3800 plus 2048 är just $4000. Räknaren går därför förbi sin högsta adress på samma ställe som bilden går runt i minnet.

Det ger två sätt att rulla. Det ena är att flytta tecknen. BASIC:en gör så: `LINFD` på `$029D` kopierar raderna 1–23 ett steg uppåt med `LDIR`, 1840 byte, och tömmer den sista raden. `LDIR` tar 21 klockcykler per byte, så flytten tar omkring 39 000 cykler, 13 millisekunder på 3 MHz. Det andra sättet är att låta tecknen stå och flytta bildens början: lägga 80 till S, skriva det i R12–R13 och

tömma raden som då kommer längst ned, den som förut var överst. Det kostar fyra `OUT` och 80 byte, mindre än en millisekund.

ROM:en har båda sätten. Terminalrutinerna på `$7000` rullar med bildens början. BASIC:en går in i dem när ett program läser från skrivar- eller V24-kanalen och en bit i kanalens inställningar är satt (`PRINP` på `$7317`). Bildens början står i `CRTADR` på `$FF57` som en adress i bildminnet. Vid ny rad på den sista raden lägger rutinerna 80 till den. Hamnar den på `$8000` eller högre drar `VWRAP` på `$7761` ifrån 2048. Sedan skriver `CRTSTA` adressen i R12–R13, och raden töms, i två delar om den går förbi `$7FFF`. En plats på skärmen räknas fram på samma sätt: raden gånger 80 plus kolumnen plus `CRTADR`, och sedan `VWRAP` (`ADR80` på `$7E92`).

Markören kräver eftertanke. Bildkretsen jämför sin räknare, alla 14 bitar, med R14–R15. När bilden har gått runt i ringen har räknaren inte gått tillbaka till $3800. Den har fortsatt förbi $3FFF till $0000, medan adressen i minnet har gått tillbaka till `$7800`. Rätt värde i R14–R15 är därför adressen innan `VWRAP` drog ifrån 2048. Terminalrutinerna gör så: har adressen dragits ned, lägger de tillbaka $08 i den höga byten innan den skrivs i R14 (`$767D`). Med 40 tecken använder de inte bildkretsens markör alls. Medan de väntar på en tangent sätter de i stället bit 7 i tecknet, så att det visas inverterat.

När terminalrutinerna slutar måste bilden tillbaka till `$7800`, eftersom BASIC:en räknar med att rad n ligger på $\$7800 + 80n$. Att bara skriva $7800 i R12–R13 skulle flytta texten på skärmen. I stället vrider `CRTRST` på `$76A1` de 2 kilobyten på plats, utan någon buffert, så att varje byte flyttas S steg bakåt i ringen. Det görs i cykler. Från en startplats flyttas byten dit den ska, den som stod där flyttas S steg bakåt i sin tur, och så vidare tills cykeln är tillbaka där den började. Antalet cykler är den största gemensamma delaren av S och 2048. Eftersom 2048 är en tvåpotens är det S:s lägsta bit som är ett, och rutinen tar fram den med S `AND` $-S$. S är en multipel av 80 och alltså av 16, så det blir minst 16 cykler om högst 128 byte, och längden ryms i B-registret för `DJNZ`. Efteråt skriver `CRTHOM` $7800 i R12–R13, och skärmen ser ut som förut.

3.5 25 rader

Bildminnet räcker till 2048/80, det vill säga 25,6 rader, men BASIC:en visar 24. Luxors bok för den som programmerar ABC 800 på djupet, *Bit för bit med ABC 800*, visar hur den 25:e raden tas fram: `OUT 56,6,57,25` sätter R6 till 25 (s. 55). ABC 802 har bildkretsen på samma portar, och det går likadant där. Den nya raden ligger på `$7F80`–`$7FCF`, i den del av bildminnet som BASIC:en inte rör: efter `OUT 56,6,57,25` och `POKE 32640,72,69,74` står HEJ längst ned. Raden finns kvar efter `PRINT CHR$(12)` och när skärmen rullar, eftersom `LINFD` bara flyttar de 24 raderna. Bit för bit använder den till en rad med fast text, som ”ej kan raderas”, och skriver den genom att låta en strängvariabel peka in i bildminnet med `POKE` och `VAROOT`. Med 50 Hz behöver R4 och R7 inte ändras: bilden har 31 rader, och synken kommer efter rad 27. Med 60 Hz har bilden 26 rader och synken kommer efter rad 25, så där ryms den nya raden precis: R6 = 25 ger raderna 0–24, och R7 = 25 lägger synken direkt efter. Den synliga bilden blir 480 × 250 punkter.

Bit för bit nämner två saker till som går att göra med bildkretsen från BASIC: en tjockare markör med R11 och att flytta bilden i sidled med R2 och i höjdled med R5, med en varning för att synken kan släppa (s. 54). Bokens normalvärden är ABC 800:s och skiljer sig något från ABC 802:s: R2 är 100 och R7 är 28 (s. 55). På ABC 802 är markören ett streck på linje 9 (avsnittet Markören, s. 27). Ett lägre värde i R10 gör den tjockare, eftersom CUR är hög på varje linje från R10 till R11.

Ordbehandlaren ORD, skriven i Forth för ABC 802 till den här boken (kapitlet ORD och KALKYL, s. 74), använder både 25 rader och rullning med bildens början. Raderna 0–23 är texten och rad 24 visar läget. ORD håller S själv och sätter R12–R13 till $\$7800 + S$, så att rulla en rad är att ändra S med 80 eller -80 och rita bara den nya raden och lägesraden; resten av bilden står kvar. Markören räknas som i terminalrutinerna, i bildkretsens 14 bitar: $(\$3800 + S + 80r + k)$ mod $\$4000$ för rad r och kolumn k. När ORD slutar sätts R6 till 24 och R12–R13 till $7800, och skärmen töms

i stället för att vridas tillbaka.

3.6 Teckengeneratorn

Teckengeneratorn på läge 3G, ABCT2-1, är ett EPROM av typen 2732A-2 med 4 kilobyte (s. 78). Sockeln tar också en 2764 med 8 kilobyte, och bygeln S4 ställs efter vilken som sitter där (ritningen, blad 5). Varje tecken har 16 byte, en för varje linje, och punkterna står i bit 0–5 med bit 5 till vänster. Adressen är alltså teckenkoden gånger 16 plus linjen: A, kod 65, ligger på 1040–1055. Adressbit 0–3 är linjen, som kommer från bildkretsens RA0–RA3 genom multiplexern 74LS257A på 1G, bit 4–10 är koden från registren 5H och 7H, och bit 11 kommer från PAL-kretsen (ritningen, blad 5). En kolumn och den sista linjen i cellen är alltid mörka, så att tecknen får mellanrum (s. 78, 80). Så här ser A ut:

```
Linje  Byte   Punkter
0      00     ......
1      04     ...#..
2      0A     ..#.#.
3      11     .#...#
4      1F     .#####
5-7    11     .#...#
8-14   00     ......
15     3F     ######
```

Bildkretsen räknar bara linjerna 0–9. Linje 10–13 är tomma i alla vanliga tecken, och de två sista används på annat sätt. Linje 14 är tom och visas i stället för tecknet när det blinkar och är i sin mörka fas, och när det är dolt. Linje 15 är hel och är markören (avsnittet Markören, s. 27). Kretsen har två uppsättningar om 128 tecken: text på `$000`–`$7FF` och grafik på `$800`–`$FFF`. Signalen RG från PAL-kretsen väljer uppsättning med adressbit 11. I textuppsättningen sitter de svenska tecknen på ASCII:s platser för hakparenteser och andra tecken: Ä, Ö, Å och Ü på 91–94, ä, ö, å och ü på 123–126, É på 64, é på 96 och ¤ på 36. Kod 127 är en hel ruta. Så här ser hela kartan ut:

Kod	**Text, `$000`–`$7FF`**	**Grafik, `$800`–`$FFF`**
0–31	attributkoder	samma koder
32–63	siffror och tecken, ¤ på 36	mosaik
64–95	versaler, É på 64, Ä Ö Å Ü på 91–94	samma versaler
96–127	gemener, é på 96, ä ö å ü på 123–126, hel ruta på 127	mosaik
128–255	som 0–127, inverterat	som 0–127, inverterat

Avläsningen av teckengeneratorn, 6490191-01, som finns på nätet är 8 kilobyte. Bara den första halvan är tecken. Den andra är Z80-kod, men inte någon egen: den är byte för byte densamma som `$1000`–`$1FFF` i BASIC-ROM:en 02-11, med rutinerna för att stänga filer, `LIST`, `SAVE`, `LOAD` och `MERGE` och början av radöversättaren. Den börjar mitt i en rutin, med `POP HL` på `$1000`. Processorn kan inte heller nå den, eftersom teckengeneratorns data bara går till skiftregistret och PAL-kretsen (ritningen, blad 5). Det är alltså en rest. 02-11 är själv en krets om 8 kilobyte, den första av BASIC:ens tre, och resten är just dess övre halva. Det passar med att en krets på 4 kilobyte lästes in i den nedre halvan av en buffert där 02-11 låg kvar sedan förra avläsningen. Emulatorn MAME har nämligen teckengeneratorn, `abc t02-1.3g` märkt 64 90191-01, som 4 kilobyte, och dess kontrollsumma (CRC `4d54eed8`) är densamma som för den första halvan av avläsningen på nätet. Den första halvan stämmer däremot med manualen och ritningen: linje 15 är hel i alla 256 tecken, linje 10–14 har något bara i kodernas platser 1–9 och 17–24, och de två uppsättningarna skiljer sig bara i koderna 32–63 och 96–127, de som är mosaik.

Av de åtta databitarna går bit 2–5 direkt till skiftregistret, en 74LS166 på 3F. Bit 0 och 1 går först genom PAL-kretsen, som kan släppa fram dem eller tvinga dem till ett, och bit 6 och 7 går bara till PAL-kretsen, som läser attributen ur dem (ritningen, blad 5; avsnittet Attributen, s. 25). Skiftregistret laddas från NAND-grinden på 13B, när RC, DEN-3 och, med 40 tecken, LA0 säger det; är bilden inte synlig kommer ingen laddning och ingen punkt (s. 77–78). Sedan skiftar det ut en punkt för varje puls på 12 MHz. Punkterna går genom en exklusiv eller-grind på 10D, där

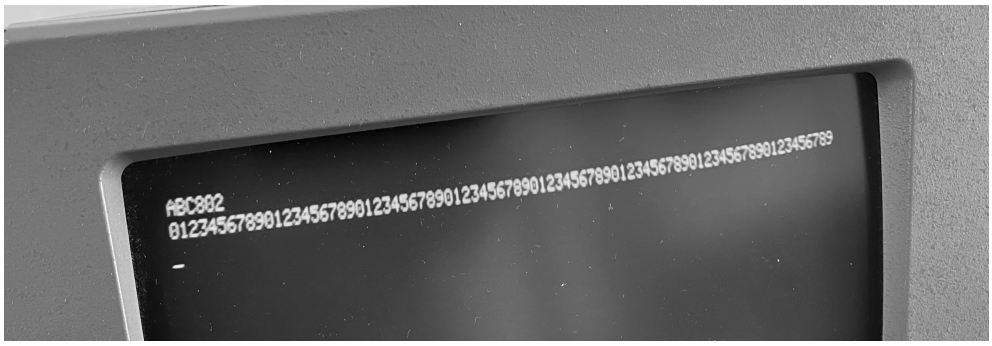
PAL-kretsens utgång RI kan vända dem, och sedan genom en D-vippa, 74ALS175 på 1E, som klockas med samma 12 MHz och lägger ut dem som videosignalen (ritningen, blad 5).



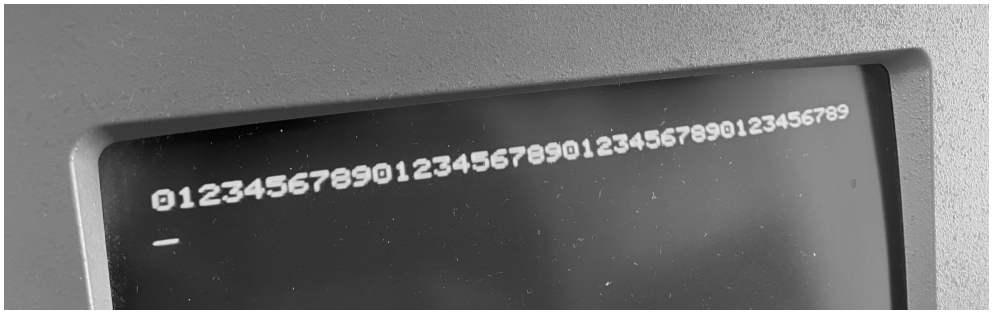
Tecknen 32–127 i teckengeneratorn 6490191-01, text överst och grafik under. Ritade ur ROM:en med bilder/teckengeneratorn.py.

3.7 40 och 80 tecken

Bygeln ”40/80 char” läses vid start på ingången RI i DART:ens kanal A och bestämmer hur många tecken BASIC:en börjar med (`CRTINI`). Sedan kan programmet byta med `WIDTH 40` och `WIDTH 80`, som står i Luxors manual för BASIC II på ABC 802 (ABC 802 BASIC II, s. 61). Bildkretsen ändras inte; den räknar alltid 80 tecken per rad. I stället ställer BASIC:en utgången RTS i DART:ens kanal B, som går till signalen MUX 80/40, och det gör den vid varje utskrift (`SETW5` på `$0420`). Med 40 tecken går skiftregistret på 6 MHz i stället för 12, och ett nytt tecken laddas bara varannan teckentid. Bara de jämna adresserna syns då, och varje tecken blir dubbelt så brett (s. 78). BASIC:en tar hänsyn till det genom att dubbla kolumnen när markörens adress räknas fram och stega två byte i taget när den skriver (`CURADR` på `$02FA`). De udda byten finns kvar i minnet men syns inte.



Skärmen med 80 tecken per rad: raden ABC802 och en rad med siffror. Markören är ett streck under nästa rad.



Skärmen med 40 tecken per rad: siffrorna 0–9 fyra gånger, med dubbelt så breda tecken.

3.8 Attributen och grafiken

ABC 802 har attribut som text-tv: en kod i raden gäller från sin plats till slutet av raden och tar själv en plats, som visas som ett mellanslag. Koderna är inte inbyggda i någon logik utan står i teckengeneratorn, i bit 6 och 7, som inget vanligt tecken använder, och i bit 0–1. PAL-kretsen ABCP2-1 på läge 2G, en PAL16R4, läser bitarna och håller attributen i sina register. Den nollställs av den horisontella synkpulsen, så varje rad börjar med text, utan blink och synlig (s. 78–80). Ur ROM:en går koderna att läsa, och de är desamma i båda uppsättningarna:

Kod	I CHR$	BASIC II	Byte	Verkan
1–7	129–135	RED–WHT	$80	text
8	136	FLSH	$C1	blinkande
9	137	STDY	$81	fast
17–23	145–151	GRED–GWHT	$C0	grafik
24	152	HIDE	$C2	dold
0, 10–16, 25–31		NRML, DBLE med flera	$00	ingen, bara ett mellanslag

Koderna i PAL-kretsen

Byten är den som står på linje 0–14 i ROM:en. Bit 7 (ATE) säger att det är en kod, bit 6 (ATD) är värdet och bit 0–1 (AT0 och AT1) väljer attribut, skrivna AT1 AT0: 00 är grafik, 01 blink och 10 dold (s. 79). Text är alltså grafik med värdet 0, och fast är blink med värdet 0. Dold finns bara med värdet 1, så ingen kod gör tecknen synliga igen; det dolda varar till radens slut. På linje 15 har koderna i stället bit 2–5 satta och ger markörens linje som andra tecken.

PAL-kretsen klockas med 12 MHz. Den har fyra register: RG för grafik, RF för blink, RC för dold och RI för inverterat tecken (RC här är inte samma signal som RC i klockan). Signalen LL säger när ett nytt tecken ska läsas in, och HS nollställer registren (s. 80). RG går till teckengeneratorns

adressbit 11 och väljer grafikuppsättningen. RI vänder punkterna i grinden på 10D. Utgången OS väljer multiplexern 1G:s andra ingångar, som ger linjen 14 i stället för bildkretsens RA0–RA3:

```
OS = FC · RF + RC
```

Ett dolt tecken blir alltså alltid den tomma linjen 14, och ett blinkande tecken blir det när blinkklockan FC är hög. Utgångarna O0 och O1, som ersätter databit 0 och 1 på väg till skiftregistret, följer AT0 och AT1 för vanliga tecken, är noll för en kod, så att den blir ett mellanslag, och är ett under markören (s. 80).

Blinkklockan FC kommer från en binärräknare CD4020B på 2E, som räknar den vertikala synkpulsen VS*. Ritningen kallar utgången på stift 4 för Q5 (ritningen, blad 2), men enligt TI:s datablad är stift 4 Q6 och stift 5 Q5 (CD4020B, Texas Instruments). Q6 delar bildtakten med 64, och med 50 Hz blir varje fas 32 bilder, 0,64 sekund; med 60 Hz 0,54 sekund. Q5 skulle ge hälften. Manualen säger en halv sekund (ABC 802 BASIC II, s. 78–79), vilket ligger närmare Q6:s 0,64 än Q5:s 0,32. Emulatorn MAME växlar också blinkklockan efter 33 bildpulser, nära Q6:s 32. Det är alltså namnet på ritningen som är fel, inte benet.

Koderna från BASIC

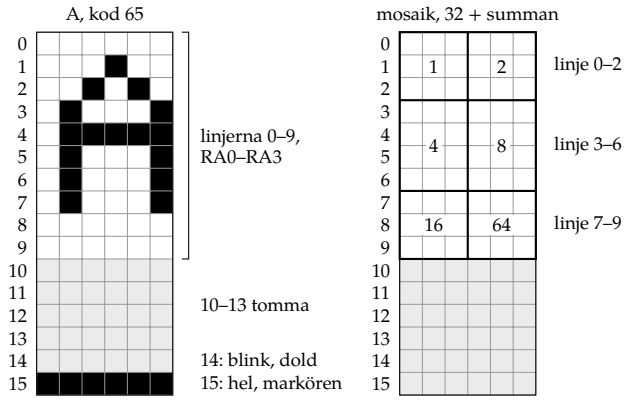
Koderna kommer inte fram med `PRINT CHR$(17)`. Utskriftsrutinen `SCRCHR` på `$01E6` känner igen 7 (ljudet), 8 (vänster), 10 (ny rad), 12 (tom skärm), 13 (radens början) och 27 (ESC) och hoppar över de andra koderna under 32. Först därefter tas bit 7 bort, med masken `CHRMSK` på `$FF61`. `PRINT CHR$(151)` skriver alltså 23 i bildminnet och slår på grafiken. BASIC II har namn på dessa tal: `PRINT GWHT` är detsamma. Namnen är gemensamma med ABC 800 C, där siffrorna 1–7 är färger, och manualen säger att ABC 802 inte kan visa färg; dubbel höjd, sammanhängande och separerad grafik och hållen grafik ”kan ej genereras på ABC 802” (ABC 802 BASIC II, s. 78–79, 81).

Grafiken

I grafikuppsättningen är koderna 32–63 och 96–127 mosaik, medan 64–95 är versalerna som förut. Cellen delas i två kolumner om tre punkter och tre rader om 3, 4 och 3 linjer, och de sex delarna har var sin bit i koden, som i text-tv:

Linje	Vänster	Höger
0-2	1	2
3-6	4	8
7-9	16	64

plus 32 i alla koder



Teckencellen, 6 punkter och 16 linjer. Till vänster A ur teckengeneratorn, till höger mosaikens sex delar med sina bitar. Bildkretsen räknar bara linje 0–9 (grått bakom de andra). Linje 14 och 15 hämtas genom multiplexern 1G.

Satserna `SET DOT R%,K%` och `CLR DOT R%,K%` tänder och släcker en sådan del, och `DOT(R%,K%)`

läser den. Raden R% går från 0 till 71, tre för varje textrad, och kolumnen K% från 2 till 79 med 40 tecken och till 159 med 80; utgångspunkten är övre vänstra hörnet. `TXPOINT X,Y` gör detsamma med utgångspunkten nere till vänster: X från 0 till 77 eller 157 och Y från 0 till 71, så att ROM:en räknar om till raden $71 - Y$ och kolumnen $X + 2$ (ABC 802 BASIC II, s. 78–82). De två första kolumnerna är platsen för grafikkoden. Den skriver `SET DOT` inte själv; raden behöver en `PRINT GWHT` först, annars visas punkterna som vanliga tecken (`SETDOT` på `$047D`).

Inverterade tecken

Bit 7 i bildminnet inverterar tecknet: den går till ingången INV på PAL-kretsen och därifrån till en grind som vänder punkterna (s. 79). `PRINT` tar bort biten, men `PUT` behåller den, eftersom masken då är 255: `PUT CHR$(193)` ger ett inverterat A. Med en attributkod blir det en hel ljus ruta. I PAL-kretsens ekvation läses RI in från INV varje gång LL är låg, utan att ATE är med (s. 80), och en kod har bara nollor på väg till skiftregistret: bit 2–5 är noll på linje 0–14, och O0 och O1 är noll. Grinden vänder då ett mellanslag. Koden verkar ändå som förut, eftersom RG, RF och RC inte bryr sig om INV. MAME ritar inget alls för en kod och visar alltså inte detta.

3.9 Markören och skärmsläckaren

Markören visas när bildkretsens adress är densamma som i R14–R15. Signalerna DEN och CUR fördröjs tre teckentider i tre D-vippor på 9H, så att de kommer samtidigt som tecknet, och heter sedan DEN-3 och CUR-3. CUR-3 stänger utgångarna på multiplexern 1G, och motstånden på dess utgångar ger då bara ettor: teckengeneratorn får linjen 15, som i alla tecken är en hel linje om sex punkter (s. 78; ritningen, blad 5). PAL-kretsen tvingar samtidigt bit 0 och 1 till ett, så att också en kod får hela linjen. Bildkretsen ger bara CUR på linjerna från R10 till R11, och med BASIC:ens värden är det linje 9. Markören är alltså ett streck under tecknet, i den linje som annars skiljer raderna åt.

Bildkretsen kan själv låta markören blinka, men BASIC:en använder inte det. Medan den väntar på en tangent växlar `KEYWT` på `$03A0` R10 mellan $09 och $29 ungefär var 0,18 sekund, det vill säga mellan markör och ingen markör. Efter 41 växlingar, ungefär sju sekunder, slutar den och låter markören stå kvar.

Bygeln S1, ”CLEAR SCREEN TIME OUT”, läses på ingången DCD i SIO:ns kanal B. Är den satt och ingen tangent har tryckts på omkring fem minuter, sätter BASIC:en R1 till 0, så att bildkretsen inte visar något. Nästa tangent sätter R1 till 80 igen, och bilden kommer tillbaka orörd, eftersom bildminnet inte har ändrats. Tangenten går sedan vidare till programmet, utom högerpilen, som sväljs.

Tangentbordet

Där inget annat anges kommer uppgifterna i det här kapitlet från Luxors servicemanual för ABC 802 (ABC 802 Servicemanual), och sidnumren inom parentes avser den. Ritningen av själva tangentbordet finns inte i manualen, utan i det finska arkivet devili.iki.fi, som också har datorkortets ritningar. Det som sägs om rutinerna är läst ur ROM:en.

4.1 Ett bord för sig

I ABC80 och ABC 800 sitter tangentbordet ihop med datorn. I ABC 802 är det tvärtom bildskärmen som är datorn, och tangentbordet är en egen låda på en sladd. Det är lågt, 30 millimeter (s. 5), och enligt bruksanvisningen 345 × 236 millimeter stort, 38 millimeter högt bak och 19 fram, med en sladd på 1,5 meter (bruksanvisningen s. 36). Bruksanvisningen för ABC 806, som har samma tangentbord, kallar det lågprofil enligt DIN 66345.

Tangentbordet är inte bara en matris av kontakter. Det har en egen processor, och den talar med datorn åt båda hållen: datorn får koder för tangenterna, och tangentbordet får kommandon från datorn (s. 4). Kontakterna är kapacitiva och ska klara mer än hundra miljoner nedslag (s. 84). Nokia-Luxors broschyr om ABC 802 säger att tangentbordet är värt ett eget kapitel, och att Statskontorets utvärdering räknade det till marknadens bästa.

4.2 Fyra tangentbord

Eftersom tangentbordet är fristående kunde Luxor sälja det i flera utföranden, och datorn såldes utan: i Luxors cirkaprislista Nr 1-83 kostar ABC 802 exklusive tangentbord 7 600 kronor. Tre tangentbord fanns från början. ABC 55 är skrivmaskinsdelen, ABC 22 en lös del med åtta funktionstangenter och en sifferdel, och ABC 77 är båda i en låda. Namnen är antalet tangenter. Tabellen över koderna (s. 88) har 55 tangenter i skrivmaskinsdelen, och åtta funktionstangenter och fjorton siffertangenter, 22 i allt, i den andra delen. Bruksanvisningen för ABC 806 beskriver de tre på samma sätt.

ABC 22 har ingen egen processor. På ritningen av tangentbordet är den ritad som ett tillval, ”numerical keyboard ABC22”, med en buffert och en kabel från dess kontakt J4 till J2 i ABC 55. Kabeln för över tre av X-ledningarna och strobpulsen till ABC 22, och ABC 22:s Y-ledningar kommer tillbaka på J2 och läses på samma buss som de andra. ABC 22 blir alltså fler rader och kolumner i ABC 55:s matris, och det är ABC 55:s processor som läser dem (ritningen 66 88055-01).

Senare kom ABC 99, som hörde till ABC 1600. Tekniska museets ABC 1600, som levererades till Uppsala universitet 1985, har bildskärmen ABC 1615 och tangentbordet ABC 99 (Tekniska museet, TEKS0047538). ABC 99 står ändå redan i Luxors cirkaprislista Nr 12-84, tillsammans med de andra tangentborden och ABC 1600, och sedan i listorna för ABC 800 från 1985. Enligt emulatorn MAME har det två processorer 8035, funktionstangenterna PF1–PF15, fyra piltangenter, ALT och INS, och en kontakt för musen ABC R8 med tre knappar. MAME låter det sitta på ABC 802 och ABC 806 också. Någon manual för ABC 99 har inte gått att hitta. I ABC 802:s ROM tar avbrottsrutinen emot vilken byte som helst från tangentbordet och sparar den, och bara CTRL-C behandlas för sig. Ingen tabell i ROM:en stänger alltså ute de nya tangenterna, men vilka koder



Tangentbordet ABC 77. Till vänster skrivmaskinsdelen med 55 tangenter och CAPS LOCK med sin lampa, i mitten funktionstangenterna PF1–PF8 och till höger sifferdelen med CE och en hög returtangent.

de ger bestäms av programmet i ABC 99, och det beskriver ingen källa. Priserna i kronor enligt Luxors prislistor:

Tangentbord	Artikelnummer	1-83	12-84	1986	1987
ABC 22	2308022-01	1 060	1 075	–	–
ABC 55	2308055-01	2 150	2 350	–	–
ABC 77	2308077-01	2 800	2 860	3 000	3 000
ABC 99	230 8450-15	–	3 375	3 500	3 500

Kolumnen 12-84 är Luxors cirkaprislista Nr 12-84, som ersatte 2-83. Samma priser står i Cirkapris 3-85 och i cirkaprislistan för ABC 800 I och II från 1985-11-01, men där finns inte ABC 77. I 3-85 står i stället ABC 22 och ABC 55 som en post för 2 860 kronor, samma pris som ABC 77. Ett streck betyder att tangentbordet inte står i listan.

4.3 Processorn

Processorn är en Intel 8035, eller en 8048 med programmet i masken. Med en 8035 ligger programmet i en EPROM 2716, och en låskrets 74LS373 håller den låga halvan av adressen. Bygeln E3/E4 väljer mellan programmet i 8048:an och det i EPROM:en; den går till processorns ben EA (s. 84, 88). Kristallen är på 4,608 MHz (s. 85).

Tangenterna sitter i en matris med 12 X-ledningar och 8 Y-ledningar. Processorn lägger ut numret på en X-ledning på port 2 (P20–P23) till en avkodare, och en strob från dess ben PROG ger två pulser om 1 mikrosekund med 50 mikrosekunders mellanrum. Eftersom kontakterna är kapacitiva går pulsen över till Y-ledningen bara där en tangent är nedtryckt, och Y-ledningarna läses på port 1 (P10–P17) genom en buffert (s. 84–85, ritningen 66 88055-01).

Svensk och amerikansk layout finns i programmet, och bygeln E5/E6 väljer mellan dem. Danska, franska, spanska och tyska tangentbord kräver en PROM för omkodningen på läge Z14, byglarna E7/E8 och E9–E12, och nya tangenthattar. E13–E18 är reserverade (s. 86, 88). På ritningen läser en buffert 74LS240 byglarna till bussen, så programmet ser dem som data. Koderna följer de svenska standarderna SIS 66 22 41 och SIS 63 61 27 (s. 84), och bruksanvisningen kallar layouten svensk skrivmaskinsstandard (bruksanvisningen s. 14).

Programmet har en vakthund. En räknare 4020 räknar pulserna från processorns ALE, och programmet måste nollställa den med jämna mellanrum. Gör det inte det, ger räknaren en återställning. Manualen säger att det sker efter 16 384 pulser, när Q13 på ben 2 går hög (s. 84), och ritningen har också ben 2. Men 4020:an räknar på den fallande flanken, och Q13 går hög första gången efter $2^{12} = 4\,096$ pulser (Texas Instruments, CD4020B). Med ALE på 307,2 kHz (se nedan) blir tiden 13,3 millisekunder. Manualens 16 384 pulser skulle ge 53 millisekunder. Tangentbordet återställs också vid tillslag, av en kondensator på 68 mikrofarad, av datorn och av återställningsknappen. Knappen sitter på tangentbordet och återställer genom ledningen i sladden också datorn (s. 85; datorkortets ritning blad 3).

Ritningen har en del som manualen inte nämner. En låskrets 74LS374 tar bussens data när processorn skriver och lägger ut dem på ben 4–11 av kontakten J3. Ben 1 har +5 V, ben 2 jord, och ben 12–13 har tonen från 556:an genom motståndet R6, samma signal som går till högtalaren. Vad som skulle kopplas till J3 har inte gått att få fram. Servicemanualen nämner inte kontakten, och inte heller någon annan källa som har hittats. Emulatorn MAME kallar den en stiftlist CN1 utan att säga vad den är till för, och tar bara emot byten utan att göra något med den (ritningen 66 88055-01).

4.4 Sladden

Sladden har sju ledare och en sjupolig DIN-kontakt (s. 7, och Nokia-Luxors broschyr). Servicemanualen kallar den på ett ställe sexpolig (s. 10), men ritningarna har sju ben. På datorns sida går de så här (datorkortets ritning blad 1):

Ben	Ledning	I datorn
1	TxD	från datorn till tangentbordet
2	jord	
3	RxD	till DART:ens kanal B, ingången RxD
4	klocka	till kanal B, ingången RTxC
5	Key Down	till kanal B, ingången DCD
6	+12 V	matningen till tangentbordet
7	Reset	till datorns återställning

I tangentbordet blir +12 V till +5 V i en regulator. Koderna skickas från processorns ben P25 genom en buffert med öppen kollektor, och ben P26 ger signalen Key Down, som är aktiv så länge en tangent hålls ned. Det som kommer från datorn går genom en Schmittrigger till processorns avbrottsingång, så att ett kommando avbryter det tangentbordet håller på med (ritningen 66 88055-01).

Förbindelsen är seriell på TTL-nivå, med en startbit, åtta databitar och två stoppbitar i positiv logik (s. 65, 85). Takten är 600 bitar per sekund. På ett ställe står 650 (s. 65), men det är ett tryckfel: på s. 85 står 600, och räkningen nedan går också jämnt ut med 600. Ett tecken tar 11 bitar, 18,3 millisekunder, och tangentbordet kan alltså skicka högst 54 tecken i sekunden.

Klockan till DART:en kommer från tangentbordet, och manualen kallar den 10 kHz (s. 65). På ritningen tar en räknare 74LS393 pulserna från ALE och delar med 32. ALE är kristallens 4,608 MHz delat med 15, alltså 307,2 kHz, och delat med 32 blir det 9 600 Hz, som är 16 gånger 600. DART:en räknar då 16 klockpulser på varje bit, och de två sidorna går i takt eftersom de har samma klocka. Samma klocka går också till processorns ingång T1 (ritningen 66 88055-01).

4.5 Koderna

Med CAPS LOCK, CTRL och de två SHIFT kan varje tangent ge åtta koder (s. 85). CAPS LOCK slår om vid varje nedslag och har en lampa som visar läget. Koderna är ASCII. CTRL med en bokstav ger koderna 0–31, och bruksanvisningen har en tabell över dem: CTRL-C avbryter, CTRL-H och ← suddar ett tecken, CTRL-I och → används när en rad redigeras, CTRL-L tömmer skärmen, CTRL-S stegar programmet en sats i taget och CTRL-X suddar raden. Tangenten CE på sifferdelen ger samma kod som CTRL-X (bruksanvisningen s. 13–14, 40; BASIC II-manualen s. 104–105).

4.6 Funktionstangenterna

ABC 77 och ABC 22 har funktionstangenterna PF1–PF8. Med SHIFT och CTRL ger de 32 koder, från 192 till 247. PF1–PF8 ger 192–199, och SHIFT lägger till 16 och CTRL 32. På ABC 55 utan funktionstangenter går samma koder att få med CTRL och en tangent i de tre översta raderna (s. 86; bruksanvisningen s. 41; BASIC II-manualen s. 104):

Funktionstangent	Koder	På ABC 55
PF1–PF8	192–199	CTRL och 1–8
SHIFT PF1–PF8	208–215	CTRL, SHIFT och 1–8
CTRL PF1–PF8	224–231	CTRL, SHIFT och Q W E R T Y U I
CTRL SHIFT PF1–PF8	240–247	CTRL, SHIFT och A S D F G H J K

Servicemanualen ger som exempel att SHIFT PF5 är CTRL, SHIFT och T (s. 86). Enligt tabellen ovan är det CTRL PF5, kod 228. BASIC II-manualen har i kapitlet om funktionstangenterna C där bilagan har Q (BASIC II-manualen s. 83, 104).

BASIC II-manualen säger att koderna är desamma som ABC 800:s funktionstangenter ger. Trycker man på en funktionstangent när programmet väntar i INPUT eller INPUT LINE blir det fel 53. Programmet kan fånga felet och ta reda på vilken tangent det var: SYS(6) lägger tillbaka det senast lästa tecknet, och GET läser det igen (BASIC II-manualen s. 83, 85).

4.7 Att styra tangentbordet

Datorn skickar kommandon till tangentbordet med OUT 34,kod, alltså på port 34 ($22), som är DART:ens kanal B. Kommandona får inte komma tätare än 20 millisekunder (s. 87–88):

Kod	Verkan
1 / 129	klockan av / på
2 / 130	klicket på 5 ms av / på
4 / 132	klicket på 20 ms av / på
5 / 133	repetitionen av / på
6 / 134	ASCII / upp och ned
7	ett pip
8 / 136	CAPS LOCK av / på; i läget upp och ned bara lampan
9	ett klick på 5 ms
10	ett klick på 20 ms

Klockan är tangentbordets egen signal och klicken hörs när en tangent slås ned. Vid start är klockan på, det korta klicket på och repetitionen på, och CAPS LOCK av: koderna 129, 130, 133 och 8. Bruksanvisningen har samma tabell utom 6 och 134, och som exempel ett program som skickar 7 (bruksanvisningen s. 15).

```
10 Data=7
20 Out 34,Data
```

4.8 Upp och ned

Med kommandot 134 går tangentbordet över till ett läge där varje tangent har ett eget nummer. Nedslaget ger numret och släppet samma nummer plus 128 (`$80`). Det gäller alla tangenter, också CAPS LOCK, CTRL och SHIFT, så programmet får själv hålla reda på dem och kan se flera tangenter som hålls ned samtidigt. Klicken tystnar i det här läget (s. 87). Numren, hexadecimalt, rad för rad som manualen ritar tangentbordet (s. 88):

```
41 40 39 38 31 30 29 28 21 20 19 18 11
04 42 43 3A 3B 32 33 2A 2B 22 23 1A 1B 12
05 44 45 3C 3D 34 35 2C 2D 24 25 1C 1D 14
07 46 47 3E 3F 36 37 2E 2F 26 27 06 17
               1F
```

Här är 04 CAPS LOCK, 12 RETURN, 05 CTRL, 14 vänsterpilen, 07 och 06 vänster och höger SHIFT, 17 högerpilen och 1F mellanslaget. Funktionstangenterna PF1–PF8 är `$58`–`$5F`, två och två i fyra rader, och sifferdelen är 50 51 48 49, 52 53 4A 4B, 54 55 4C 4D och 56 4F, där 4D är dess RETURN, 56 nollan och 4F punkten (s. 88).

BASIC:en har inget eget stöd för läget, men den behöver det inte heller. Avbrottsrutinen `KEYINT` (se nedan) lägger varje byte från tangentbordet i `LASTKY` utan att se efter vad den är, så `GET` ger numren och släppen på samma sätt som vanliga tecken. Bara koden 3 får en särskild behandling, och den förekommer inte bland numren. Bufferten rymmer däremot bara ett tecken, så ett program som inte hinner läsa varje nedslag och släpp tappar några av dem.

4.9 Två ljud

ABC 802 har två ljudkällor. Den ena sitter i tangentbordet: en timer 556 som startas från processorns ben P24 och driver en liten högtalare. Det är den som ger klockan, klicken och pipet när datorn skickar kod 7. Den första halvan svänger fritt med R7 2,7 kΩ, R5 15 kΩ och C15 22 nF, vilket ger omkring 2 kHz. Den andra halvan är en monostabil vippa. Dess triggeringång, ben 8, sitter på ledningen RESET i sladden, som också återställningsknappen drar låg, och dess utgång RST håller tangentbordets processor i återställning i 1,1 × 100 kΩ × 100 nF, omkring 11 millisekunder (ritningen 66 88055-01).

Den andra källan sitter i datorn. Datorkortets ritning visar en avkodare för läsportarna, och dess utgång för port 5 går till en räknare 74LS390 vars första steg delar med två. Utgången kallas SOUND och går till kommunikationsmodulen, där en transistor driver en piezosummer genom en trimpotentiometer, volymratten på baksidan (datorkortets ritning blad 3; kommunikationsmodulens ritning; s. 20, 30). Varje IN 5 slår alltså om summern, precis som i ABC 800. Bussen ut till korten får ingen strob för port 5 (s. 42), men det gör inget, eftersom vippan sitter på datorkortet.

Bit för bit med ABC 800 beskriver vippan och hur man spelar på den. Det enklaste ljudet är pipet från PRINT CHR$(7). En ton får man genom att läsa port 5 i en slinga, och mellan läsningarna avgör väntan tonhöjden. Boken har en funktion i maskinkod, där Frek är väntan och Längd antalet omslag (Bit för bit med ABC 800 s. 52–53):

```
10 DEF FNTut(Frek,Längd) LOCAL Tut$=15
20 Tut$=CHR$(66,22,0,219,5,197,16,254,193,27,122,179,32,245,201)
30 RETURN CALL(VARPTR(Tut$),Frek*256+Längd)
40 FNEND
```

Byten är LD B,D, LD D,0, IN A,(5), PUSH BC, DJNZ till sig själv, POP BC, DEC DE, LD A,D, OR E, JR NZ tillbaka till IN och RET. Boken varnar för att klockavbrottet ger ett litet klick varje sekund i tonen och att en ton längre än 10–20 millisekunder får klockan att gå fel, eftersom klockan räknas fram var tionde millisekund; DI och EI runt tonen tar bort klicket. Det bör gälla ABC 802 också, men det är inte prövat (Bit för bit med ABC 800 s. 52–53).

ROM:ens rutin BEEP på $0241, som anropas när CHR$(7) skrivs till skärmen, gör båda sakerna. Den läser port 5 200 gånger med 413 T-tillstånd mellan, 137,7 mikrosekunder med 3 MHz, så att summern får en ton på ungefär 3,6 kHz i 27,5 millisekunder. Det är räknat ur instruktionerna; ett klockavbrott under tiden gör tonen lite längre. Sedan skickar den kod 7 till tangentbordet, men bara om DART:ens sändbuffert är tom. Väntan gör också att två pip inte kommer tätare än 20 millisekunder till tangentbordet.

4.10 Datorns sida

I datorn tas tangentbordet emot av DART:ens kanal B på portarna 34 och 35 ($22 och $23). När ett tecken har kommit ger kanalen ett avbrott, och avbrottsrutinen KEYINT på $043E läser koden och lägger den i LASTKY på $FFE3. Är koden CTRL-C sätts dessutom en bit i den byte som BRKPTR pekar på, så att det program som kör kan avbrytas. Sist sätter KEYSET KEYFLG på $FFE2 till 128 för att visa att ett tecken väntar. Bufferten rymmer bara ett tecken. Rutinen ser inte efter om flaggan redan är satt, så kommer ett till innan det första har lästs skrivs det över.

Det märks i ett program som gör annat en stund mellan läsningarna. Ordbehandlaren ORD i Forth ritar om skärmen på omkring 0,16 sekunder, och en tangent som kom under tiden skulle skriva över den förra. ORD läser därför tangenterna till en egen kö medan det ritar, med ett ord som ser efter i KEYFLG och LASTKY mellan raderna. Med tangenterna nere 30 millisekunder och 30 millisekunder mellan dem, 17 tecken i sekunden, tappas då inget, inte heller när skärmen rullar (forth/README.md).

Tidsprovet SNAKE läser tangenten på ett annat sätt, med ABS(SYS(8)*INP(34)), alltså DART:ens dataregister direkt så länge en tangent hålls ned. Avbrottsrutinen har då redan läst tecknet, men registret ger det senast mottagna tecknet igen. Att kretsen gör så är slutet ur programmet, som går på maskinen. MAME:s efterbildning av kretsen gör likadant: när kön i mottagaren är tom står det sista tecknet kvar i registret.

BASIC II har tre funktioner för tangentbordet. SYS(5) är flaggan, 128 när ett tecken väntar och 0 annars; GET, INPUT och INPUT LINE nollar den. SYS(6) lägger tillbaka det senast lästa tecknet, och SYS(8) är -1 så länge en tangent är nedtryckt. Den läser Key Down på DART:ens ingång DCD genom rutinen KEYDN på $3B70 (BASIC II-manualen s. 76).

En tangent som trycks under en FOR-slinga ger SYS(5) = 128, och efter GET är den 0. Efter SYS(6) är den 128 igen, och nästa GET ger samma tecken en gång till. Skickas PF1 till ett INPUT blir det fel 53. Fel 57, att tecknet inte kom i tid, och fel 58, ett ogiltigt tecken, hör enligt manualen också till tangentbordet (BASIC II-manualen s. 85).

CTRL-C och CTRL-S verkar medan programmet kör. Det första CTRL-C stoppar utskriften, ett andra ger ”Stop in line”, och en annan tangent låter programmet fortsätta. BASIC II-manualen säger att det krävs två CTRL-C för att avbryta. CTRL-S kör programmet en sats i taget, ungefär en programrad för varje tryck (BASIC II-manualen s. 105; bruksanvisningen s. 40).

4.11 Att skriva en rad

En rad skrivs in av CONINP på $0322. RETURN och CTRL-C avslutar raden, CTRL-X och CTRL-L suddar den, och vänsterpilen (kod 8) suddar ett tecken. Högerpilen (kod 9) tar nästa tecken ur en mall, och andra styrtecken struntar rutinen i. En funktionstangent, med kod 128 eller mer, ger fel 53.

Mallen gör att en rad med fel går att rätta med pilarna (BASIC II-manualen s. 19). När en rad inte går att översätta blir den mallen för nästa inmatning. Raden 10 PRUNT 5 ger fel 220, och sedan ger fem högerpilar, två vänsterpilar, IN och tre högerpilar raden 10 INUNT. Det visar hur mallen

fungerar: vänsterpilen suddar på skärmen men flyttar inte tillbaka i mallen, så efter IN fortsätter mallen där den var.

Medan datorn väntar på en tangent blinkar markören i `KEYWT` på `$03A0`, och efter omkring fem minuter släcks skärmen (se s. 27). Den tangent som väcker skärmen kommer fram till programmet, utom högerpilen, som sväljs.

Terminalprogrammet läser tangentbordet på sitt eget sätt i `TRMIN` på `$7EA8`. CTRL-S stoppar och CTRL-Q släpper, om inte en bit i dess parametrar säger annat. En kod på 128 eller mer söks upp i en tabell, och terminalen skickar i stället den sträng som står där, så att funktionstangenterna kan programmeras. Finns ingen tabell eller inte koden i den blir det fel 53. Det här är läst ur ROM:en men inte prövat.

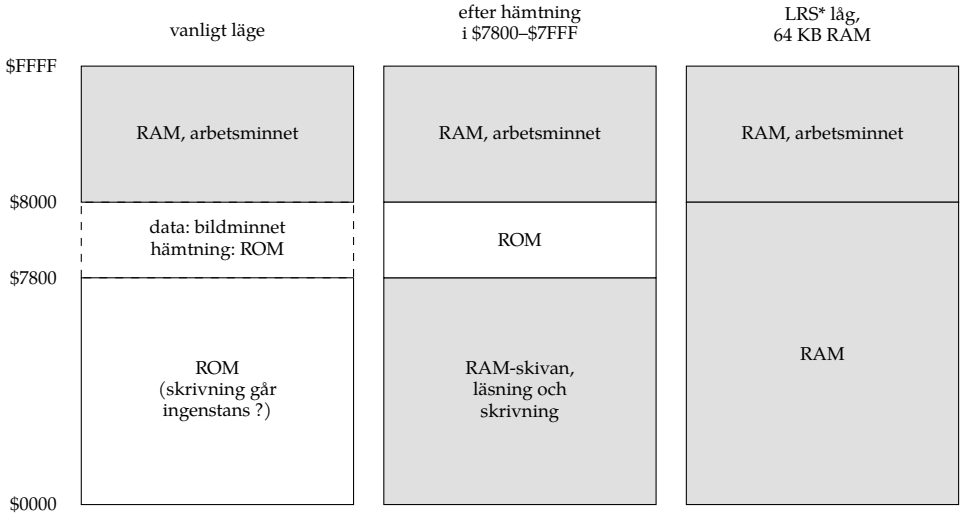
Minnet

Där inget annat anges kommer uppgifterna i det här kapitlet från Luxors servicemanual för ABC 802 (ABC 802 Servicemanual), och sidnumren inom parentes avser den. Ritningen av minnesdelen, datorkortets blad 4, finns i det finska arkivet devili.iki.fi. Det som sägs om rutinerna är läst ur ROM:en.

5.1 Tre minnen på samma adresser

Z80:an når 64 kilobyte, men ABC 802 har mer än så: 32 kilobyte ROM, 64 kilobyte dynamiskt RAM och 2 kilobyte bildminne, 98 kilobyte i allt. Det övre RAM:et, `$8000`–`$FFFF`, är alltid arbetsminnet. Under `$8000` ligger tre saker på samma adresser: ROM:en, det nedre RAM:et och, på de sista 2 kilobyten, bildminnet (s. 18). Kartan finns i kapitlet Maskinen (se s. 13). Det här kapitlet handlar om hur maskinen väljer bland dem.

Manualen kallar det nedre RAM:et datalagringsminnet, och BASIC:en når det som enheten MEM:. De olika områdena inkräktar inte på varandra, skriver manualen, eftersom ABC 802 går över i ett särskilt läge när kassett- och datalagringsminnet adresseras. När CP/M laddas går datalagringsminnet åt till det (s. 18). Det är alltså tre lägen, och figuren visar vad processorn når i vart och ett.



De tre lägena under `$8000`. Det vanliga läget, läget efter en hämtning i `$7800`–`$7FFF` och läget med 64 kilobyte RAM, där DART:ens DTR i kanal B har satt LRS* låg. Det övre RAM:et är detsamma i alla tre. Mittkolumnen är härledd ur ritningen och stöds av ROM:en.*

5.2 ROM:en

ROM:en är fyra kretsar 2764 om 8 kilobyte, på platserna 9F, 11F, 12F och 14F. De tre första heter BASIC 0, 1 och 2 på ritningen och har BASIC II. Den fjärde heter DOS/OPT. Dess första halva har DOS:en och den andra tillvalen: skrivar- och V24-rutinerna, terminalen och kassetten med MEM: (s. 48; ritningen, blad 4).

Adressbitarna A0–A12 går till alla fyra. A13–A15 går till en avkodare 74LS139, som manualen kallar 18H men ritningen 8H, och dess fyra utgångar väljer kretsarna: 0–8K, 8–16K, 16–24K och 24–32K. A15 är avkodarens enable, så den svarar bara under `$8000`. Kretsarnas utgångar öppnas av ROME*, och databitarna går ut på bussen genom bufferten 81LS95 på 14G, som öppnas av MRD*, minnesläsning (s. 48; ritningen, blad 4).

ROME* kommer från en 74LS55 på 9C, två AND-grindar med fyra ingångar vardera och en NOR efter. Den högra grinden ger ROME* för `$0000`–`$77FF` och den vänstra för `$7800`–`$7FFF`. Den högra har A15 inverterad på en ingång, så den är stängd över `$8000`. Övriga ingångar kommer från en låskrets på 7D, en multiplexer på 8D och signalen LRS* (s. 49). Det är dessa tre som bestämmer läget, och de beskrivs nedan.

5.3 Det dynamiska RAM:et

RAM:et är åtta kretsar på 64 kilobit, en bit i varje, på platserna 11A–18A. Manualen kallar dem 4864, och ritningen anger Hitachis HM4864-3 eller motsvarande. Varje krets har hela adressrymden för sin bit, så det är först adressen som delar RAM:et i två block om 32 kilobyte (s. 46; ritningen, blad 4).

Adressen läggs in i två steg, först raden med RAS* och sedan kolumnen med CAS*. Manualen säger att kretsarna bara har sju adressingångar, men ritningen visar åtta, på ben 5, 7, 6, 12, 11, 10, 13 och 9, och åtta behövs: 256 rader gånger 256 kolumner är 65 536 bitar. Två multiplexrar 74LS258A väljer vad som går dit. Med väljaringången hög går A8–A15 ut som rad, och när den går låg går A0–A7 ut som kolumn. Manualen kallar multiplexrarna 14B och 15B, men på ritningen heter de 4B och 5B (s. 46; ritningen, blad 4).

RAS* är MRQ* genom en buffert. Väljarsignalen ADR SEL kommer från en vippa i en 74ALS175 på 11B, som har BMRQ som insignal och klockas av F3, en signal med samma fas och frekvens som processorns klocka. När BMRQ har gått hög dröjer det ungefär 80 nanosekunder innan vippan slår om och ADR SEL går låg. Sedan går signalen genom NOR-grinden på 9D och NAND-grinden på 12D, en 74LS132, och fördröjs i ett RC-nät, 220 ohm och 47 pikofarad. Tidskonstanten blir ungefär 10 nanosekunder, så att kolumnadressen hinner bli stabil. Sist kommer en NAND-grind 74F10, som ger CAS* (s. 46; ritningen, blad 4). Manualen kallar den 13B, men ritningen 3B och RAM DIS. Manualen och ritningen skiljer sig med en etta på flera ställen, 14B, 15B, 13B och 18H mot ritningens 4B, 5B, 3B och 8H. På kortet sitter kretsarna där manualen säger (se fotot och skissen av kortet, s. 8), så det är ritningens platsnamn som avviker. Ritningen gäller samma kort, 55 20892-01, och inte en annan utgåva. Platsnamnen står där i små svarta ringar. På 11A syns ettan som ett eget streck innanför ringen, men på 4B och 5B är ringens vänstra kant bara tjockare, och RAM-kretsarna bredvid 11A heter på samma sätt 2A, 3A, 4A och så vidare. Ettan har alltså gått upp i ringens kant, på ritningen eller i kopian av den.

Fyra signaler kan hindra CAS*: ROME*, LXM*, RFSH* och CAS DIS*. ROME* gör det när ROM:en läses, genom en annan låskrets i 7D. LXM* gör det när ett kort på bussen har tagit över minnet, se nedan. RFSH* gör det under uppfriskningen och CAS DIS* när bildminnet adresseras (s. 46–47).

Uppfriskningen sköter Z80:an själv. I den senare delen av varje hämtning lägger den registret R på adressbussen och sätter RFSH* och MRQ* låga. RAS* går då, men inte CAS*, och kretsarna friskar upp den raden. R räknar sju bitar, 128 rader, och manualen anger BA0–BA6 (s. 47), fast den på sidan innan skriver A0–A7. Uppfriskningen fortsätter också när ett externt minne används

(s. 47).

Om det ska läsas eller skrivas avgör RD*, som går till kretsarnas WR* genom en låskrets i 11B. En annan vippa i samma krets delar F3 med två och ger 1,5 MHz till CTC:n (s. 47).

5.4 Hämtning eller data

På `$7800`–`$7FFF` ligger både ROM:ens sista 2 kilobyte och bildminnet. Maskinen skiljer dem åt genom att se på vad processorn gör. När den hämtar en instruktion där, är det ROM:en som svarar. När den läser eller skriver data där, är det bildminnet (s. 49). Det är därför det inte går att köra kod i bildminnet.

Multiplexern på 8D är en 74LS158 med inverterade utgångar. Dess enable är samma signal som väljer DOS/OPT-kretsen, så den är bara aktiv på `$6000`–`$7FFF`, och annars är alla dess utgångar höga. Väljaringången kommer från NAND-grinden på 12C, en 74ALS10, med A11, A12 och RFSH* som ingångar. Den är låg på `$7800`–`$7FFF` och hög på `$6000`–`$77FF`. Med RFSH* låg är utgången hög, som på `$6000`–`$77FF`, så en uppfriskning tas aldrig för en adress på `$7800`–`$7FFF`, vad registret I än innehåller. BASIC:en sätter I till `$FF` vid starten (`$0096`), så där skulle det inte hända ändå, men ett annat program kan sätta I lägre. Utgången 3Y, ben 9, är därmed låg bara på `$7800`–`$7FFF` (s. 49; ritningen, blad 4).

3Y går till D-ingången på låskretsen 7D, en 74LS375. Låskretsen öppnas av NOR-grinden på 9D, en 74ALS02, med M1* och F3 som ingångar. Den följer alltså adressen bara under hämtningar, och håller sedan värdet till nästa hämtning. När processorn hämtar på `$7800`–`$7FFF` låser den en nolla, Q*-utgången går hög och den vänstra AND-grinden i 9C ger ROME*. Låskretsen är öppen redan under hämtningen, så det är själva hämtningen som går till ROM:en. Hämtar processorn någon annanstans låser den en etta. Då är Q hög, och vid en läsning eller skrivning på `$7800`–`$7FFF` ger multiplexerns utgång 4Y, ben 12, signalen VR*, som väljer bildminnet (s. 49).

Manualen beskriver bara de två fallen på `$7800`–`$7FFF`. Men ritningen visar mer. Q från 7D går också till den högra AND-grinden, den för `$0000`–`$77FF`. När den senaste hämtningen var i `$7800`–`$7FFF` är Q låg, och då ger den grinden inte ROME*. CAS* hindras inte, och RAM:et svarar på `$0000`–`$77FF`, både vid läsning och skrivning. Det är det särskilda läget som manualen talar om (s. 18), mittkolumnen i figuren. Under tiden är `$7800`–`$7FFF` ROM också för data, eftersom den vänstra grinden ger ROME* oberoende av M1*, och CAS DIS* stänger RAM:et där.

Det följer av ritningen att det nedre RAM:et i det läget bara nås på `$0000`–`$77FF`, 30 kilobyte. De sista 2 kilobyten under `$8000` nås bara med 64 kilobyte RAM. ROME* har ingen ingång för RD*, så i det vanliga läget hindras CAS* också vid skrivning på `$0000`–`$77FF`, och det som skrivs dit försvinner.

5.5 LDIR och RET på `$7FFD`

ROM:en har tre byte för det här allra sist, på `$7FFD`–`$7FFF`: `LDIR` och `RET`, `$ED` `$B0` `$C9`. Listningen kallar rutinen `LDIR7F`. Manualen beskriver den som rutinen som tillsammans med speciell hårdvara kopplar in och ur minnet, och ger ett exempel som flyttar 2 kilobyte från `$C000` till `$0000` (s. 50):

```
LD  HL,0C000H    ; varifrån
LD  DE,00000H    ; vart
LD  BC,0800H     ; hur många byte
JP  07FFDH       ; LDIR och RET
```

Hoppet gör att `RET` återvänder till den som anropade rutinen. Med `CALL` i stället återvänder den till instruktionen efter.

Det fungerar för att `LDIR` hämtas på nytt för varje byte. Instruktionen flyttar en byte, och är BC inte

noll backar den programräknaren två steg och börjar om. Varje varv hämtas alltså `$ED` och `$B0` från `$7FFD` och `$7FFE`, låskretsen 7D låser en nolla varje gång, och hela flytten sker i det särskilda läget. `RET` hämtas från `$7FFF`, i samma område, och läser återhoppsadressen från stacken i det övre RAM:et. Den första hämtningen efter återhoppet sätter maskinen i det vanliga läget igen. Om ett avbrott kommer mitt i flytten, hämtas avbrottsrutinen utanför området och går i det vanliga läget, och när den återvänder till `$7FFD` slår låskretsen om igen. Z80:an tar nämligen emot avbrott mellan två varv i `LDIR`, och då har programräknaren redan backats till instruktionens början. Det är `$7FFD` som läggs på stacken, och efter återhoppet hämtas `$ED` därifrån igen (Z80 CPU User Manual, Zilog).

Rutinen flyttar åt båda hållen, mellan det övre RAM:et och `$0000`–`$77FF` i det nedre. Den läser också ROM:en på `$7800`–`$7FFF`, som annars döljs av bildminnet när den läses som data. ROM:en har två sådana tabeller strax före `LDIR7F`: `V24DEF` på `$7FE0` och `PRDEF` på `$7FEB`, 11 byte vardera med texten VSA36B7255A, standardinställningen för V24 och skrivaren. `GETDEF` på `$7B11` kopierar dem med `LDIR7F`. Bildminnet nås däremot inte alls under flytten.

5.6 64 kilobyte RAM

Det tredje läget styrs av signalen LRS*, Low RAM Select. Den går till ben 2 och 12 i 9C, en ingång på var och en av AND-grindarna. När LRS* är låg ger ingen av grindarna ROME*, ROM:en kopplas bort och alla 64 kilobyte är RAM. Bildminnet kopplas också ur (s. 49). På ritningen går LRS* dessutom till multiplexerns ingång 2A, så att CAS DIS* från 2Y inte längre stänger RAM:et på `$7800`–`$7FFF`. Samma CAS DIS* går tillsammans med VR* och LXM till en NOR-grind, 74ALS27 på 11C, och grindens utgång går till ingången CLR* (ben 13) på vippan 8C, som släpper fram bildminnet (se s. 17). Så länge någon av de tre är hög hålls vippan nollställd. När LRS* är låg är CAS DIS* hög, så vippan släpper aldrig fram bildminnet, och det är så bildminnet kopplas ur (ritningen, blad 2 och 4).

LRS* kommer från DTR*-utgången i DART:ens kanal B, ben 25. Kanal B tar annars emot tangentbordet, och DTR och RTS behövs inte för det. De styrs från Write Register 5 genom kontrolladressen `$23` (s. 50):

Bit	Funktion	Värde
D0	Tx CRC enable	0
D1	RTS	0 ger 80 tecken per rad, 1 ger 40
D2	SDLC/CRC16	0
D3	Tx enable	1
D4	send break	0
D5, D6	ordlängd	1, 1
D7	DTR	0 ger 32 kilobyte ROM, 1 ger RAM

En etta i registret ger en låg utgång, så D7 = 1 sätter DTR* och därmed LRS* låg. BASIC:en skriver Write Register 5 vid starten med `$68` från `WR5INI` på `$0181`, D3, D5 och D6, och lägger till D1 för 40 tecken (`$0428`–`$043A`). D7 är alltså noll och ROM:en inkopplad.

Ett program som sätter D7 tar bort ROM:en under sig, så det måste ligga i det övre RAM:et när det gör det. Det är så CP/M får plats: manualen säger att datalagringsminnet åtgår när CP/M laddas (s. 18), och CP/M har ingen användning för BASIC:en. Mer om det i kapitlet om CP/M (se s. 55).

5.7 Externt minne

Ett kort på bussen kan ta över minnet genom att sätta XM* låg i busskontakten. Signalen låses i den andra halvan av 7D och blir LXM och LXM*. LXM* hindrar CAS*, så att RAM:et inte svarar, och hindrar MRD*, så att ROM:en på `$0000`–`$7FFF` kopplas bort. Uppfriskningen fortsätter ändå (s. 47). LXM går också vidare till I/O-avkodningen, där den blir flaggan XMEMFL genom JK-vippan 12B och multiplexern 16B (s. 42–43). Ledningarna finns i ABC-bussen sedan ABC80, med XM* på stift 28 och XMEMFL* och XMEMWR* på stift 4 och 3. Där använde till exempel Luxors RAM-kort på 16 kilobyte för ABC80 dem, för minne på `$8000`–`$BFFF` (MAME). Något kort för ABC 802 som tar över minnet på det sättet har inte gått att hitta.

5.8 MEM:

Från BASIC når man det nedre RAM:et som enheten MEM: (s. 50). Manualens exempel lagrar 256 byte:

```
10 OPEN "MEM:" AS FILE 1
20 FOR I=0 TO 255  :! Lagra 256 byte i MEM:
30  PUT ?1 CHR$(I)
40 NEXT I
50 CLOSE 1
```

Enhetens rutin `MEMDRV` ligger på `$7800`, först i kassettrutinernas område. Den tar emot öppning, läsning och skrivning. Vid öppningen läser `MEMOPN` siffrorna i filnamnet fram till första mellanslaget och sparar talet som filens nummer. Det får vara högst 255, och ett tecken som inte är en siffra ger fel 21. Ett namn utan siffror ger nummer 0. Radlängden blir 132 tecken. Bufferten sätter `MEMARE` på `$FC00`, och den flyttar ned stacken under `$FC00` om den ligger högre.

RAM:et delas i sidor om 256 byte, och ett block på 256 byte hamnar på en sida. `MEMPAG` räknar ut sidan som filens nummer plus blockets nummer, och ger fel 41 om sidan blir högre än `$77`. Det är just `$0000`–`$77FF`, den del som nås i det särskilda läget. `MEMWR` skriver ett S och blockets nummer, två byte, först i bufferten, och kopierar de 256 byten till sidan med `LDIR7F`. `MEMRD` kopierar tillbaka, och om det inte står S och rätt nummer först ger den fel 37. Det blir 253 byte data per block och 120 sidor, drygt 30 000 byte.

Filens nummer är alltså bara den första sidan. Två filer med nummer som ligger närmare varandra än antalet block går in i varandra, och det är upp till programmet att hålla isär dem.

`MEMDRV` ligger själv i området, och dess hämtningar sätter alltså det särskilda läget. Den hade alltså kunnat flytta med en egen `LDIR`, men anropar ändå `LDIR7F`. Varför framgår inte av listningen. Det särskilda läget förklarar också manualens ord om kassetten: kassettrutinerna ligger på samma 2 kilobyte och går därför också i det läget (s. 18).

Del II

Programvaran

BASIC II i ABC 802

6.1 Samma tolk, ny översättning

BASIC II i ABC 802 är samma tolk som i ABC 800 M. Hur den arbetar, med raderna som översätts vid inmatningen, variablerna, flyttalen och felhanteringen, står i *ABC80 inifrån*, band 2, i kapitlen BASIC II jämförd med ABC80-BASIC:en och BASIC II på ABC80 och ABC 800. Det här kapitlet tar bara upp det som skiljer ABC 802 från ABC 800 M. Uppgifterna kommer från de två ROM:arna, där inget annat anges.

I BASIC II för ABC80 är tolken ABC 800 M:s byte för byte, och ändringarna är gjorda på plats. ABC 802:s tolk är i stället översatt på nytt från källkoden. Bara 9 796 av ROM:ens 32 768 byte är lika på samma adress som i ABC 800 M, men 25 624 byte finns där i block om minst 8 byte, på andra adresser. Koden är densamma men har flyttats, och skillnaden växer och krymper genom tolken där något har lagts till eller tagits bort.

Tolken börjar 163 byte lägre än i ABC 800 M, eftersom kassetten inte längre ligger i det låga området (se avsnittet Det låga området, s. 42). Nyckelordstabellen för funktionerna, `FNHDR`, ligger på `$0661` och för satserna, `STHDR`, på `$088B`. Hopptabellerna efter dem har samma ordning men nya adresser. Sedan minskar skillnaden i steg till 61 byte. Vid `$3B2C` ligger `SYS`, som i ABC 800 M ligger på `$44BF`, och därefter ligger ABC 802:s kod 19 och sedan 24 byte högre. Där `SYS` låg i ABC 800 M byter skillnaden tecken, och från `$4565` ligger koden 42 byte lägre än i ABC 800 M. De 42 byten fylls till sist av satsen `WIDTH` på `$4BF6`–`$4C1F`, så att `$4C20`–`$7002` ligger på samma adress i båda: slutet av tolken och DOS:en på `$6000`–`$6FFF`, sammanlagt 9 187 byte. DOS:en är version 20, byte för byte densamma som i ABC 800 M (se kapitlet DOS och skivminnet, s. 45). Skillnaderna däremellan är inte bara flyttade adresser. Jämförda instruktion för instruktion, med adresserna i ROM:en räknade som lika när de pekar på motsvarande ställe, skiljer sig de två tolkarna på ett 90-tal ställen. På 16 av dem har ABC 802 bytt fyra instruktioner som läser eller skriver ett ord genom HL, till exempel `LD (HL),C`, `INC HL`, `LD (HL),B` och `INC HL`, mot ett anrop av en rutin som redan fanns i ABC 800 M, `LDBCHL`, `LDDEHL`, `STBCHL` eller `STDEHL`. Det sparar en byte varje gång, och ABC 802 har 28 sådana anrop mot ABC 800 M:s 17. Tre `JP` har blivit `JR`. Några rutiner är omskrivna, bland dem jämförelsen på `$0FE4`, `$1791`–`$17E2` och `$1D7F`–`$1DAE`. Det ser ut som att koden har krympts för att ge plats åt det nya, `SYS` och `WIDTH`.

Nyckelorden är desamma som i ABC 800 M, med `SET DOT`, `CLR DOT`, `DOT` och `TXPOINT`. Det enda nya är `WIDTH`. Det som fattas är ABC 800 M:s satser för det högupplösta grafikkortet, `FGPOINT`, `FGLINE`, `FGFILL`, `FGCTL` och `FGPAINT`, som där ligger i tillägget på `$7000`.

6.2 Det som syns

Ledtexten är `ABC802` i stället för `ABC800`, och felutskriften slutar på `ABC802 in line`. Texten ligger på `$3D5B`, i ABC 800 M på `$3D46`. På adress 39 står maskinens kod: `PEEK(39)` ger 3 i ABC 802, mot 10 i ABC 800 M:s ROM.

Starttexten på `$0519` lyder ”**FB T{by 1983-03-31”, följd av sex BEL. I ABC 800 M står på samma sätt ”**JF T{by 1982-05-11”, och JF är Johan Finnved. Vem FB är har inte gått att få fram; varken handböckerna, DIAB:s årsredovisningar för 1982–84 eller sökningar på nätet nämner någon med

de initialerna. Täby var DIAB:s ort. Som i ABC 800 M refererar ingen kod till texten.

SYS har fått ett värde till. SYS(8) är −1 så länge en tangent är nedtryckt och läser signalen Key Down på DART:ens ingång DCD (KEYDN på $3B70). Resten av SYS är som i ABC 800 M, och SYS(7) och SYS(9) ger fortfarande fel. Tangentbordet och SYS(5), SYS(6) och SYS(8) står i kapitlet Tangentbordet (se s. 33).

6.3 WIDTH

Manualen beskriver satsen så här (BASIC II-manualen s. 61):

```
WIDTH [# filnummer,] antal
```

Den anger hur många tecken det får plats på en rad i en fil, och utan filnummer gäller den bildskärmen, med 40 eller 80 tecken. Manualens exempel är WIDTH 40 och en fil med 64 tecken per rad.

WIDTH är inte invävd i tolkens tabeller. Den ligger sist i tolken, på $4BF6–$4C1F, med en egen liten nyckelordstabell som börjar med en länk till den vanliga, STHDR på $088B. Vid starten lägger RAMINI adressen $4BF7 i pekaren till satsernas nyckelord på $FF7D, så att tolken ser WIDTH först och sedan resten. Det är samma kedja som ABC 800 M:s tillägg använder för sina grafiksatser.

Rutinen läser ett eventuellt # och filnummer (CFILNR), letar upp filen (FNDFIL) och skriver antalet i filens radlängd, byte 8 i dess filblock. Utan filnummer är det fil 0, bildskärmen, vars filblock ligger på $FF4C. Radlängden hamnar då på $FF54, CONWID. Mer gör inte satsen. Det är CON: som ser efter i CONWID varje gång den skriver: SETW5 sätter högst 80 och väljer bildens läge, 40 tecken för 40 eller mindre och 80 tecken annars, och CURADR dubblar kolumnen i 40-teckensläget. Hur det går till i maskinvaran står i kapitlet Bildskärmen (se s. 24).

Därför går också andra tal än 40 och 80. WIDTH 60 ger 80-teckensläget men en ny rad efter 60 tecken, och WIDTH 20 ger breda tecken med 20 på varje rad. Det som redan står på skärmen ändras inte, men vid ett byte mellan 40 och 80 tecken visar bilden det på ett annat sätt: i 40-teckensläget syns bara de jämna byten.

6.4 Det låga området

Före tolken, på $0000–$061F, ligger det som hör till maskinen: starten, RST-vektorerna, CON: med skärmen och tangentbordet, avbrotten och klockan. Det mesta är skrivet för ABC 802, och bara 114 byte är lika på samma adress som i ABC 800 M. Skärmen och tangentbordet står i sina egna kapitel.

Grafiken ligger på $0475–$0518: DOT, SET DOT, CLR DOT och TXPOINT, som sats och som funktion. Satserna finns också i ABC 800 M, men rutinerna i ABC 802 är egna; ingen bit av dem finns i ABC 800 M:s ROM. Vad de gör står i kapitlet Bildskärmen (se s. 25).

I ABC 800 M ligger kassetten CAS: på $044A–$06AC. I ABC 802 har den flyttats till tillägget på $7000, och efter starttexten är $0536–$05FF tomt.

6.5 Tillägget på $7000

Den sista kretsens andra halva, $7000–$7FFF, har enheterna som BASIC:en når utöver CON: och skivminnet. DEVLNK på $7021 är densamma som i ABC 800 M och länkar in dem i enhetslistan. Kedjan blir CAS:, MEM:, V24:, PR:, NUL: och CON:.

- PR:, skrivaren, på $7003–$7365. Starten PRINIT och rutinerna för utskriften är ABC 802:s egna.
- V24:, serieporten, och terminalprogrammet är ABC 800 M:s. Rutinerna ligger delvis 147 byte lägre än där, på $7086–$725F, delvis 188 byte högre, på $74B5–$77BA, där terminalen startas med ”LOAD V24:”, och delvis 449 byte högre, på $79C1–$7E62.
- CAS:, kassetten, är ABC 800 M:s från det låga området, flyttad hit. Delar av den ligger på $736D–$74B4 och resten, CASDRV, på $78A9.
- MEM:, RAM-skivan, på $7800 (se kapitlet Minnet, s. 39).

ROM:ens sista byte är också desamma som i ABC 800 M. Standardinställningarna står som text, V24DEF på $7FE0 och PRDEF på $7FEB, båda ”VSA36B7255A”, och på $7FFD står LDIR och RET, LDIR7F. Rutinen fanns alltså redan i ABC 800 M.

Hälften av tillägget, $7800–$7FFF, ligger på samma adresser som bildminnet. När kassetten och MEM: arbetar går maskinen därför över i det läge där den hämtar ur ROM:en men läser och skriver i det nedre RAM:et (se kapitlet Minnet, s. 37).

6.6 Adresser

Tabellen visar var ABC 802:s ROM finns i ABC 800 M:s. Kolumnen Flyttad är adressen i ABC 800 M minus adressen i ABC 802, som i band 2:s tabell. Där det står ”olika” har ROM:arna olika innehåll, och där det står ”ändrad” är det samma sak med nya adresser i. Gränserna är ungefärliga, några byte hit eller dit.

ABC 802	ABC 800 M	Flyttad	Innehåll
$0000–$061F	$0000–$06AC	olika	starten, CON:, avbrotten, grafiken; i ABC 800 M också CAS:
$0620–$0A21	$06C3–$0AC4	+163	tolkens början, FNHDR, STHDR, nyckelorden
$0A22–$0C5E		ändrad	hopptabellerna
$0C5F–$1DB3		+163 till +81	tolken, i flera steg
$1DB4–$27CB	$1E05–$281C	+81	tolken
$27CC–$35CA		+79 till +74	tolken
$35CB–$3B2B	$3608–$3B68	+61	tolken, klockan
$3B2C–$3B7B	$44BF–	olika	SYS, med SYS(8)
$3B7C–$4056	$3B69–$4043	−19	tolken
$4057–$4396	$403F–$437E	−24	tolken
$4397–$4564		olika	här låg SYS i ABC 800 M
$4565–$4BF5	$458F–$4C1F	+42	tolkens slut
$4BF6–$4C1F		ny	WIDTH
$4C20–$7002	$4C20–$7002	0	tolken, DOS:en
$7003–$7020		olika	PRINIT
$7021–$7054	$7021–$7054	0	DEVLNK
$7055–$74B4		olika	PR:, delar av V24: och CAS:
$74B5–$77BA	$73F9–$76FE	−188	terminalen
$77BB–$77FF		olika	tabeller
$7800–$79C0		olika	MEM:, CASDRV

ABC 802	ABC 800 M	Flyttad	Innehåll
$79C1–$7E62	$7800–$7CA1	−449	V24:
$7E63–$7FDF		olika	terminalens skärm och tangenter
$7FE0–$7FFF	$7FE0–$7FFF	0	inställningarna, LDIR7F

DOS och skivminnet

7.1 Samma DOS som i ABC 800 M

Den första halvan av den fjärde ROM-kretsen, `$6000`–`$6FFF`, är DOS:en. Den ger BASIC:en filer på skivminnen som sitter på ABC-bussen. Den är byte för byte densamma som i ABC 800 M:s ROM, Luxors DOS v.20. Signaturen på `$6FA4` lyder ”GN T{by 1984-04-03”, och på `$606F` står versionsnumret: `PEEK(24687)` ger 20.

ABC80 inifrån, band 2, beskriver samma DOS i den något äldre v.19 från 1984-03-02, som finns i BASIC II för ABC80. Kapitlet BASIC II-ROM:en går igenom DOS:en från avsnittet DOS:en på $6000 till och med avsnittet BYE, KILL, NAME och UNSAVE: hopptabellen, tabellerna i RAM, hur en post hittas, kanalerna, katalogen, starten med BASICINI.SYS, feltexterna och drivrutinen. Hur DOS:en läser och skriver en sektor, hur skivan är ordnad och hur kanalerna fungerar står i kapitlet UFD-DOS v.20, för samma DOS i ABC80-BASIC:en. Det här kapitlet upprepar inte det. Det tar upp hur skivminnet ansluts till ABC 802, enheterna, starten och minnet, och hur DOS:en delar minne med MEM:.

Mellan v.19 och v.20 skiljer sig bara två ställen (band 2, kapitlet BASIC II på ABC80 och ABC 800, avsnittet DOS). I v.20 anropar `SETREC` inte längre `RECLFT` och `RECSEC` när postnumret har lagrats. Felet vid `KILL` är också rättat, så att felindexet hamnar i A och inte i C. Den första ändringen tar 8 byte. Därför ligger allt från `$646A` och uppåt 8 byte lägre i ABC 802 än band 2:s adresser, medan allt under `$6462` har samma adress. Tabellen visar de adresser som behövs i det här kapitlet:

Namn	ABC 802	band 2 (v.19)	Uppgift
`DOSINI`	`$6000`	`$6000`	första platsen i hopptabellen, går till `DOSSTA`
`DOSSTA`	`$6C64`	`$6C6C`	starten
`DOSRST`	`$6CDE`	`$6CE6`	alla kanaler lediga, felvektorerna nollade
`DOSDRV`	`$6D85`	`$6D8D`	drivrutinen för alla DOS-enheter
`OPNINI`	`$6DF2`	`$6DFA`	öppnar BASICINI.SYS
`DOSDEV`	`$6E2D`	`$6E35`	enhetsposterna
`CRDTAB`	`$6ED3`	`$6EDB`	korttabellen
`DOSRUN`	`$6F81`	`$6F89`	körning av `BYE`, `KILL` och `NAME`
`DOSSTH`	`$6F9A`	`$6FA2`	satstabellens huvud

7.2 Skivminnet på ABC-bussen

ABC 802 har ingen skivenhet inbyggd. Skivminnet ansluts till ABC-bussen, den 64-poliga Europakontakten, som har plats för ett kort (se s. 14). Där sitter kontrollerkortet. Det är en egen dator med en Z80, en PROM med styrprogrammet och en FD1791 mot drivenheterna, och DOS:en talar med det genom portarna 0–4 på bussen. Kortet och samtalet på bussen står i band 2, kapitlet Skivminnet. I portkartan är det adresserna `$00`–`$07` (se s. 13).

Vilket kort DOS:en väljer avgörs av korttabellen CRDTAB. Den är densamma som i v.20 för ABC80 (band 2, kapitlet Skivminnet, avsnittet Kortnummer och korttyper):

```
namn  kort  kortbyte  skivenhet
MF    44    $6C $02   ABC 832, 5¼ tum, 80 spår
MO    45    $AD $00   ABC 830, 5¼ tum, 40 spår
SF    46    $6E $02   ABC 838, 8 tum
HD    36    $24 $05   hårddisk
DR          $08 $00   pekar på MF
```

DR är det namn som kan pekas om. Från början pekar det på MF, kort 44. DR0: är alltså första drivenheten i en ABC 832, och det är det namn BASIC II-manualen använder: ”Drivenhet 0 i flexskiveenheten betecknas DR0:” (BASIC II-manualen s. 10). Manualen nämner ingen modell av skivenhet. För autostarten hänvisar den till bruksanvisningen för flexskiveenheten (s. 2).

7.3 Enheterna

DOSDEV på $6E2D–$6EB4 är 17 enhetsposter om 8 byte: länken, namnet i tre tecken, drivrutinen och enhetskoden. Alla har samma drivrutin, DOSDRV, och skiljs åt bara av koden:

```
"   "    $FF     enheten utan namn
DR0-DR2  $00-$02  UFD      $1E
MF0-MF2  $08-$0A  MO0-MO1  $0C-$0D
SF0-SF2  $10-$12  HD0-HD3  $04-$07
```

Bit 2–4 i koden väljer posten i korttabellen och bit 0–1 drivenheten på kortet. Den första posten, utan namn, gäller ett filnamn utan enhet. Då letar DOS:en efter filen på alla enheterna i tur och ordning (band 2, kapitlet BASIC II-ROM:en, avsnittet Starten och BASICINI.SYS).

Vid starten länkas posterna in först i enhetslistan (DEVLST, $FF7B), före enheterna i tillägget på $7000. Hela kedjan blir DOS:ens 17 enheter och sedan CAS:, MEM:, V24:, PR:, NUL: och CON: (se s. 43). DOS:en länkar också in satserna BYE, KILL och NAME och gör så att UNSAVE fungerar.

DOS:ens fel har numren 21–53, och BASIC II-manualen räknar upp dem, till exempel 42 ”Enheten ej klar” när ingen skiva sitter i eller luckan är öppen (BASIC II-manualen s. 101). Själv skriver ROM:en bara numret, till exempel `Error 21`. Texten på svenska, ”Hittar ej filen”, kommer från filen BASICINI.SYS på skivan, som DOS:en läser vid starten (se nästa avsnitt).

7.4 Hårddisken

Kort 36 och klustren

HD0:–HD3: är kort 36 i korttabellen, med kortbyten $24 $05. Bit 6–7 i den första byten är 0. Sektornumret går då till kortet som det står, och hela sektorn flyttas med INIR eller OTIR, 256 byte i ett svep. Den andra byten ger klustrets storlek, och 5 i bit 0–2 betyder 2 upphöjt till 5, alltså 32 sektorer eller 8 kilobyte. Bitkartan på sektor 14 har 1 912 bitar, en per kluster, så en hårddisk kan ha högst 1 912 kluster. Det är 61 184 sektorer eller 15 663 104 byte. En skiva på 10 megabyte har 40 960 sektorer och 1 280 kluster (band 2, kapitlet UFD-DOS v.20, avsnittet En sektor: SECIO).

Formatering med DOSGEN

Luxors skivminne ABC 850 har en winchesterdrivenhet, det som här kallas hårddisk. Innan DOS:en kan använda en ny hårddisk måste den formateras och ”dosgenereras” (Luxor service-information 1984). Det gör programmet DOSGEN.ABS på systemskivan, ”Ver 6.01, 1983-04-21”. Det startas från kommandotolken, efter BYE, och frågar så här (Luxors systemskiva):

```
-DOSGEN
** Disk generation **
   Ver 6.01, 1983-04-21
Enhet ? HD0:
Ska skivan formateras (F/-) ? F
Skivan i HD0: kommer att rensas helt
vid initieringen. Är det OK ? <J/N> J
ABSOLUT säker ?? <J/N> J
Skivan formateras !
Skivan testas !
```

DOSGEN talar med kortet som `SECIO` gör, med fyra byte till port 0, men med två kommandon som DOS:en själv aldrig skickar. `$10` formaterar: den andra byten är enheten plus 8, och kommandot skickas 256 gånger, med 0–255 i de två sista byten. `$80` provar en sektor och ger bara statusen, inga data.

Medan DOSGEN provar skriver det `Nu testas sektor` och numret. DOSGEN frågar aldrig hur stor skivan är. Det provar kluster för kluster upp till 1 912, och de kluster som ligger efter skivans slut går inte att läsa. De tas bort som trasiga, ett i taget: `Sektor 40960 är dålig - borttagen!` och så vidare upp till sektor 61 152. Till sist står antalet sektorer som DOS:en får använda, för 10 megabyte `40928 användbara sektorer.` Det första klustret, sektor 0–31, hör till systemet (Luxors systemskiva).

Ordningen på skivan

Skivan som DOSGEN lämnar är ordnad som en flexskiva, med hårddiskens kluster:

```
sektor 0-13   oanvända
sektor 14     bitkartan; på $EF-$FE räknarna för katalogen
sektor 15     en karta med bara kluster 0; sista byten 5
sektor 16-31  katalogen, 16 sektorer
sektor 32-33  SYSDIR.SYS i kluster 1
```

I bitkartan är kluster 0 och 1 i bruk, och så är alla kluster efter skivans slut, så att DOS:en aldrig tar dem. Sista byten i sektor 15, 5, är klusterstorleken som potens av två. Den andra posten i SYSDIR.SYS har volymnamnet, elva blanktecken, och sektornumren till bitkartan, sektor 15 och katalogen: 14, 15 och 16. På en flexskiva står där `$62`, `$63` och `$80`, i den form som `SECIO` räknar om.

`LIB HD0:` i kommandotolken visar nu bara `SYSDIR .SYS` och `40896 av 40928 sektorer lediga.` Sedan fungerar `SAVE`, `LOAD`, `RUN`, `PREPARE` och `OPEN` på HD0: som på DR0:.

Serienumret

Varje winchesterdrivenhet i ABC 850 har dessutom ett serienummer skrivet magnetiskt på skivan, samma nummer som på etiketten på enhetens baksida. Byts drivenheten ut läser programmet GETID.BAC numret på den gamla, och MARKHD.BAC (”Numrera winchester”, ver. 1.03) skriver de åtta tecknen på den nya. Det kan göras före eller efter formateringen och DOSGEN (Luxor serviceinformation 1984).

MICA:s hårddisk

Luxor var inte ensam om hårddiskar till ABC-bussen. I abc80.net:s förteckning över PROM:ar finns en ”DOS /Option till ABC802 med MICA winchester” (PROM-listan på abc80.net). Den är DOS v.19, samma som i BASIC II för ABC80, med fyra byte ändrade. På `$612B` och `$613E` står två `JR Z`, som väljer `OTIR` och `INIR` när bit 6–7 i kortbyten är 0. I MICA-PROM:en är båda ersatta med `NOP NOP`. Där flyttas alltså hårddiskens sektorer också en byte i taget, och före varje byte väntar `SECIO` på bit 0 i port 1, som för flexskivorna. MICA:s kontroller hann tydligen inte med 256 byte

i ett svep. Adresserna under $6462 är desamma i v.19 och v.20, så de två hoppen står på samma platser i ABC 802:s DOS.

7.5 Starten och minnet

Vid kallstarten letar EXTSCN efter ett JP först på varje sida från $6000 till $7700. På $6000 finns det alltid, och då körs DOSSTA ($6C64). Starten gör som i band 2: den kopierar enhetsposten utan namn, satstabellens huvud och korttabellen till RAM på $FDD0–$FE02 och länkar in enheterna och satserna. Sedan flyttar den stacken till $F500 och lägger DOS:ens buffertar ovanför, en sida om 256 byte per kanal. Till sist öppnar den BASICINI.SYS på vilken enhet som helst och läser in feltexterna och en eventuell startkod.

DOS:en frågar aldrig om något kort sitter på bussen. Den startar på samma sätt med skivminne och utan. Minnet för buffertarna, $F500–$FCFF, tas därför alltid, och pekaren till minnets topp, MEMTOP på $FF0E (TOP, 65294, i manualen), blir 61830 i båda fallen. BASIC II-manualen har två minneskartor, en ”utan flexskiveenhet ansluten” med BASIC:ens minne upp till $FD00 och en ”med flexskiveenhet ansluten” med åtta DOS-buffertar, DOSBUF0–DOSBUF7, på $F500–$FCFF (BASIC II-manualen s. 102–103). Med den här ROM:en gäller alltid den andra. Den första gäller om sidan $6000 inte börjar med ett JP. BASIC:en räknar MEMTOP ur stackpekaren när EXTSCN är klar, 630 byte under den ($00F2–$00FA). Utan DOS:en ligger stacken kvar under $FD00, inga buffertar tas, och MEMTOP blir 63662, 1 832 byte mer; så blir det i emulatorn med ROM:en ändrad på den enda byten $6000. I ABC 802 sitter DOS:en på samma krets som tillvalen på $7000, så det skulle kräva en annan DOS/OPT-krets.

Ovanför arbetsminnet ser det alltså ut så här (BASIC II-manualen s. 103):

```
$FF00-$FFFF  enkla variabler
$FD00-$FEFF  systemvariabler, också DOS:ens
$FC00-$FCFF  DOSBUF7, också MEM:s buffert
$F500-$FBFF  DOSBUF0-DOSBUF6
   -$F4FF  stacken och arbetsminnet
```

Det sista ger en följd för MEM:. Dess rutin MEMARE lägger bufferten på $FC00 (se s. 39), och det är DOS-buffert 7. Ett tillägg i manualen säger just det: när MEM: används kan sex filer samtidigt vara öppna mot flexskivan, och MEM: har själv bara en fil öppen. DOS:en söker en ledig kanal bland 1–7. Kanal 0 används inte för BASIC:ens filer, och kanal 7 delar sida med MEM:, så det blir kanalerna 1–6 kvar.

BYE stänger de öppna filerna och laddar kommandotolken CMDINT.SYS (BASIC II-manualen s. 26). Den är inte Luxors egen. På Luxors systemskiva för ABC 800 och ABC 802 skriver den ”Disc operating system” och ”Ver 6.02, 1983-04-21, Copyright 1982 Dataindustrier AB” (Luxors systemskiva). Kommandot BAS går tillbaka till BASIC.

7.6 ABC NET

ABC 802 är byggd för ABC NET, och för nätet byts DOS-ROM:en ut. LUXNET har en egen DOS-PROM. Där frågar varje anrop först nätet, och först om det inte finns något nodkort kör den lokala DOS:en. Den PROM:en och nätet står i nästa kapitel (s. 49).

ABC NET och LUXNET

8.1 Nätet

Luxor sålde ABC 802 som ”perfekt som beslutsfattardator och terminaldator”. Enligt produktbladet fanns hårdvaran för ABC NET inbyggd, medan programmet var ett tillval (Luxor, produktblad ABC 802). Nätet hette LUXNET och hade en egen bruksanvisning. Det här kapitlet bygger på den bruksanvisningen och på programmet i nätets DOS-PROM. Bruksanvisningen beskriver hur nätet byggs upp och installeras, men inte hur det fungerar inuti. Den hänvisar till en systemprogramhandbok, som inte har gått att hitta.

LUXNET är ett bussnät. Alla noder sitter på samma ledning, och den som vill sända lyssnar först efter om ledningen är ledig. Under sändningen jämför noden det den sänder med det som står på ledningen, och om de skiljer sig har två noder krockat. Då väntar noden en slumpvis vald tid och sänder igen. Protokollet är ett modifierat HDLC. Ledningen är en tvinnad parkabel på högst 500 meter eller en koaxialkabel på högst 1 000 meter, och den går med 500 kbit/s, kodad som Manchester II enligt V.11. Högst 32 datorer kan arbeta på nätet samtidigt, och upp till 50 linjeanslutningar kan sitta på ledningen. Varje dator har ett HDLC-nummer på 20 bitar, som centralen översätter till ett nodnummer när datorn loggar in (LUXNET 1984, s. 4–5, 19).

Nätet är inte jämlikt. Det har en central, till exempel ett skivminne ABC 850, med två kort i expansionslådan: en centraldator, enkortsdatorn 1005, som läser och skriver på skivorna, och en kommunikationsprocessor, kort 4004, med sändnings- och mottagningsbuffertar mot ledningen. Till centralen kan upp till fyra skrivare och ett modem anslutas (LUXNET 1984, s. 6–7, 12–13).

Varje dator, nod, ansluts med ett nodkort på ABC-bussen. Från nodkortet går en kabel med 25-polig D-sub till en linjeanslutning på väggen, som skiljer noden galvaniskt från ledningen med en transformator. Nodkortet innehåller enligt bruksanvisningen ”drivrutiner för kommunikationen på linjen och delar av datorns operativsystem (DOS)”, och ingen del av datorns minne behöver tas i anspråk för nätet (LUXNET 1984, s. 8–10, 19–20).

Datorn behöver också en annan DOS-PROM. Bruksanvisningen har en tabell över dem: ABC 800 tar ABC 6-XX, ABC 802 ABC 32-XX och ABC 806 ABC 66-XX. Den nya PROM:en sätts i stället för DOS-ROM:en. Hur det går till står i en handledning som följde med kretsen (LUXNET 1984, s. 14, tabell 1). I ABC 802 sitter DOS:en på den fjärde ROM-kretsens första halva, `$6000`–`$6FFF` (se kapitlet DOS och skivminnet, s. 45).

8.2 Nätets DOS-PROM

Två utgåvor av PROM:en för ABC 802 har bevarats, ABC802-dos-LUXNET-322n01 och 322n02. MAME kallar dem ”node 1” och ”node 2”. Kapitlet bygger på n02. De har samma satser och enheter, men n01 är 146 byte längre. På `$606F`, där v.20 har sitt versionsnummer, står 1 i n02 och 0 i n01. Varken PROM:en, nodkortets ROM eller nätprogrammen läser platsen, så det är troligen utgåvans nummer, på samma plats som versionen i v.20. Någon signatur som v.20:s finns inte, och `$7000`–`$7FFF` är desamma som i DOS v.20.

Hopptabellen på `$6000` har samma platser som i v.20, så att BASIC:en kan anropa DOS:en på samma sätt. Den lokala DOS:en finns kvar och läser filer på skivminnen på bussen, men mycket

av koden är omskriven. Att skriva en post (DWRREC), läsa och skriva bytevis i kanalens buffert (DBUFRD–DBUFW2) och katalogfunktionerna (DFNJP) går bara via nätet. Utan nät ger de `Error 200`. Några platser i tabellen är nya:

Adress	Namn	Uppgift
$6009	NETOUT	`OUT (C),A` och `JP NETBAK`: här tar nodkortet över, och här lämnar det tillbaka
$6027	DNET	nätfunktion $0B
$604B–$605A		tomma, hoppar till DNONE (RET)
$6060	DSELCD	frågar nätet (funktion $0C) innan kortet väljs
$6069–$607C	DOSADR	platser i RAM för programmen på skivan

DOSADR har samma ordning som motsvarande ställe i v.20: korttabellen, CRDRAM ($FDE2), DSYSUN ($FFF7), versionen och en byte 8 (0 i v.20). Därefter kommer adresser som bara finns här: kroken $FE02, $FD3D, $FD3F (det valda kortet), $FFFB, $FD19 och $FE05. Tabellen pekar ut de platser i RAM som ett program utanför ROM:en behöver. Nodkortets program läser den inte; det använder adresserna direkt. Det är nätprogrammen på skivan som läser tabellen, till exempel med `PEEK2(24685)` (se nedan, s. 53).

8.3 NETHK frågar nätet

Varje DOS-anrop börjar med att fråga nätet. Det gör rutinen NETHK på $6DC1, med ett funktionsnummer 1–$16 i L. Svaret kommer i Z-flaggan. Z betyder att ingen svarade, och då gör den lokala DOS:en jobbet. NZ betyder att nätet har gjort det.

NETHK går igenom tabellen NETCRD på $6E1E. Den har tre byte per nodkort: kortnumret, en port och ett värde. Där står två kort, kort 3 med värdet $40 till port 0 och kort $30 med $C0 till port 2, och en nolla sist. Den första posten stämmer med nodkortets ROM: $40 till port 0 kopplar in kortet, och kortet kopplar ur sig själv med 0 till samma port (se nästa avsnitt). Vad den andra posten gäller säger ingen av handböckerna, och MAME har inget kort med det numret. För varje post väljs kortet med `OUT (1),A`. Kortnumret sparas på $FD3F, och sedan hoppar NETHK till NETOUT med porten i C och värdet i A:

```
NETHK:  PUSH AF
        PUSH DE
        LD   DE,NETCRD
L6DC6:  LD   A,(DE)                  ; kortnumret
        OR   A
        JR   NZ,L6DCF
        POP  DE                      ; tabellen slut: ingen svarade
        POP  AF
        BIT  7,L                     ; Z
        RET
L6DCF:  LD   (VFD3F),A
        OUT  ($01),A                 ; välj kortet
        INC  DE
        LD   A,(DE)
        PUSH BC
        LD   C,A                     ; porten
        INC  DE
        LD   A,(DE)                  ; värdet
        INC  DE
        JP   NETOUT                  ; $6009: OUT (C),A, JP NETBAK
NETBAK: POP  BC
        BIT  7,L
        JR   Z,L6DC6                 ; kortet tog inte funktionen
        POP  DE
```

```
        POP  AF
        RES  7,L
        INC  L
        DEC  L
        JR   NZ,L6DED              ; datorn ska göra något
        DEC  L                    ; L = $FF, NZ: klart
        RET
L6DED:  CALL NETFN
        LD   L,$00                 ; resultatet till nätet
        JR   NETHK
```

Det intressanta är det som inte syns. Utan nodkort händer ingenting vid `OUT (C),A`. ROM:en hoppar vidare till `NETBAK`, bit 7 i L är fortfarande 0, och nästa kort prövas. När tabellen är slut kör den lokala DOS:en. Med ett nodkort tar kortet över just vid `OUT`-instruktionen. Kortet har ett eget RAM på `$6000`–`$6FFF`, och `OUT (0),$40` kopplar in det i stället för PROM:en. Instruktionen efter `OUT` hämtas då ur kortets RAM, och där står ett annat hopp. Hur det ser ut beskrivs i nästa avsnitt.

Svaret står i L. Är bit 7 noll tog kortet inte funktionen. Är bit 7 satt och resten noll är jobbet gjort, och `NETHK` returnerar med L = `$FF` och NZ. Är resten inte noll vill nätet att datorn ska göra något åt det, till exempel läsa en sektor på en egen skiva eller öppna en lokal fil. `NETFN` kör då den lokala funktionen L & `$7F`, och `NETHK` frågar nätet igen med L = 0. Nätet kan alltså låna DOS:ens egna rutiner, och det förklarar varför DOS:en finns kvar i PROM:en.

Funktionsnumren går att läsa ut ur anropen:

L	Funktion	L	Funktion
1	läs eller skriv en sektor	`$0C`	`DSELCD`, välj kort
2	`PREPARE`	`$0D`, `$0E`	läs bytevis
3	`OPEN`	`$0F`, `$10`	skriv bytevis
4	`CLOSE`	`$12`	`CD`
5	läs en post	`$13`	`FILESTAT #a,b`
6	skriv en post	`$14`	`FILESTAT`, när raden matas in
7	`SETEND` (`KILL`)	`$15`	`DOSDRV` 12 och uppåt, `FILESTAT`
8	katalogfunktion (`NAME`)	`$16`	lokal funktion 7
9	starten (`DOSSTA`)		
`$0A`	skrivarna i centralen		
`$0B`	`DNET` (`$6027`), ett meddelande		

8.4 Nodkortet

Nodkortets ROM har bevarats. Den är en 27128 på 16 kilobyte, märkt 55 21141-01 och daterad 85-04-12. Kortet har en egen Z80. Vid start hoppar dess processor till `$0100`, tömmer kortets RAM på `$6E00`–`$6FFF`, lägger stacken på `$6FC0` och arbetar med avbrott i läge 2. Ledningen sköts genom portarna `$7C`–`$7F`, troligen en seriekrets för HDLC. Det mesta av ROM:en är kortets eget program för ledningen.

I början kopierar kortet en avbild ur sin ROM till sitt RAM. Adressen och längden står sist i ROM:en, på `$3FFC` och `$3FFE`: 2 516 byte från `$3400` till `$6000`. Avbilden börjar med datumet och har sedan samma utseende som PROM:en på `$6009`, men med ett annat hopp:

```
          PROM:en             nodkortets RAM
6009:  OUT (C),A         OUT (C),A
600B:  JP  NETBAK         JP  $684F
```

`NETHK` har lagt AF, DE och BC på stacken och kör `OUT (0),$40`. Därmed byts `$6000`–`$6FFF` mot kortets RAM, och processorn hämtar `JP $684F` i stället för `JP NETBAK`. Där tar kortets kod tillbaka registren från stacken och väljer funktion efter L, 1–`$16`. Andra värden ger fel `$C8` (200) med C-flaggan satt. Det är alltså datorns egen processor som kör avbildens kod; kortets Z80 sköter

ledningen. När funktionen är klar lägger koden AF, DE och BC på stacken igen, sätter L = $80, väljer kort 3 och kör OUT (0),0 på samma plats, $6009. Kortet kopplas ur, PROM:en kommer tillbaka och processorn hämtar JP NETBAK. Samma adress används åt båda hållen, och det är bara instruktionen efter OUT som skiljer mellan de två minnena.

Vill kortet att datorn ska göra något lokalt sätter det L = $80 plus funktionen, till exempel $88. NETBAK kör den med NETFN, och NETHK anropar kortet igen med L = 0. Då gör kortets kod bara RET och fortsätter där den var, som en korutin.

Avbilden anropar BASIC:ens rutiner direkt: $001D, som räknar ut ett uttryck, $0063, som läser ett filnamn, och RST $18, som hoppar över blanktecken. De här ingångarna är desamma i BASIC II för ABC 800 och ABC 802. Kortet kan därför arbeta i alla datorerna i ABC 800-familjen, och det är bara DOS-PROM:en som skiljer. Avbilden använder också platserna i RAM direkt, till exempel $FD3F och kanalernas kedja från $FF4C. Att kortet bara kopplar bort $6000–$6FFF, och inte hela $0000–$7FFF med XM*, följer av att BASIC:en och $7000–$7FFF måste finnas kvar medan avbilden kör. Kortet får adressen på bussen och kan dra XM* låg bara när den ligger på $6000–$6FFF. Låskretsen 7D tar XM* i varje minnescykel, på samma sätt som ROME*, och då kopplas ROM:en och RAM:et bort bara för de adresserna (servicemanualen s. 46–47; se kapitlet Minnet).

Delar av avbilden är samma kod som i nätets DOS-PROM, i n02 på $6C97–$6D6C och i n01 på $6D20 (103 byte). Koden har alltså flyttats mellan PROM:en och kortet.

8.5 Enheterna och satserna

Enhetsposterna DOSDEV ligger på $6AEF, 29 stycken om åtta byte: länken, namnet, drivrutinen och enhetskoden. De lokala enheterna är DR0–DR3, HD0–HD3, MF0–MF3, MO0–MO3 och SF0–SF3, fyra av varje (v.20 har tre DR, MF och SF och två MO). Sedan kommer nätets enheter. MFD, PFD, UFD och LFD har enhetskoderna $1C, $1B, $1E och $1F och samma drivrutin DOSDRV som de lokala. Skrivarna i centralen heter RPR och RP0–RP2 och har en egen drivrutin, RPDRV, som frågar nätet med funktion $0A. Bruksanvisningen nämner RP0:–RP3: (LUXNET 1984, s. 13); RP3 finns inte i PROM:en.

Filerna i centralen är ordnade i bibliotek i ett träd. Överst står MFD, och under den ligger filer och UFD:er, som i sin tur kan ha filer och UFD:er. Varje konto är knutet till ett bibliotek, och när datorn startar frågar den efter konto och lösenord (LUXNET 1984, s. 11, figuren). UFD: är kontots eget bibliotek. Vad LFD och PFD står för säger varken bruksanvisningen eller DataSwedens handbok för ABC-net, som bara har UFD:, men de är två pekare till bibliotek till, som sätts på samma sätt som UFD: (se nedan, s. 53). Rutinen NETUNT översätter enheten: UFD:, PFD: och LFD: tar sin startsektor och sin enhet ur RAM, UFD: ur DSYSUN ($FFF7) och $FFF9, PFD: ur $FD19 och LFD: ur $FFFB. Är ordet 0 blir det Error 46. MFD: blir HD0.

PROM:en har sex satser, mot tre i v.20. BYE, KILL och NAME finns kvar. Dessutom finns AS, som bara används i NAME, CD och FILESTAT, som kan förkortas FS. CD tar en sträng och skickar den till nätet med funktion $12; den byter alltså bibliotek i centralen. FILESTAT skickar två uttryck med FILESTAT #a,b och funktion $13, eller utan # med funktion $15. Redan när en rad med FILESTAT matas in frågar PROM:en nätet, med funktion $14. Ingen källa beskriver vad FILESTAT gör, men nodkortets kod visar en del. Funktion $14 läser #a,b med BASIC:ens rutiner och ger fel $E3 om kommat saknas. Funktion $13 letar i BASIC:ens kedja av öppna kanaler, från $FF4C, efter kanalen med numret a. Funktion $15 skickar frågan till centralen. Satsen tycks sätta ett läge för en öppen fil snarare än lämna något tillbaka. Programmet PRATA.BAC av Kristoffer Eriksson från 1985 låter två noder samtala genom en gemensam fil i MFD:. Det ger FILESTAT #1,4+32 direkt efter PREPARE, fyller filen med nollor och ger sedan FILESTAT #1,0, innan filen får sina skydd och den andra noden kan öppna den. Vad bitarna 4 och 32 betyder säger programmet inte, och ingen av handböckerna nämner satsen. Handböckerna känner bara två skydd för en fil, skrivskydd och raderingsskydd, som sätts med REPROT (Anvisning UFD-DOS, avsnitt 3.22; DataSweden, ABC-

net). PRATA.BAC sätter de skydden för sig, efter FILESTAT #1,0, så bitarna gäller troligen något annat. Utan nät ger alla tre Error 200.

8.6 LIB och biblioteket

Luxors LIB 6.01 är skrivet för både skivminnena och nätet. Rubriken i programmet lyder ”Biblioteksprogram för alla diskenheter & RAM:” och ”Tillhör Systemdisk, UFD-DOS”. Det listar UFD: genom att läsa biblioteket som sektorer, på samma sätt som det läser en lokal skiva. Det finns ingen särskild väg för nätet. I funktionen FNUdesc läser LIB enhetskoden ur $FFF9 (PEEK(-7)) och bibliotekets första sektor ur DSYSUN (PEEK2(-9)). Är den 0 är ingen UFD: aktiv. Enheten väljs med POKE -767,30, där 30 är UFD:s enhetskod $1E. Sektorn läses med CALL(24678,Secnr), platsen $6066 i hopptabellen, och felet hämtas på $FD15 (PEEK(-747)).

Med nätet blir sektorläsningen funktion 1, och centralen får svara med sektorer som ser ut som en skiva. Sektorn före biblioteket ger antalet poster i varje sektor av biblioteket, och sektorerna 14 och 15 är klusterbitarna, varav LIB räknar ut hur mycket som är använt och ledigt. LIB får alltså sektorer i DOS:ens format, hur centralen än lagrade filerna. I ABC-net, DataSwedens nät för Facit DTC, ABC 80 och ABC 800, satt centralkortet i det centrala skivminnet, som kunde ha 5,25-tums- och 8-tumsskivor och Winchesterskivor. Enheterna hette som de lokala, MF0:, SF0: och HD0:, och UFD: var användarens bibliotek (DataSweden, ABC-net, bilaga A). Luxors UFD-DOS beskriver ett sådant bibliotek på en lokal skiva. Det är en fil som är indelad i filer med ett eget bibliotek, minst 18 sektorer, och det används som en extra enhet, UFD: (Anvisning UFD-DOS, ”Allmänt om UFD”). Programmet SYS i UFD-DOS kan dessutom märka en skivenhet som ”central”, för att vara kompatibel med ABC-net (avsnitt 3.23). I ABC-net var centralens skivor alltså vanliga DOS-skivor. LUXNET:s central behöver inte ha varit det, för L.BAC nedan visar att filerna där hade ägare och rättigheter, som DOS:ens format saknar.

Det finns ett annat LIB, L.BAC (”Library listing”, version X.19 från 1985), bland nätprogrammen nedan. Det läser biblioteket på ett annat sätt. Det öppnar enheten själv som en fil, OPEN "UFD:" AS FILE 1, och läser biblioteket som poster. Den första posten har bibliotekets karta, 16 byte som säger vilka av de 16 sektorerna som används, och en byte om klusterstorleken. Sedan kommer sektorerna med 16 namn à 16 byte, där de två första byten är filens första sektor (LDA) och namnet står på plats 4–14. För varje fil öppnar L.BAC därefter skivan i centralen och läser filens *RIB*, sektorn som beskriver filen. Där står storleken (plats 234), gruppen (236), ägaren med åtta tecken (238), tiden (246–251) och skydden (253). Skydden är åtta bitar, läsa och skriva för ägaren, gruppen och övriga och två bitar till (D och L). Filerna i centralen hade alltså ägare, grupper och rättigheter som i Unix.

8.7 Nätprogrammen

ABC-klubbens programbank har ett trettiotal hjälpprogram till LUXNET, med källkoden i BASIC. Programmen är skrivna 1984–85 av Benny Löfgren och Göran Nordenborg, och upphovsrätten är Dataindustrier AB:s (ABC-klubben, LUXNET-programmen). Namnen är hämtade från Unix: LOGIN, LOGOUT, PASSWD, USERS, CHMOD, CHOWN, MKDIR, RMDIR, MOUNT, UMOUNT, FSCK, DF och CD. Där finns också programmet för centralen, NETC.SYS, och kommunikationsprocessorns COMPROG.BIN, som inte har undersökts.

Programmen talar med centralen genom DNET på $6027, som de kallar NETCALL. Före anropet läggs ett meddelande i RAM: funktionskoden på $FDC1 (MSG.FC), underfunktionen på $FDC2 (MSG.SFC), BC på $FDC3–$FDC4 och DE på $FDC5–$FDC6. På $FD1E skrivs $FF, så att svaret hamnar i DOS-buffert 0 på $F500. Efter anropet står felkoden på $FDC2. Funktionskoden är 14, och bit 6 (14 + 64) betyder att en buffert med data följer med. Underfunktionerna som programmen använder är:

Under	Vad centralen gör
0	lämnar tabellen över enheterna (E = 254) på `$F500`
1	inloggningen: namnet, lösenordet och gruppen ligger i bufferten (`LOGIN.BAC`)
8	sätter pekaren till LFD (E = 31) eller PFD (E = 27); C = `$FF` tar bort den
15	lämnar centralens namn (32 byte på `$F500`), när den genererades (`$F5F7`–`$F5FC`) och versionen (`$F5FD`–`$F5FF`)

Pekarna till UFD, LFD och PFD är tre byte var, startsektorn och enheten. Platserna står i hopptabellen `DOSADR`: `PEEK2(24685)` (`$606D`) för UFD, `PEEK2(24695)` (`$6077`) för LFD och `PEEK2(24697)` (`$6079`) för PFD. `LOGIN.BAC` kopierar alla tre ur tabellen över enheterna efter inloggningen, och `CDLFD.BAC` och `CDPFD.BAC` sätter dem genom att öppna biblioteket som en fil och läsa startsektorn ur kanalens tabell. LFD och PFD är alltså bibliotek som datorn har pekare till vid sidan av UFD:, och de ingår i sökvägen. Står ett filnamn utan enhet, som i `LOAD "X"`, letar PROM:en först i UFD:, sedan i LFD: och i PFD:, om deras pekare inte är 0, och sist i MFD: och de lokala skivminnena. Det är rutinen på `$6C48` som stegar fram till nästa enhet, och ordningen syns när pekarna sätts med `POKE` i emulatorn och nätets sektorläsningar skrivs ut. ABC-klubbens program använder också LFD: direkt: `PSEUDO.BAC` öppnar `lfd:pseudo.aut`, och `USERS.BAC` skriver ut varje inloggad användares LFD: och PFD: med bibliotekets namn.

Är bit 7 satt i felkoden är det ett fel. Koderna `$80`–`$8E` är DOS:ens vanliga fel, från ”CRC or AM error during read” till ”Invalid physical file number”. Från `$90` kommer nätets egna: ”Login please”, ”You must be a superuser”, ”Illegal subfunction code”, ”Central task already running”, ”Task not found”, ”Can’t change directory”, ”Bad login”, ”Illegal terminal number”, ”Login is not allowed at the moment”, ”Can’t unmount this device” och ”Someone is already exclusive”. `$FF` betyder att filen inte finns.

8.8 Det som inte är känt

PROM:en visar hur ABC 802 frågar nätet, men inte vad som går på ledningen eller hur nodkortet fungerar. Följande är öppet:

- vad kortets Z80 sänder på ledningen;
- vad den andra posten i `NETCRD`, kort `$30` med `$C0` till port 2, gäller;
- vad bitarna i `FILESTAT` betyder och vad förkortningarna LFD och PFD står för;
- vad centralen svarar på funktionerna 1–`$16` och vilka underfunktioner till 14 som finns utöver 0, 1, 8 och 15.

Emulatorn i den här boken spelar både nodkortet och centralen. Den har en krok på `$6009` som läser registren på stacken, gör jobbet och hoppar till `NETBAK` med L = `$80`. Det är emulatorns eget gränssnitt mot PROM:en, inte LUXNET:s protokoll, men det räcker för att `SAVE`, `LOAD`, `OPEN`, `PREPARE`, `KILL`, `NAME`, `CD`, LIB 6.01 och `L.BAC` ska fungera mot UFD:. För `L.BAC` öppnar kortet biblioteket som en fil när namnet bara är blanktecken. De första 17 posterna är katalogens sektorer, och varje fil får en RIB med storleken, ägaren `ABC802`, filens tid och skydden `--WRWRWR`. Hur det går till står i emulatorns plan i kodförrådet, `emulator/PLAN.md`.

CP/M på ABC 802

9.1 RAM från adress 0

CP/M var det vanligaste operativsystemet för datorer med 8080 och Z80. Det vill ha RAM från adress 0 och uppåt. Sidan noll har hoppen till systemet, och programmen läses in från `$0100`. Där har ABC 802 sin ROM med BASIC:en. ABC80 och ABC 800 behövde ett extra kort för att få CP/M, som MyAB:s UNI-80 och UNI-800 (*ABC80 inifrån*, band 2, kapitlet CP/M). I ABC 802 är omkopplingen inbyggd från början. Servicemanualen säger att datorn kan ställas om så att hela RAM:et laddas med program, för andra operativsystem som CP/M (s. 6).

Det är en enda bit som gör det. D7 i DART:ens Write Register 5 för kanal B sätter DTR*, som blir LRS*. Då kopplas ROM:en och bildminnet bort, och alla 64 kilobyte är RAM (se kapitlet Minnet, s. 38). Det som BASIC:en annars har som RAM-skiva, det nedre RAM:et under `$8000`, blir CP/M:s programminne. Manualen säger just det: datalagringsminnet åtgår när CP/M laddas (s. 18).

Bilden har en följd som präglar allt som följer. Bildminnet på `$7800`–`$7FFF` syns bara när ROM:en är inkopplad. Ett CP/M-system som vill skriva på skärmen måste alltså slå av DTR, skriva och slå på den igen, och under tiden är `$0000`–`$77FF` ROM. Koden som gör det, och allt den behöver, måste ligga över `$8000`.

Luxor gav själv ut en CP/M för ABC 800. Den startade genom ett litet program i sektor 16 på skivan, som kopplade om minnet och läste in systemet, och samma skiva gick enligt Torbjörn Alm också i ABC 802 och ABC 806 (Alm 1983, s. 25). I Luxors prislista 2-83 står den som CP/M 160 K, artikelnummer 230 8004-01, för 1 800 kr, och i skolprislistan 86-02-01 finns också CP/M 640 K (230 8005-02) och CP/M för åttatumsskivor (230 8006-02), alla tre för 1 800 kr i skolpris (Luxors prislistor 1982–1987). Ingen bevarad skiva bär Luxors namn. I abc80.net:s arkiv finns en skiva på 160 kilobyte med CP/M 2.2 och Pascal/MT+ och en handskriven etikett, där systemet kallar sig ”ABC800 - CP/M VER 2.2F 83-04-07”. Den startar på det sätt Alm beskriver: sektor 16 är en katalog för DOS:en med bara `BASICINI.SYS`, som läser in systemet. MyAB:s egen CP/M 2.2 kallar sig i stället ”64k CP/M ver 2.2 - MYAB BIOS ver 3.7”. Men MyAB:s CP/M Plus startar också genom `BASICINI.SYS` i sektor 16, så att skivan är Luxors är inte visat.

9.2 MyAB:s CP/M Plus

Den CP/M för ABC 802 som har bevarats är MyAB:s. MyAB, MYAB Mikrokonsult AB i Göteborg, 1985 med adress i Sundbyberg, hade sedan 1981 sålt CP/M till ABC80 och ABC 800, med kortet UNI-80 och en egen BIOS (*ABC80 inifrån*, band 2, kapitlet CP/M). Till ABC 802 behövdes inget kort. Skivan i abc80.net:s arkiv, `disk001.img`, har CP/M Plus, version 3.0, med MyAB:s BIOS 1.2C, ”ABC802 C version”. Etiketten säger ”CP/M Plus (3.0) för ABC8XX”, ”För ABC-800, 802 och 806”, skivformatet Luxor 832 och ett serienummer, och skivan har system för alla tre: CPM3800A–C.SYS, CPM3802A–C.SYS och CPM3806A.SYS (MyAB:s CP/M Plus för ABC 802). Filerna fick sina datum den 25 augusti 1984, och tre av systemen gjordes om den 18 oktober samma år. Luxor sålde inte själv systemet till ABC 802. I Luxors prislistor från 1985 och 1986 står bara ”CPM (säljes av MYAB)” utan pris. I skolprislistan 86-02-01 kostar CP/M 2.2 med en licens 4 300 kr och CP/M 3.0 med femton licenser 12 400 kr i skolpris, och där står att skolorna måste ha CP/M i systemet för att få statsbidrag. I listan från 1987-01-01 beställs CP/M 3.0 från MYAB

Mikrokonsult (Luxors prislistor 1982–1987). I MyAB:s prislista från 1985 finns ingen egen bruksanvisning för ABC 802, bara en uppsättning handböcker till CP/M Plus för 500 kr. För ABC 800 finns MyAB:s bruksanvisning bevarad.

Version 3.0

Det som erbjöds var alltså inte CP/M 2.2, som ABC80 och ABC 800 först fick, utan Digital Researchs nyare version 3.0 från 1983, som också kallades CP/M Plus. Den viktigaste nyheten var bankat minne. Ett bankat system har minst 96 kilobyte och upp till 16 banker. Överst ligger ett gemensamt område, oftast 16 kilobyte, och där finns den del av systemet som byter bank. Det mesta av systemet ligger i bank 0, så att programmen i bank 1 får nästan hela minnet under det gemensamma området (Digital Research 1983, *Programmer's Guide*, s. 1-1–1-5).

Systemfilerna och laddaren

ABC 802 har bara 64 kilobyte och ingen bankning, men MyAB:s skiva har båda sorterna. Laddaren i BASICINI.SYS tar reda på vilken dator den sitter i och sätter ihop namnet på systemfilen, CPM3 och 800, 802 eller 806 och en bokstav. I ABC 802 provar den först om det finns RAM på `$0000` efter `OUT (6),1`, sedan om ett minneskort med nummer `$34` svarar, valt med `OUT (1),$34` och påslaget med `OUT (0),1`. Med extra minne startar CPM3802A.SYS eller CPM3802B.SYS. De är bankade; deras GENCPM har lagt det mesta av BDOS:en i bank 0, och programmen får 59 kilobyte. Utan extra minne startar CPM3802C.SYS, det obankade systemet, där hela systemet delar de 64 kilobyte med programmen som i CP/M 2.2 och programmen får 51 kilobyte.

Minneskorten

Port 6 är kontrollporten på MyAB:s eget minneskort UNI802, som sattes in i datorn och hade 64 till 512 kilobyte i två banker. Bit 0 kopplar in bank 1, och de två högsta bitarna väljer var banken syns; med `OUT (6),1` ligger den på `$0000`–`$3FFF`. Kortet kostade 2 200 kr med 64 kilobyte och 7 850 kr med 512 (MyAB:s prislista 1985). CPM3802A.SYS är alltså systemet för UNI802. MyAB:s kort UNI800 till ABC 800 har sin port på samma adress, och laddaren provar den först också där. Laddaren väljer bokstaven efter vad den hittar: A för UNI802, B för kortet med nummer `$34` och C när inget extra minne svarar. I CPM3802B.SYS byter BIOS:en bank genom att välja kort `$34` och skriva 0 eller `$7F` till portarna 0 och 2. Båda 0 ger bank 0, `$7F` till port 0 ger bank 1 och `$7F` till port 2 ger bank 2. Samma kod finns i CPM3800C.SYS för ABC 800. Där provar laddaren först port 6, sedan ett kort på port `$31` (CPM3800B.SYS) och sist kort `$34`, och svarar inget av dem skriver den ”Det finns inget understött extra minne” och stannar. MyAB:s egen beskrivning av UNI800 har bara kontrollporten på I/O-adress 6, fyra banker om 64 kilobyte och mapmoderna (MyAB, UNI800). Luxors anvisning för UFD-DOS nämner Luxors minneskort på 64 och 128 kilobyte (art. nr 190 9626-11 och 190 9627-11), som DOS:en kan använda som RAM:, men inte deras kortnummer. Vilket kort som har nummer `$34` säger alltså ingen av källorna.

Det fanns också nyheter som inte hängde på bankningen. Filerna kunde få datum och klockslag när de skapades, ändrades eller lästes, och de kunde skyddas med lösenord. Filerna fick vara större, upp till 32 megabyte, och skivorna upp till 512 megabyte. Systemet kunde ha flera buffertar för skivan, läsa flera sektorer i följd och hitta i katalogen genom en hashtabell. Programmen kunde utökas med små residenta tillägg, RSX, och kommandona `GET` och `PUT` kunde ta in- och utmatningen från och till filer. CCP:n kunde söka efter program på flera enheter, och `DIR`, `SHOW`, `DATE`, `SET`, `DEVICE` och `HELP` gav mer än 2.2:ans `STAT`. En del fanns bara i det bankade systemet. Det obankade har inga lösenord på filerna, och när en rad skrivs in kan bara det sista tecknet rättas; det bankade kan flytta markören i raden och ta fram den förra raden. Buffertarna och hashtabellerna finns i båda, men i det bankade ligger de i bank 0 och tar inget från programmen (Digital Research 1983, *Programmer's Guide*, s. 1 och tabell 3-2, s. 65).

Klockan

MyAB:s skiva använder datum. Dess katalogetikett, CPMPLUS V12, slår på datum när filerna skapas och ändras men inte när de läses, och inga lösenord. Tiden kommer inte från någon klockkrets. BIOS:ens anrop TIME är bara ett `RET`. I stället ställer BIOS:en vid starten in CTC:ns kanal 3 med `$A7` och `$7D` på port `$63`, alltså 3 megahertz delat med 256 och med 125, vilket ger 93,75 avbrott i sekunden. Avbrottsrutinen på `$F2D9` räknar 94 avbrott för tre sekunder av fyra och 93 för den fjärde. Sedan räknar den upp sekunderna, minuterna och timmarna i BCD i systemets kontrollblock, och dagnumret när timmarna går runt. Det är samma kanal och samma takt som ROM:ens klocka till `TIME$`. Ingenting läser tiden från en klocka med batteri, så den börjar om vid varje start och ställs med `DATE` (Digital Research 1983, *User's Guide*).

Starten

Skivan startar genom DOS:en. Vid kallstarten öppnar `OPNINI` filen BASICINI.SYS och läser dess första post till sektorbufferten på `$F500`. Börjar posten med ett `JP`, körs den (se kapitlet DOS och skivminnet, s. 48). Det är en krok för startkod, och MyAB:s BASICINI.SYS använder den för att läsa in CP/M. Maskinen behöver alltså ingen egen start-PROM, bara en skiva i den första enheten.

9.3 Laddaren i BASICINI.SYS

En CP/M 2.2 för ABC 802 har Digital Researchs CCP och BDOS, kommandotolken och filsystemet, och en BIOS och en laddare skrivna till maskinen. CCP:n ligger på `$E000`, BDOS:en på `$E806` och BIOS:en på `$F600`, så att programmen har `$0100`–`$DFFF`, knappt 56 kilobyte. Laddaren använder samma krok som MyAB:s. Den är post 0 i BASICINI.SYS och körs från `$F503`, efter postens tre första byte. Den får alltså vara högst 253 byte. DOS:en har redan läst den från skivan i den första enheten, kort 44 enhet 0, MF0, och laddaren läser resten därifrån. Den talar själv med kortet, utan DOS:en: `OUT (1),A` väljer kort 44, `OUT (2),A` säger att ett nytt kommando kommer, och kommandot är fyra byte på port 0, 3 för läsning, enheten och sektorns nummer i två byte. Statusen på `IN 1` säger när kortet tar emot och när data kommer, och slutstatusen på `IN 0` är 0 om det gick bra (`emulator/cpm/start.asm`).

Laddaren läser 32 sektorer, sektor 40–71, de 8 kilobyte som ska ligga på `$E000`–`$FFFF`. De hamnar först på `$8000`, som är RAM i båda lägena. Sedan kopierar den fyra instruktioner till `$A000` och hoppar dit. Laddaren ligger själv på `$F500`, mitt i det som ska skrivas över, och kopian på `$A000` går utan hopp, så att den kan köras var som helst:

```
FLYTT:  LD   A,5          ; DART B WR5: DTR på, 64 KB RAM
        OUT  ($23),A
        LD   A,$E8
        OUT  ($23),A
        LD   HL,LADDA     ; $8000
        LD   DE,SYSTEM    ; $E000
        LD   BC,ANTAL*256
        LDIR
        JP   BOOT         ; $F600
```

`$E8` är BASIC:ens `$68` med D7 tillagd (se kapitlet Minnet, s. 38). D1 är 0, så skärmen har 80 tecken. Efter `LDIR` är maskinen en CP/M-dator, och BIOS:ens `BOOT` på `$F600` tar över. Går läsningen fel, återvänder laddaren till DOS:en med HL satt till `$6F00`. Då startar maskinen BASIC som om BASICINI.SYS inte hade funnits.

9.4 BIOS:en

BIOS:en börjar med de 17 hopp som BDOS:en väntar sig i CP/M 2.2, från `BOOT` till `SECTRAN`. Efter dem kommer en byte, `BREDDEN`, med skärmens bredd, 80. Den är till för editorn SKRIV, som läser bredden ur minnet. Kallstarten slår av avbrotten och stänger av dem i DART:en, SIO:n och CTC:n. ROM:ens tabell för avbrotten i läge 2 ligger på `$FFB0`, och den är borta när RAM:et har tagit över. Ett program som gör `EI` ska inte kunna hoppa ut i det okända. Sedan skriver BIOS:en hoppen på sidan noll, `JP` till varmstarten på `$0000` och till BDOS:en på `$0005`, och går till CCP:n. Vid varmstart, till exempel efter `^C`, läses CCP:n och BDOS:en om från sektor 40 och framåt.

Tangentbordet är detsamma som för BASIC:en. Dess processor skickar koden på DART:ens kanal B (se kapitlet Tangentbordet, s. 28). Utan avbrott läser BIOS:en bit 0 i Read Register 0 på `$23`, som säger att ett tecken har kommit, och tecknet på `$22`. Efter varje tecken återställer den felen med `$30` i Write Register 0, som ROM:ens `KEYINT` gör.

Skärmen har 24 rader om 80 tecken, och markören är 6845:ans egen, register 14 och 15. `KONUT` byter till en egen stack över `$8000`, slår av DTR, skriver i bildminnet och slår på DTR igen. All BIOS:ens kod och alla dess variabler ligger över `$8000`, där RAM:et syns i båda lägena. Koderna är ett urval av terminalen ADM-3A:s, och raderingen av rader och skärmen som hos TeleVideo, så att färdiga CP/M-program kan använda dem:

Kod	Verkan
CR (`$0D`)	till radens början
LF (`$0A`)	ned en rad; på sista raden flyttas skärmen upp
BS (`$08`), `^L` (`$0C`), `^K` (`$0B`)	vänster, höger, upp
`^Z` (`$1A`)	radera skärmen, markören överst till vänster
`^^` (`$1E`)	markören överst till vänster
ESC = r c	markören till rad r − 32, kolumn c − 32
ESC T, ESC Y, ESC *	radera till radens slut, till skärmens slut, hela skärmen
ESC E, ESC R	infoga en tom rad, radera raden

Koder under `$20` syns inte. I teckengeneratorn är de grafik- och blinkkoder (se kapitlet Bildskärmen, s. 17). Tecknen `$20`–`$7E` skrivs som de är, och i ABC 802:s teckensats är `[\]{|}` ÄÖÅääöå. Därför skriver BIOS:en sin hälsning som `p}` i källan, och skärmen visar ”på”. Skrivaren och hålremsan, LST: och PUN:, finns inte än, och tecknen dit försvinner. RDR: ger alltid `^Z`, filslut (`emulator/cpm/bios.asm`).

9.5 Skivformatet

Enheterna A: och B: är kort 44 enhet 0 och 1, MF0 och MF1. Avbilden har 1 280 sektorer om 256 byte, 320 kilobyte. Hela skivan är på samma gång en DOS-skiva och en CP/M-skiva. De första sektorerna är DOS:ens, så att DOS:en hittar BASICINI.SYS vid starten. Resten är CP/M:s:

```
sektor 0-39     DOS:ens del: etiketten, bitkartan (14), katalogen
                (16-23), SYSDIR.SYS och BASICINI.SYS med laddaren
sektor 40-71    CCP, BDOS och BIOS:en
sektor 72-79    tomma
sektor 80-1279  spår 5-79: 150 block om 2 KB, katalogen i block 0-1
```

DOS:ens bitkarta säger att alla kluster är upptagna. Då skriver DOS:en aldrig i CP/M:s del, och en `SAVE` från BASIC får felet att skivan är full i stället för att förstöra CP/M-filerna. CP/M ser 80 spår om 32 sektorer om 128 byte, alltså 16 av kortets sektorer per spår. De fem första spåren, sektor 0–79, är systemspår, och CP/M rör dem inte. Spår 5–79 är 150 block om 2 kilobyte med 128 katalogposter. Med blocknummer i en byte rymmer en katalogpost 16 block, 32 kilobyte, och därför är EXM 1. Kortet vill ha sektorns nummer N som $32 \cdot (N \,/\, 4) + (N \text{ AND } 3)$, och BIOS:en räknar om spåret och sektorn till det. `STAT DSK:` visar parametrarna:

```
A>STAT DSK:
     A: Drive Characteristics
  2400: 128 Byte Record Capacity
   300: Kilobyte Drive  Capacity
   128: 32  Byte Directory Entries
   128: Checked  Directory Entries
   256: Records/ Extent
    16: Records/ Block
    32: Sectors/ Track
     5: Reserved Tracks
```

BIOS:en har en buffert för en sektor om 256 byte, två av CP/M:s. En skrivning går genast till skivan, utan fördröjning, så att inget blir kvar i bufferten när ett program slutar. Det är långsammare men säkrare.

Systemskivan i A: har programmen. B: finns bara när det sitter en skiva i MF1.

9.6 Programmen

När maskinen startar med systemskivan i den första enheten skriver BIOS:en sin hälsning, och CCP:n väntar på ett kommando:

```
CP/M 2.2 på ABC 802, 64 KB
A>DIR
A: ASM      COM : DDT      COM : DUMP     COM : ED       COM
A: LOAD     COM : PIP      COM : STAT     COM : SUBMIT   COM
A: XSUB     COM : SKRIV    COM : SKRIV    TXT : BBCBASIC COM
A: ZTOLK    COM : ZORK1    Z3
A>STAT
A: R/W, Space: 130k
```

De första nio är Digital Researchs program, oförändrade ur distributionsskivan till Xerox 1800 i samma fria utgåva: assemblern ASM, felsökaren DDT, DUMP, radeditorn ED, LOAD som gör en COM-fil av assemblerns HEX-fil, PIP som kopierar filer, STAT, SUBMIT och XSUB (Digital Research, CP/M 2.2). Med dem kan man skriva, assemblera och köra ett program på maskinen själv: `ED HEJ.ASM`, `ASM HEJ`, `LOAD HEJ` och `HEJ`.

SKRIV

SKRIV är en editor som visar texten på hela skärmen och har WordStars tangenter. Samma SKRIV finns till ABC80. Bara bredden ändras: SKRIV läser den i BIOS:ens `BREDDEN`. På de 23 översta raderna står texten, och den nedersta visar filen, raden och kolumnen. ESC och en bokstav är detsamma som Ctrl och bokstaven. `SKRIV SKRIV.TXT` visar bruksanvisningen, som talar om ABC80 och 40 kolumner.

BBC BASIC

BBC BASIC är R. T. Russells BBC BASIC (Z80), version 5.00, den allmänna utgåvan för CP/M. Russell lämnar programmets första sida, `$0100`–`$01FF`, åt den som anpassar det till en dator. Här har den en egen sida, efter den till ABC80: `CLS` och markören med BIOS:ens koder, `POS` och `VPOS` ur BIOS:en, bredden 80 och en räknad klocka, eftersom avbrotten är av. `TICKS` väntar 1 154 varv om 26 T, 10 millisekunder vid 3 MHz. Resten av programmet är Russells, byte för byte. Det får spridas fritt under zlib-licensen (Russell, BBC BASIC (Z80)). `*BYE` går tillbaka till CP/M:

```
A>BBCBASIC
BBC BASIC (Z80) Version 5.00
(C) Copyright R.T.Russell 2025
>PRINT PI
```

```
3.14159265
```

Zork I

Zork I är Infocoms äventyrsspel, berättelsefilen för Z-maskinen version 3, release 119. Filen är 86 838 byte och sedan 2025 fri under MIT-licensen (Infocom, Zork I). Tolken ZTOLK är densamma som till ABC80. Den använder BIOS:en direkt genom hopptabellen och läser bredden i `BREDDEN`. Den dynamiska delen av spelet ligger fast i minnet, och resten läses i sidor om 256 byte när de behövs. `ZTOLK ZORK1` startar spelet. Översta raden är en statusrad med platsen, poängen och antalet drag, och `save` och `restore` skriver och läser ZORK1.SAV.

Pascal/MT+

Pascal finns på en egen skiva, eftersom systemskivan bara har 130 kilobyte lediga. Det är Digital Researchs Pascal/MT+ 5.6.1 för 8080 och Z80 (Digital Research, Pascal/MT+): kompilatorn MTPLUS med sju överlägg, länkaren LINKMT, körbiblioteket PASLIB och flyttalen i FPREALS. Skivan är en hel CP/M-skiva som går att starta från, med ED, PIP, STAT och SKRIV, och två exempel, HEJ och PRIMTAL. Den har 94 kilobyte lediga.

Ett program kompileras med `MTPLUS HEJ` till HEJ.ERL och länkas med `LINKMT HEJ,FPREALS,PASLIB/S` till HEJ.COM. Ett program utan reella tal behöver inte FPREALS. Eftersom } är å på ABC 802 skrivs kommentarerna med `(* *)`:

```
A>LINKMT HEJ,FPREALS,PASLIB/S
...
Total data: 052DH bytes
Total code: 2E3FH bytes
Remaining : 82EEH bytes
A>HEJ
Hej från Pascal/MT+ på ABC 802!
    1    4    9   16   25   36   49   64
Roten ur 2 är   1.414214
```

PRIMTAL

PRIMTAL är Eratosthenes såll upp till 8 191. Det skriver primtalen under 100 och sist ”1028 primtal under 8192”.

Pascal på ABC 802

10.1 Ett språk från tiden

ABC 802 har BASIC II i ROM:en och kunde med en skiva få CP/M och de program som fanns till det (se kapitlet CP/M på ABC 802, s. 55). Luxor sålde också ett Pascal som går under DOS:en utan CP/M, Pascal 800 från Dataindustrier (Pascal 800). Det har en kompilator som ger p-kod, en länkare, en editor och två tolkar, och språket är utvidgat som UCSD Pascal. Det beskrivs sist i kapitlet (avsnittet Pascal 800, s. 66).

Det här kapitlet visar ett mindre: en delmängd av Pascal med en kompilator och en tolk för p-kod, som går i BASIC-läget och använder ROM:en och DOS:en utan CP/M, på en skiva som startar från BASIC.

Förebilderna är från tiden före ABC 802. Tiny Pascal var en delmängd vars kompilator, skriven i BASIC, gav p-kod som tolkades eller översattes till 8080-kod (Chung och Yuen 1978). Pascal-S hade kompilatorn och tolken i ett och samma program, för en delmängd tänkt för undervisning (Wirth 1975). UCSD Pascal gjorde p-koden till en bytekod som kunde gå på många maskiner och delade programmen i segment som laddades vid behov. Därifrån kommer kodningen med korta former och formatet för segmenten, inte instruktionerna i detalj.

Maskinen har två delar. Kompilatorn `pc.pas` är skriven i delmängden och översätter Pascal till p-kod. Tolken `pvm.asm` är skriven i Z80-assembler och kör p-koden. Kompilatorn är själv ett p-kodprogram, `PC.PK`, och ABC 802 kompilerar och kör alltså Pascal på egen hand. Provet på att det går är att `PC.PK` på ABC 802 kompilerar `pc.pas` och får samma byte som den kompilerades från.

Allt går i BASIC-läget, där tolken, kompilatorn och dess tabeller ska rymmas mellan `$8000` och `$ED00`. Det som inte ryms, som RECORD och fält av fält, hör till en större version i 64 kilobyte utan BASIC (avsnittet Mot 64 kilobyte, s. 66).

10.2 Delmängden

Typerna är INTEGER med 16 bitar, CHAR, BOOLEAN och REAL, och fält av dem med en dimension och konstanta gränser. Strängar är `PACKED ARRAY [1..n] OF CHAR`. De kan tilldelas och jämföras hela, och tecknen nås ett och ett med index. Det finns CONST, TYPE och VAR, procedurer och funktioner i varandra med värde- och VAR-parametrar, och FORWARD. Satserna är de vanliga: tilldelning, anrop, `BEGIN` `END`, `IF`, `WHILE`, `REPEAT`, `FOR` med `TO` och `DOWNTO`, och `CASE`. Till standardprocedurerna hör `WRITE` och `READ` med sina `LN`-former, `EOF` och `EOLN`, `ORD`, `CHR`, `SUCC` och `PRED`, och för REAL `SQRT`, `SIN`, `COS`, `ARCTAN`, `LN`, `EXP`, `TRUNC` och `ROUND`.

Filerna är textfiler. Ett program deklarerar en variabel av typen `TEXT` och öppnar den med `RESET` eller `REWRITE`. Filnamnet är andra parametern, som i `RESET(F, 'HEJ.PAS')`, och så gör UCSD Pascal och Turbo Pascal också. Två filer kan vara öppna på samma gång, och kompilatorn behöver just två, källan och p-koden. `HALT` avslutar programmet, också det som i UCSD Pascal.

Pekare, mängder, GOTO, WITH, RECORD, fält av fält och procedurer som parametrar finns inte. Det mesta av det hör till den större versionen.

Teckensatsen ställer till det. I ABC 802 är `[\]{|}` ÄÖÅääöå (se kapitlet Bildskärmen, s. 17). Med `{ }` som kommentar skulle varje ä i en svensk kommentar börja en ny, så kompilatorn godtar bara `(* *)`. Index kan skrivas både `A[I]` och `A(.I.)`, som standarden tillåter. På skärmen syns `A[I]` som AÄIÅ, och program som ska läsas på maskinen skrivs därför med `(. .);` i boken står `[ ]`. Ett namn får bara ha A–Z, a–z och 0–9, så TALÅR går inte som namn. I strängar och kommentarer är de svenska bokstäverna bara byte.

Ett litet program, `HALSA.PAS`, som finns på skivan:

```
PROGRAM HALSA(OUTPUT);
(* En hälsning och kvadraterna. *)
CONST N = 10;
VAR I, S : INTEGER;
BEGIN
  WRITELN('Hej fran Pascal!');
  S := 0;
  FOR I := 1 TO N DO
  BEGIN
    WRITE(SQR(I):4);
    S := S + I
  END;
  WRITELN;
  WRITELN('Summan 1..', N:1, ' = ', S:1)
END.
```

Programmet kompileras och körs från BASIC. Tolken frågar efter programmet med `Program:`, och `PC` är kompilatorn, som läser namnet på källan och skriver `HALSA.PK`. Efter den går `Z=CALL(32768)` till tolken igen, och nu körs `HALSA`:

```
Program: HALSA
Hej fran Pascal!
   1   4   9  16  25  36  49  64  81 100
Summan 1..10 = 55
ABC802
```

`ABC802` är BASIC:ens uppmaning när tolken har gått tillbaka.

10.3 P-koden

P-maskinen är en stackmaskin med ord om 16 bitar. Uttrycken räknas på stacken, och stacken är Z80:ans egen. Varje procedur har en aktiveringspost med märket: den statiska länken till posten för den omgivande proceduren, den dynamiska länken till anroparens post och återhoppet. Under märket ligger argumenten och de lokala variablerna, och en funktion har en plats för sitt värde över märket. Variablerna i en omgivande procedur nås genom att följa den statiska länken. I riktiga program är det sällan mer än ett eller två steg, så en kedja räcker, och någon display som i Pascal-S behövs inte.

Varje instruktion börjar med en opkod på en byte. De vanligaste fallen har operanden inbyggd i opkoden, som i UCSD: `LDC0`–`LDC15` lägger en konstant, `LDL0`–`LDL7` och `STL0`–`STL7` laddar och lagrar de första åtta lokala orden, och `LDG0`–`LDG7` de första åtta globala. Hoppen är relativa, med en byte när målet ligger nära och annars med ett ord. Det blir 126 opkoder, 98 för heltalen, fälten, hoppen och anropen i `$00`–`$8F` och 28 för REAL i `$A0`–`$BB`.

Allt som har med världen utanför att göra går genom en enda instruktion, `CSP n`, en standardprocedur med nummer n: skriva och läsa tecken, heltal och strängar, radslut och filslut, och öppna och stänga filer. Tecknen och filerna hänger på maskinen, så det är bara CSP som behöver skrivas om för en annan omgivning, som CP/M. Ett körfel, som index utanför gränserna eller division med noll, skriver felets nummer och var i p-koden det hände och går tillbaka till BASIC.

Slingan i `HALSA` blir så här, med adresserna i proceduren till vänster. `I` är globalt ord 0, `S` ord

1 och gränsen för FOR, 10, ett gömt ord 2. SQRI är SQR för heltal, och CSP 1 skriver ett heltal i fältbredden 4 på fil 0, skärmen. Som lista:

```
001D  LDC0
001E  STG   1
0021  LDC1
0022  STG   0
0025  LDC10
0026  STG   2
0029  LDG0
002A  LDG2
002B  LEQ
002C  FJP   0042
002F  LDC0
0030  LDG0
0031  SQRI
0032  LDC4
0033  CSP   1
0035  LDG1
0036  LDG0
0037  ADD
0038  STG   1
003B  LDG0
003C  INC1
003D  STG   0
0040  UJPS  0029
```

Hela HALSA.PK är 120 byte. En p-kodfil har procedurernas kod först, i den ordning de blev klara, och sist en tabell över var varje procedur börjar, storleken på de globala och antalet procedurer. Tabellen ligger sist eftersom den blir känd sist, och då kan kompilatorn skriva ut varje procedur så snart den är färdig.

10.4 Tolken

Tolken är 4 089 byte och ligger på $8000–$8FF8. PASCAL.ABS läses in av DOS:ens DRUN, som hoppar till LADDAD, och den flyttar BASIC:ens början till $ED00 så att BASIC inte skriver i p-koden. Sedan startar Z=CALL(32768) tolken. På skivan laddar BASICINI.SYS tolken och startar den när maskinen startar. Tolken frågar efter programmet, lägger till .PK om namnet saknar punkt, läser filen till $9000 och kör den. De globala variablerna ligger direkt efter p-koden, och p-stacken växer nedåt från $ED00:

```
$ED00-         BASIC, DOS:ens buffertar och BASIC:s stack
$DD00-$ECFF  FPP.BIN, flyttalen (bara när programmet har REAL)
           p-stacken, nedåt från $ED00 eller $DD00
           de globala variablerna, direkt efter p-koden
$9000-         p-koden
$8000-$8FFF  tolken
```

I tolken är DE IP, pekaren i p-koden, IX är MP, den aktuella aktiveringsposten, och Z80:ans SP är p-stacken. IY rörs inte, eftersom ROM:en använder det. Opkoden slås upp i två tabeller om 256 byte på $8100 och $8200, en för den låga och en för den höga byten i rutinens adress. Då behöver tabellen inget ADD HL,HL, och nästa instruktion är nio Z80-instruktioner bort:

```
NEXT:   LD   A,(DE)
        INC  DE
        LD   L,A
        LD   H,TABL>>8
        LD   C,(HL)
        INC  H
        LD   H,(HL)
        LD   L,C
```

```
        JP    (HL)
```

En opkod som inte finns pekar på en rutin som ger körfel 6. De flesta rutinerna är korta, eftersom stacken är Z80:ans. ADD är så här:

```
O_ADD:  POP   BC
        POP   HL
        ADD   HL,BC
        PUSH  HL
        JP    NEXT
```

Standardprocedurerna och laddaren går på BASIC:s stack, under den SP som CALL kom med. Där finns ett par hundra byte för ROM:en och DOS:en, medan p-stacken bara har en marginal på 64 byte över de globala. Tecknen går genom ROM:ens rutiner för skärmen och tangentbordet, och filerna genom DOS:ens hopptabell på $6015–$6030, med kanal 1 och 2 och buffertarna på $F600 och $F700. På skivan är en fil poster om 253 byte, som BASIC:s egna filer: data på plats 3–254 och ETX sist. Byten 0, 3 och 16 skrivs som 16 och byten plus 64, så att de inte tas för filens slut. I en textfil är radslutet CR, och CR, CR LF och LF läses alla som radslut.

10.5 Flyttalen

ROM:en har flyttal, och BASIC II räknar med dem. Det enklaste vore att anropa dem från tolken. Men en version under CP/M eller i 64 kilobyte utan BASIC har inte ROM:en, och att kopiera Luxors rutiner dit går inte. Samma p-kod ska ge samma tal i båda versionerna. Därför räknar Pascal på ABC 802 med flyttalen ur BBC BASIC. R. T. Russell skrev dem för BBC BASIC (Z80), och de ligger i en egen modul, FPP, som inte rör maskinen och går att flytta. (Russell, BBC BASIC (Z80)). Den ändring som behövs är att FPP sparar SP i en egen variabel och inte i IY, som ABC 802:s ROM använder.

Ett REAL har 40 bitar, cirka nio siffror, och tar tre ord, sex byte, i p-maskinen: FPP:s fem byte och en byte utfyllnad. En konstant står som text i p-koden, och FPP gör om den när den laddas, så att kompilatorn inte behöver räkna med flyttal alls. FPP och limmet till p-stacken, preal.asm, är FPP.BIN, 3 857 byte på $DD00–$EC10. Tolken laddar den bara när p-kodfilen säger att programmet använder REAL, och då börjar p-stacken under $DD00. Ett program utan REAL har alltså hela minnet kvar.

10.6 Kompilatorn

pc.pas är 1 595 rader med 77 procedurer och funktioner. Den läser källan i ett pass, med rekursiv nedstigning: en procedur för varje regel i grammatiken, som i Pascal-S. NASTA läser nästa symbol, ett tecken i taget från filen, och SATS, UTTRYCK, TERM och FAKTOR anropar varandra som reglerna gör. Eftersom FAKTOR anropar UTTRYCK, som står efter, behövs FORWARD, och därför finns FORWARD i delmängden.

Koden skrivs direkt, utan något träd emellan. Ett hopp framåt skrivs med ett tomt mål, och målet fylls i när det blir känt. FRAMAT skriver hoppet och ger dess plats, och FYLL skriver avståndet dit där kompilatorn nu är. KLEN är längden på koden hittills:

```
FUNCTION FRAMAT(O : INTEGER) : INTEGER;   (* hopp med tomt mål *)
BEGIN
  KB(O); KW(0);
  FRAMAT := KLEN - 3
END;

PROCEDURE FYLL(P : INTEGER);
VAR R : INTEGER;
BEGIN
```

```
    R := KLEN - (P + 3);
    SATT(P + 1, R MOD 256);
    SATT(P + 2, R DIV 256)
  END;
```

Med dem blir WHILE några rader i SATS. Villkoret räknas, FJP hoppar förbi satsen när det är falskt, och efter satsen hoppar BAKAT tillbaka till villkoret, med den korta formen om den räcker:

```
    ELSE IF SYM = SWHILE THEN
    BEGIN
      NASTA;
      BORJAN := KLEN;
      TA(UTTRYCK, BOO);
      VANT(SDO);
      F := FRAMAT(OFJP);
      SATS;
      BAKAT(OUJP, BORJAN);
      FYLL(F)
    END
```

För att hoppen ska kunna fyllas i hålls koden för den procedur som kompileras i minnet, och den skrivs till filen vid procedurens END. Det räcker med en buffert, eftersom de nästlade procedurerna står före BEGIN och blir färdiga först. Därför kommer de innersta procedurerna först i p-kodfilen och huvudprogrammet sist.

Namnen ligger packade två tecken per ord i ett fält, POOL, och symboltabellen har namnets plats där, slaget, typen och nivån i ett ord, värdet och en länk. Länken går till nästa namn med samma hashsumma, summan av tecknen modulo 61. Ett namn söks bara i sin kedja, och det senast deklarerade kommer först, så att det närmaste namnet vinner. När en procedur är klar kopplas dess namn ur kedjorna. De reserverade orden och de fördefinierade namnen ligger i samma tabell. De står i sex strängar om 64 tecken som läses in när kompilatorn startar, vilket är mindre kod än ett anrop för varje namn.

Det första felet avbryter kompileringen, med rad, kolumn och nummer. Texterna till felen står inte i pc.pas, för de skulle ta plats. Någon återhämtning efter ett fel finns inte heller.

10.7 Att kompilera sig själv

Kompilatorn är skriven i det språk den kompilerar. PC.PK på skivan kompilerar PC.PAS till en ny PC.PK, och den blir densamma, byte för byte. En ändring i kompilatorn görs alltså i PC.PAS och kompileras med den gamla PC.PK. Den nya prövas genom att den kompilerar sig själv en gång till och ger samma byte.

Det svåra är att få kompilatorn att rymmas, för den ska ha sin egen kod, sina tabeller och källans namn i minnet på en gång. Koden för en procedur ligger packad två byte per ord, slag, typ och nivå delar ett ord, de fördefinierade namnen är strängar, och kompilatorns egna strängar är packade två tecken per ord. Tabellernas gränser är satta efter vad kompilatorn själv behöver, med lite marginal:

Del	Byte	Adresser
tolken	4 089	$8000–$8FF8
kompilatorn, PC.PK	13 432	$9000–$C477
de globala	9 400	$C478–$E92F
stacken vid självkompileringen	620	
kvar under $ED00	976	

Gränserna står som konstanter först i pc.pas: 1 536 byte kod i en procedur, 370 namn, 1 300 ord för namnen, 90 procedurer och 30 fälttyper. Självkompileringen använder 352 namn och 1 261 ord

för dem, och den största proceduren blir 1 328 byte. Ett program som är större än kompilatorn ryms alltså inte. Kompilatorn använder inte REAL själv och har hela minnet upp till `$ED00`.

På ABC 802 tar självkompileringen omkring 750 sekunder, tolv och en halv minut, för 11,5 miljoner p-instruktioner. Det är i medeltal omkring 65 mikrosekunder för en p-instruktion, med läsningen och skrivningen av filerna.

Skivan har tolken `PASCAL.ABS`, `FPP.BIN`, `PC.PK`, `PC.PAS` och åtta små program: `HALSA`, primtal, rekursion, alla satserna, quicksort, läsning av en fil, REAL och strängar.

10.8 Mot 64 kilobyte

I BASIC-läget finns inte mer att hämta. Det är under en kilobyte kvar när kompilatorn kompilerar sig själv, och varje ny del av språket gör kompilatorn större. RECORD, fält av fält och fält som värdeparametrar har plats i p-koden, men inte i minnet. I BASIC-läget får Pascal på ABC 802 den form som beskrivs här.

Under CP/M har ABC 802 alla 64 kilobyte som RAM (se kapitlet CP/M på ABC 802, avsnittet Laddaren i BASICINI.SYS, s. 57). Där är tolken ett CP/M-program, `PVM.COM`, på 9 489 byte från `$0100`, med flyttalen i samma fil. Skärmen och filerna går genom BDOS i stället för ROM:en och DOS:en, och p-koden, de globala och stacken har `$2611`–`$E7FF`, 49,5 kilobyte, mot 23,75 kilobyte i BASIC-läget. Det är samma p-kod och samma flyttal, och ett program från BASIC-läget går oförändrat där. `PVM PC` kompilerar `PC.PAS` till samma `PC.PK` som i BASIC-läget, på omkring 730 sekunder, lite fortare än där.

I `PVM.COM` finns också det som inte fick plats: RECORD med WITH, fält av fält, fält och poster som värdeparametrar, uppräknade typer, delomfång, pekare med NEW, MARK och RELEASE, och mängder. En mängd är 32 byte med en bit för vart och ett av elementen 0–255, och tolken har tretton nya opkoder för dem, `$90`–`$9C`. Pekarna tar sitt minne ur en hög som växer uppåt från de globala, mot stacken som växer nedåt. Det mesta av det andra klaras med de gamla opkoderna. Ett program som använder det nya märks med en annan version i filen, och tolken i BASIC-läget vägrar köra det. `PC.PK` kompilerar inte det nya, men skivan har en andra kompilator, `PCS.PAS`, som gör det. Den är också skriven i delmängden och bygger på `PC.PAS`, med 2 157 rader och 100 procedurer och funktioner. Typerna står i en egen tabell med slaget och tre värden, som gränserna och elementets typ för ett fält, och fälten i posterna i en tabell för sig. Versionen i filen blir känd först när hela källan är läst, så koden går först till en fil med två bokstäver för varje byte, och `.PK`-filen skrivs när källan är stängd. `PCS.PK` är 18 931 byte och har 21 362 byte globala, och går bara under CP/M. `PVM PCS` kompilerar `PCS.PAS` till samma `PCS.PK` på omkring 1 180 sekunder, nästan 20 minuter, för 17,7 miljoner p-instruktioner. GOTO, procedurer som parametrar och variantposter saknas i båda kompilatorerna. Skalet, editorn och kompilatorn kan sedan vara segment i p-kod som laddas vid behov, som i UCSD Pascal och i Pascal 800.

10.9 Pascal 800

Pascal 800 från Dataindustrier är ett större system för ABC 800, som också går på ABC 802 (Pascal 800). Det går under DOS:en i BASIC-läget och klarar sig ändå med en kompilator på 29 kilobyte, eftersom programmen är delade i segment som läses från skivan när de anropas. Skivan har fem program för Pascal:

Fil	Byte	Innehåll
PASSYS.ABS	8 960	tolken med kompilatorns sökrutiner
PASCAL.ABS	8 704	tolken med REAL
PASCOMP.COD	32 000	kompilatorn P-1.01, p-kod i 12 segment
PASLINK.COD	11 008	länkaren P-1.02, p-kod i 2 segment
EDIT.ABS	3 584	radeditorn, $C700–$D47D

Tolkarna och p-koden

Båda tolkarna laddas på $8000. De är samma p-maskin men har olika standardprocedurer. I PASSYS.ABS finns två rutiner i maskinkod som bara kompilatorn använder: IDSEARCH, som slår upp de reserverade orden i en tabell med åtta tecken per ord på $8FD5, och TREESEARCH, som söker i symbolträdet. Flyttalen saknas där. PASCAL.ABS har flyttalen och SIN, COS, ATAN, LN, EXP, SQT och slumptal, men inte sökrutinerna. Kompilatorn och länkaren körs därför med PASSYS, och under PASCAL stannar kompilatorn med RUNTIME ERROR NR.11. Ett program utan REAL går med båda.

P-koden är UCSD Pascals. I tolken håller BC p-räknaren, och Z80:ans egen stack är beräkningsstacken. En instruktion hämtas med LD A,(BC), INC BC och ADD A,A, som skjuter ut den högsta biten i carry. Är den noll, är byten under 128 och själv konstanten som ska läggas på stacken (SLDC). Annars är den dubblade byten platsen i en tabell med 128 adresser på $8000. Standardprocedurerna anropas med CSP och UCSD:s nummer, och läsning och skrivning med CXP till segment 0, som är tolkens egna rutiner. NIL är 1.

Ett anrop till ett annat segment läser segmentet ur kodfilen och lägger det på stacken, under den som anropar. Tolken räknar hur många anrop som är aktiva i varje segment, och när det sista återvänder är platsen fri igen. Det är så kompilatorn ryms: dess 12 segment är tillsammans större än det fria minnet, och det största är 9 552 byte med 42 procedurer. Minnet för ett litet program ser ut så här:

Adresser	Innehåll
$FA00–$FCEF	tolkens bofasta del, med skyddsprovet
ca $F600	bufferten för skivan
från $EFBE	programmets kodsegment
under det	stacken med de globala och procedurernas data
från $999D	heapen
$8000–$999C	tolken

Minnet och filerna

Mellan heapen och stacken är det omkring 21 kilobyte fritt. Tolkens startkod ligger där heapen börjar och skrivs över när programmet kör.

Text till skärmen går genom ROM:ens $000B, och filerna går genom BASIC:s drivrutiner för enheter, med ett filblock och en radbuffert som när BASIC skriver med PRINT #. En fil som BASIC har skrivit kan alltså kompileras, och Pascals textfiler kan läsas i BASIC. När programmet är slut läser tolken in DOS:ens kommandotolk CMDINT.SYS igen.

En kodfil ligger på DOS:ens sektorer om 256 byte. De tre första byten i varje sektor är filens typ, sektorns nummer och en nolla, och resten, 253 byte, bildar en följd av byte där segmenten står. Första sektorn har antalet segment, var varje segment börjar och hur långt det är, och sedan programmets namn med en text från kompilatorn, som PROV A800 Pascal 1.01 File: MF1:PROV. Sist i ett segment står dess nummer, antalet procedurer och en förteckning över dem. För varje procedur finns numret, nivån, var koden börjar och slutar och hur mycket plats parametrarna och de lokala tar. Huvudprogrammet är procedur 1.

Språket och kommandoraden

Språket har UCSD Pascals tillägg, som STRING med CONCAT, COPY, POS, INSERT och DELETE, MOVELEFT, FILLCHAR och SCAN, BLOCKREAD och BLOCKWRITE, SEEK och IORESULT, MARK, RELEASE och MEMAVAIL. Till dem lägger Pascal 800 sina egna: PEEK, POKE, INP, OUT och CALL för maskinen, IAND, IOR, IXOR, ISHIFT och ISWAP för bitar, GOTOXY och GRAPH för skärmen, DISPOSE, ADDRESS, RENAME, DATE, STARTPAR, OPTION och ISAM.

På kommandoraden står tolken först. Bokstäver efter ett komma på tolkens namn är flaggor, och det som står efter kodfilens namn och ett komma är en parameter som programmet kan läsa med STARTPAR. Till kompilatorn är parametern källfilen, och flaggan L ger en lista och B information för länkaren. Ett program kan alltså delas i moduler som kompileras var för sig. Här har huvudprogrammet `M` proceduren `P` som `EXTERNAL`, och modulen `C` har proceduren och ett tomt huvudprogram. Länkaren tar sina kommandon en rad i taget, `T` för filen den ska skriva, `I` för en modul och `E` för att länka:

```
-PASSYS,B PASCOMP,MF1:M
-PASSYS,B PASCOMP,MF1:C
-PASSYS PASLINK
T MF1:MC
I MF1:M
I MF1:C
E
Linking complete.
-PASCAL MF1:MC
14
```

Länkaren har också `P` för en lista över modulerna, segmenten och de globala variablerna och `A` för att avbryta, och kommandona kan stå i en fil.

Editorn och skyddet

Editorn är en radeditor med kommandon om två bokstäver. `EDIT MF1:M.PAS` ger uppmaningen `>`, och `RE` läser in filen, `PR` skriver ut den med radnummer, `DL` tar bort en rad och `IL` infogar rader efter en, med nummer som `4.01` och `4.02`.

Skivan har ett kopieringsskydd. Tolken läser 64 sektorer och räknar dem som inte går att läsa, och originalskivan har ett 14-tal sådana. På en kopia stämmer inte antalet, och då tömmer tolken minnet. Provet ligger i tolkens bofasta del och är krypterat.

Två sätt att köra Pascal

Pascal 800 och Pascal i det här kapitlet går båda i BASIC-läget, men på olika sätt. Pascal 800 delar kompilatorn i segment och har två tolkar, och kan därför vara ett helt UCSD-system med länkare. Kapitlets kompilator ryms i ett enda stycke och är därför en delmängd.

Del III

Kontoret

Ord 800 och Kalkyl 800

11.1 Vad Luxor sålde

Luxor sålde två egna kontorsprogram till ABC 800 och dess efterföljare: ordbehandlaren Ord 800 och kalkylprogrammet Kalkyl 800. Båda står i prislistornas avsnitt om programvara för ABC 800-familjen, som omfattade ABC 800, ABC 802 och ABC 806, och båda står kvar i den sista listan, 1-1987. Ord 800 kostade 3 500 kr i alla listor från 1983 till 1987. Kalkyl 800 kom ut i december 1982 och kostade först 1 750 kr, i december 1984 2 500 kr och från 1985 3 000 kr. Från 1986 fanns båda också för nätet LUX-NET, och kostade då 7 000 och 5 250 kr. Till återförsäljarna kostade de omkring 60 procent av cirkapriset (Luxors prislistor 1982–1987).

Program	Art. nr	Cirkapris, kr
Ord 800	68 20031-	3 500
Kalkyl 800	68 22011-	1 750–3 000
Diagram II 800	68 22031-	3 000–3 500
Brevrutin 800	68 20041-	550–650
Bas-Ord-Brev	68 28011-	7 500

Priserna är utan moms. Diagram 800 och Diagram II 800 ritar diagram av siffror från Kalkyl 800, och Brevrutin 800 skriver brev. Bas-Ord-Brev har ingen beskrivning i listorna, men namnet och priset visar vad det är. I 1987 års lista kostar det 7 500 kr, och Basregister 800, Ord 800 och Brevrutin 800 kostar 3 500, 3 500 och 650 kr, tillsammans 7 650 kr. Det är alltså de tre programmen i ett paket (Luxors prislistor 1982–1987).

Programmen såldes genom Datorhandlarna Team 100. Luxors katalog *Svenska programbörsen* 1984 skiljer på ”Luxor-program”, som ingick i Luxors programbibliotek och såldes av Team 100, och program från andra, som såldes direkt av dem som gjort dem. Katalogen har ett avsnitt om kalkylprogram och ett om ordbehandling, och där finns, förutom Luxors två program, tillägg och färdiga modeller till dem (Programbörsen 1-84, s. 38, 68).

11.2 Ord 800

Ord 800 är gjort av RTB-program i Lund. Handboken är från 1981 och skriven för ABC 800 M, med 24 rader om 80 tecken, men en rad i dokumentet får vara 126 tecken lång. Programmet går också på ABC 800 C, med 40 tecken på raden. Hela dokumentet ligger i arbetsminnet, och därför får det vara som mest omkring fem fullskrivna A4-sidor (Ord 800, s. 2–3).

Handboken räknar upp de principer som programmet har byggts efter. Det ska vara lätt att lära. Inget är dolt: det som står på skärmen är just det som ligger i minnet. Det ska behövas få tangenter, och i början räcker fem: vänster- och högerpilen, RETURN, PF8 som raderar och PF1 som ger kommandona. De andra kommandona står hela tiden överst på skärmen, och ett kommando väljs med pilarna och RETURN, ”vilket motsvarar att peka på en meny”. Programmet avstavar och bryter raderna medan man skriver, så att man inte behöver titta på skärmen. En text flyttas, kopieras eller tas bort genom att man först markerar den (Ord 800, s. 3–4).

Programmet startas med RESET eller `RUN START` och tar omkring 15 sekunder att läsa in. En mall

läggs över funktionstangenterna PF1–PF8. Programskivan i `DR0:` är skrivskyddad, och dokumenten sparas på `DR1:`. Skärmen har två kommandorader, en rad för programmets frågor och en räknare för sida, rad och position (Ord 800, s. 5).

RETURN avslutar ett stycke, och slutet märks med en vinkel, <, i högermarginalen. Förkortningar sparas som vanliga dokument, en ”texthandbok” med ord och fraser. Kommandot `summa` lägger ihop talen i ett markerat område, ett tal per rad, och behåller decimaltecknet och skrivsättet med `./.` för minus. Med `byt` byts en text ut i hela dokumentet. På ABC 800 C bryts raden vid 38 tecken i stället för 78, en rad får vara 114 tecken i stället för 126, och en markerad text visas mot vit bakgrund i stället för att blinka (Ord 800, s. 20, 29, 46).

Version 2.4 av Ord 800 känner igen ABC 802. Startprogrammet läser byten på adress 39 i ROM:en, som är 3 på ABC 802, och väljer en egen väg för den. Programmet är skrivet i BASIC, med en del i maskinkod, `YRASM.ABS`, som läses in till `$BB00`–`$DC00` och `$F900`–`$FAFF`. Det letar efter de extra 32 kilobyte, som på ABC 802 är RAM:et under `$8000`, och skriver `Graphic 32 K` när det har hittat dem (se kapitlet Minnet, avsnittet 64 kilobyte RAM, s. 38). BASIC-delen går in i maskinkoden genom en hopptabell på `$F900` (Ord 800, skivan).

Skivan har ett kopieringsskydd. En rutin på `$BE37`–`$BEFF` är krypterad, med varje byte XOR:ad med den låga byten i sin adress. Koden före den skriver om sig själv, så att en instruktion blir ett anrop av den rutin som packar upp. Rutinen ger skivkontrollern två kommandon. Det första är `10 1d` med sektor 0, där d är enheten, och det andra, `02`, hämtar kontrollerns buffert på 256 byte. Byte 1 i bufferten ska vara `$42`. Med en annan inställning, som tar sektorn och svaret ur en annan rad i samma tabell, är det sektor 4 035 och `$10`. Är den något annat, blir sidan `$F900`–`$F9FF` skräp, och det första anropet genom hopptabellen startar om BASIC. Är den rätt, packas i stället sidan `$C600` upp (Ord 800, skivan).

Vad kommandot gör framgår av programmet i ABC 830:s kontroller, Luxor 55 10828, som är en Z80 med 2 kilobyte EPROM. Kontrollern delar upp kommandobyten bit för bit, och bit 4 går till en rutin som läser nästa byte. Med `$10`–`$13` där söker den till spåret för sektorn och ger skivkretsen MB8876 kommandot `$C4`, READ ADDRESS, som läser in sex byte ur nästa sektorhuvud: spår, sida, sektor, längd och två byte kontrollsumma. MB8876 har inverterad databuss, och programmet skriver därför `$3B`. Byte 1 är alltså sidnumret i sektorhuvudet på spår 0. En vanlig skiva har 0 där, men originalskivan är formaterad med `$42` i alla huvudena på spår 0, och avbilden av skivan har dem kvar. Kopieras skivan med DOS:en, kopieras bara sektorernas innehåll, och kopian får nollor i huvudena (programmet i kontrollern, BASF 6106, `basf .02.7c`; Ord 800, skivan).

11.3 Kalkyl 800

Kalkyl 800 är gjort av Watchdog Computer Assistance i Täby, och handboken är skriven av Idé-Data i Täby. Den är från december 1982. Programmet är ett kalkylblad av samma slag som VisiCalc. Handboken förklarar det så: ”bildskärmen = pappret, tangentbordet = pennan, mikroprocessorn = räknemaskinen”. Varje ruta i en stor matris kan ha ett tal, en formel eller en text, och bildskärmen är ett fönster mot matrisen, som kan delas så att flera delar syns på en gång (Kalkyl 800, s. 1–2).

Matrisen har 250 rader och 702 kolumner. Kolumnerna heter A–Z och sedan AA–ZZ, och en ruta heter till exempel A1 eller CQ210. Hur många rutor som kan användas beror på minnet. Kommandona börjar med ett snedstreck, som `/FA`, som avslutar, och de är ett dussintal, bland dem att infoga en kolumn, att ta bort en rad, att redigera en ruta och att skriva ut. De 24 funktionerna börjar med `&`: bland andra `&SUM`, `&AVERAGE`, `&COUNT`, `&MAX`, `&MIN`, `&NPV` för nuvärdet, `&LOOKUP` för sökning i tabeller, `&IF`, `&AND`, `&OR`, `&NOT`, `&PI` och `&SIN` (Kalkyl 800, s. 17, 46).

Programmet vill ha ABC 800 M eller C, en skivenhet ABC 830, 832 eller 838 och extra RAM-minne. Det startar som Ord 800, med RESET eller `RUN START`, och visar en meny. Där finns konfigureringen, som tar Luxors minneskort om 16, 64 och 128 kilobyte, Facits om 32 och 128 kilobyte och

MYAB:s UNI800 om 64–256 kilobyte, och inställningen av skrivaren. Inställningarna sparas på skivan (Kalkyl 800, s. 3, 67–68). Demoskivan, version 2.13, har dessutom valen `ABC802 32K` och `ABC806`. På ABC 802 är det extra minnet alltså RAM:et under `$8000`, som för Ord 800 (Kalkyl 800, demoskivan).

11.4 Filerna emellan

Ett kapitel i handboken till Kalkyl 800 handlar om hur programmet utbyter data med Diagram 800, Ord 800, Basregister 800 och egna program. Allt går genom filer på skivan (Kalkyl 800, s. 57):

- `/FD` skriver siffror ur kalkylen till en grafikfil, `.GRA`, som Diagram 800 läser och ritar som ett av sex slags diagram.
- `/SF` skriver en rektangel ur kalkylen till en textfil, `.TXT`, som Ord 800 läser, så att tabellen kan ingå i en längre text. Raderna får vara högst 126 tecken, eller 114 med 40 tecken på skärmen. Ord 800 kan inte lämna något tillbaka: ”Kommunikationen är enkelriktad!”
- `/FT` läser in poster i kalkylen, och `/FR` sparar rutor som poster. Från Basregister 800 läses en post per fil.
- Ett eget program i BASIC kan läsa `/SF`-filen rad för rad med `INPUT LINE`, och det kan skriva en fil med poster som `/FT` läser.

En post har rutans namn, ett mellanslag och rutans innehåll, ett tal eller en formel. En post med ett snedstreck efter namnet ger rutans format, åtta tecken som vart och ett är punkt eller `X` (Kalkyl 800, s. 59–64). Ett BASIC-program kan alltså bygga en liten modell, som Kalkyl 800 sedan läser med `/FT`:

```
100 PREPARE "SUMMA.TXT" AS FILE 1
110 PRINT #1,"A1 100"
120 PRINT #1,"A2 150"
130 PRINT #1,"A3 200"
140 PRINT #1,"A4 &SUM(A1:A3)"
150 CLOSE
160 END
```

Programmen kunde inte vara i minnet på en gång. Den som ville ha ett diagram av kalkylen gjorde `/FD` och fick sedan ”ta ut programskivan med KALKYL 800 och sätta i DIAGRAM 800 istället”, och trycka på RESET. Till Ord 800 var det samma sak (Kalkyl 800, s. 59, 61). Kontoret var tre program och en dataskiva, och det som höll ihop dem var filernas form.

11.5 Andra än Luxor

Runt Luxors två program sålde andra företag tillägg och innehåll. I *Svenska programbörsen* 1984 finns SKRIVSTOPP 800 från Norrköpings Datorbutik, som lägger in koder i ett dokument från Ord 800 så att utskriften stannar, till exempel för att man ska kunna byta typhjul. FOTOSÄTT 800 från Grafisk Kommunikation gör om dokument från Ord 800 för fotosättning, och Svensk Byggtjänst sålde texterna i REDA 83 på skivor för Ord 800. Till Kalkyl 800 fanns färdiga modeller: ANALYS OCH BUDGETSIMULERING från Pollux Data i Örebro, flera utgåvor av SE OM DITT FÖRETAG och en LIKVIDITETSBUDGET (Programbörsen 1-84, s. 24, 40–43, 69–70).

Det fanns också andra ordbehandlare. EDIT 800 från NANCO Elektronik är en textredigerare där funktionstangenterna används som i Ord 800, och ORDET från Lidén Data i Enköping går på ABC 80 med extraminne och på ABC 800, 802 och 806 (Programbörsen 1-84, s. 17, 71). P-data AB i Lund sålde ORD III till ABC 802, ABC 806 och Facit DTC 2, med automatisk avstavning, kopplade brev och summering i tabeller (P-datas annons). I skolprislistan 1986 är ordbehandlaren ORD III och inte Ord 800, medan Kalkyl 800 och Diagram II 800 ingår i baspaketet (Luxors prislistor 1982–1987).

Den tredje vägen var CP/M. Prosoft AB i Stockholm sålde WordStar med MailMerge, CalcStar och InfoStar till ABC-datorerna. I WordStar var kommandona lagda på ABC:s funktionstangenter, och träningshandboken var svensk. Programmen gick bara under CP/M och krävde en skivenhet om 640 kilobyte eller 1,2 megabyte (Programbörsen 1-84, s. 71–73). Luxor sålde inte själv CP/M. Från 1985 står det i prislistorna att MYAB säljer det, och i skolprislistan 1986 att skolorna måste ha CP/M för att få statsbidrag (Luxors prislistor 1982–1987). CP/M vill ha RAM från adress 0, där ABC 800 har sin ROM med BASIC. På ABC 800 behövdes därför ett extra kort för CP/M, som MYAB:s UNI-800. ABC 802 kan själv koppla bort ROM:en och lägga RAM där, så på den behövs inget sådant kort (se kapitlet CP/M på ABC 802, s. 55).

ORD och KALKYL

Till den här boken hör en programskiva med tre kontorsprogram för ABC 802: ordbehandlaren ORD, kalkylprogrammet KALKYL och VERK, som är de två i ett. De är inte program från den tiden och inte kopior av Luxors. De är skrivna efter samma uppgift, en text att skriva och rätta och en kalkyl att räkna om, men på ABC 802:s villkor: med det andra RAM:et som RAM-skiva, bildkretsen med 25 rader och ROM:ens egna flyttal. Det här kapitlet beskriver ORD och KALKYL och jämför dem med Ord 800 och Kalkyl 800 (kapitlet Ord 800 och Kalkyl 800, s. 70). VERK har ett eget kapitel (kapitlet VERK, s. 80).

12.1 Programskivan

Programskivan har Forth, `FORTH.ABS`, och de tre programmen. När maskinen startar läser DOS:en `BASICINI.SYS`, som laddar Forth och visar `RUN ORD`, `RUN KALKYL` och `RUN VERK`. Forth är bokens Forth för ABC80, flyttat till RAM:et på `$8000` och anpassat till BASIC II och DOS:en. Det startas från BASIC med `Z=CALL(32768)` och går tillbaka till BASIC med `BYE`. Mellan Forth och BASIC går en gräns, som för Forth ensamt ligger på `$B800`. Forth är 3–6 gånger fortare än BASIC II (Programskivan).

ORD är ett BASIC-program med Forth i REM-raderna. En rad som börjar med `REM FORTH` hoppar BASIC över, och `Z=CALL(32768,2)` låter Forth läsa alla sådana rader i programmet och översätta dem. Det tar omkring 25 sekunder. Sedan anropar BASIC ordet `ORD`: med `Z=CALL(32768,VAROOT(A$))` får Forth strängen `A$` och utför den som en rad. Menyn och introduktionen är vanlig BASIC, och det som måste gå fort, att visa texten och flytta den, är Forth. KALKYL är för stort för det sättet. Det ligger färdigöversatt på skivan som `KALKYL.ABS`, 21 815 byte, och `RUN KALKYL` läser in det med DOS:ens `DRUN` på några sekunder (Programskivan).

Luxors två program var också BASIC med en del i maskinkod. Ord 800 hade `YRASM.ABS` och en hopptabell på `$F900`. Skillnaden är var maskinkoden kommer ifrån: där var den färdig på skivan, här görs den av Forth i datorn.

12.2 ORD

ORD gör det som behövs för att skriva och rätta en text. Det skriver in där markören står och skjuter undan resten, bryter raderna vid ett blanksteg, så att `RETURN` bara behövs när ett stycke slutar, tar bort tecknet före eller efter markören och flyttar markören ett tecken, en rad eller en sida i taget. Den nedersta raden visar hela tiden var markören står, hur lång texten är och hur mycket som är fritt. ORD kan ännu inte spara texten, skriva ut den eller söka i den; det gör VERK (ORD, avsnitt 1.3).

Ord 800 hade kommandona på skärmen och funktionstangenterna. PF1 gav kommandona, som stod överst på skärmen och valdes med pilarna, och en mall låg över PF1–PF8. ORD har inga kommandon i texten, bara tangenter med `CTRL`, och de är lånade från WordStar. `CTRL-E`, `CTRL-S`, `CTRL-D` och `CTRL-X` är ett kors under vänster hand för upp, bakåt, framåt och ned. Bredvid ligger `CTRL-A` och `CTRL-F` för radens början och slut och `CTRL-R` och `CTRL-V` för en sida upp och ned (ORD, avsnitt 3.1, 3.4).

Skälet är tangentbordet. ABC 802 har pilar för vänster och höger men inte för upp och ned, och ingen DEL eller ESC. Vänsterpilen ger koden 8, samma som CTRL-H, och högerpilen 9, samma som CTRL-I (se kapitlet Tangentbordet, avsnittet Att skriva en rad, s. 33). ORD gör därför som BASIC när en rad rättas: vänsterpilen tar bort tecknet före markören. Att gå bakåt utan att ta bort något blir CTRL-S, som i WordStar. CTRL-G tar bort tecknet efter markören. CTRL-Q gör ORD om till ESC, 27, och det ger menyn: fortsätta, ny text, introduktionen eller sluta. Q sitter längst till vänster bredvid CTRL, så att det går med en hand (ORD, avsnitt 3.4 och 4).

Båda programmen bygger på att det som står på skärmen är det som ligger i minnet. Ord 800 räknar det bland sina principer, och i ORD gäller det av sig självt, eftersom det inte finns något annat att visa: inga koder för fetstil eller understrykning, ingen sidbrytning. Ord 800 hade dessutom avstavning, markerade stycken att flytta och kopiera, förkortningar, byte av en text och kommandot summa, och det var ett program för kontoret. ORD är ungefär där de första ordbehandlarna för hemdatorer var, en skrivmaskin på skärmen (ORD, förordet).

12.3 Luckan på RAM-skivan

Texten ligger inte i det RAM som BASIC och Forth använder, utan på RAM-skivan: de nedre 30 kilobyten, $0000–$77FF, som bara nås med ROM:ens LDIR7F på $7FFD (se kapitlet Minnet, avsnittet 64 kilobyte RAM, s. 38). Forth har tre ord för det: RMOVE flyttar ett stycke åt båda hållen, och RC@ och RC! läser och skriver en byte. ORD använder 27 392 byte från $0000, omkring tio maskinskrivna sidor. Ord 800 hade dokumentet i arbetsminnet, och på ABC 800 M räckte det till omkring fem sidor (Programskivan; Ord 800, s. 3).

Texten har en lucka där markören står. Det som står före markören ligger från början av området till GS, och det som står efter från GE till slutet. Ett tecken som skrivs hamnar på GS, som ökar med ett. Vänsterpilen minskar GS och CTRL-G ökar GE. Inget annat tecken flyttas. Luckans storlek, GE minus GS, är det som lägesraden kallar fritt. Först när markören flyttas går tecken från ena sidan av luckan till den andra (Programskivan).

Framåt flyttar RMOVE texten efter luckan ned i ett svep. Bakåt måste texten flyttas upp, och LDIR7F kopierar bara framåt, så ett stycke som flyttas med den skriver över sig självt. Ordet VANSTER flyttar därför 256 tecken åt gången genom en buffert i vanligt RAM, bakifrån. Med 20 000 tecken tar det 0,18 sekunder att gå från början till slutet och 0,44 sekunder att gå tillbaka (Programskivan).

12.4 Skärmen

ORD ställer bildkretsen på 25 rader. Raderna 0–23 visar texten och rad 24 läget. För att rulla ändrar ORD bara bildens början med 80 och ritar den nya raden och lägesraden; resten av bilden står kvar (se kapitlet Bildskärmen, avsnittet 25 rader, s. 21). Bildkretsens markör, i R14–R15, är släckt medan ORD arbetar, med R10 lika med 41, och blinkar när ORD väntar på en tangent, med R10 lika med 73 (Programskivan).

Att rita hela skärmen tar omkring 0,16 sekunder, och det är för långsamt för varje tangent. ORD minns därför var texten ändrades och ritar från den första rad som påverkas, tills raderna börjar där de gjorde förut. Resten flyttas bara i tabellen över radernas början. Medan ORD ritar läser det tangenterna till en egen kö, eftersom ROM:en bara har plats för en tangent (se kapitlet Tangentbordet, avsnittet Datorns sida, s. 33). Med 17 tecken i sekunden tappas inget, inte heller när skärmen rullar (Programskivan).

12.5 KALKYL

Kalkyl 800 var ett kalkylblad som VisiCalc, med 250 rader och 702 kolumner och rutor som `A1` och `CQ210`. KALKYL har inget rutnät. Varje cell har ett namn, som `PRIS` eller `MOMS`, och cellerna står i en lista, en per rad, i den ordning de skrevs in. En formel blir då `NETTO PRIS*ANTAL` och inte `C3=A3*B3`, och den går att läsa utan att se på arket (KALKYL, avsnitt 1.2).

Förebilderna är två program i ABC-klubbens programbank. CALC från 1985, ursprungligen för ABC80, är ett rutnät där varje ruta har en enda funktion av sex slag, och villkor går inte att skriva. SPRIT från 1986 är gjort efter CALC, som inte räckte till. Det har en lista med namngivna variabler, och formlerna är BASIC-satser som fogas till ett program och kedjas in. De räknas i nummerordning, så en formel som pekar framåt får förra omgångens värde (CALC och SPRIT). KALKYL tar SPRIT:s namn och gör om de två svagheterna: formlerna översätts direkt, och cellerna räknas i den ordning de behövs.

KALKYL har ingen skärmbild att flytta runt i. Kommandona är Forth-ord som skrivs på en rad, och Forth svarar `OK`. `SATT` sätter en cell till ett tal, en formel eller en rubrik med `"` först, `TABORT` tar bort den, `VISA` listar cellerna med namn, värde och formel, och `NY` tömmer kalkylen. Tolv stolar med moms (KALKYL, avsnitt 2.2):

```
SATT PRIS 1250  OK
SATT ANTAL 12  OK
SATT NETTO PRIS*ANTAL  OK
SATT MOMS NETTO*0.25  OK
SATT SUMMA NETTO+MOMS  OK
VISA
PRIS                 1250
ANTAL                  12
NETTO               15000  PRIS*ANTAL
MOMS                 3750  NETTO*0.25
SUMMA               18750  NETTO+MOMS
 OK
```

Formlerna har de fyra räknesätten, parenteser och minus framför ett tal. Funktionerna `SUMMA`, `MEDEL`, `MIN`, `MAX` och `ANTAL` tar ett område, `JAN..DEC`, som är cellerna från `JAN` till `DEC` i listans ordning, eller en lista med semikolon, `A;B;C`. `OM(NETTO>1000;0;95)` väljer med en jämförelse, det som CALC saknade. Kalkyl 800 hade 24 funktioner, bland dem nuvärdet, tabellsökning och sinus. KALKYL har högst 40 celler, namnen högst åtta tecken, och potenser går ännu inte att räkna (KALKYL, avsnitt 1.4, 3.4 och 3.5).

12.6 Formeln blir Forth

När `SATT MOMS NETTO*0.25` skrivs in läser KALKYL formeln tecken för tecken, med rekursiv nedstigning: ett uttryck är termer med `+` och `-` emellan, en term är faktorer med `*` och `/`, och en faktor är ett tal, ett namn, en funktion eller ett uttryck inom parentes. Medan formeln läses skriver KALKYL ett Forth-ord utan namn i ordlistan. Det börjar som ett vanligt kolonord, med ett anrop av Forths ingång för kolonord, och det slutar med cellens adress, `F!` och `EXIT`. Ett tal blir `(FLIT)` med talets åtta byte, och ett namn blir cellens adress och `F@`. Formeln ovan blir ungefär

```
NETTO F@  (FLIT) 0.25  F*  MOMS F!  EXIT
```

Ordets adress sparas i cellen. När kalkylen räknas om körs orden, och ingen formel behöver läsas om. Funktionerna är Forth-ord som heter `=SUMMA`, `=MAX` och så vidare och lägger sin egen kod, och `OM` blir ett hopp med `0BRANCH` och `BRANCH`, som `IF` och `ELSE` i Forth. Formelns text sparas på RAM-skivan, 80 byte per cell från 27 392, och det är den som `VISA` visar (Programskivan).

Ett fel i formeln anges med det tecken där KALKYL märkte det, till exempel `FEL: ) saknas vid tecken 12`. Det halvfärdiga ordet tas bort, och cellen behåller sitt gamla innehåll. Det finns 17

sådana fel. Kompilatorn räknar också hur många tal formeln lägger på flyttalsstacken och vägrar mer än 16 (Programskivan; KALKYL, kapitel 7).

SPRIT gjorde något liknande med BASIC: satserna fogades till ett program, som kedjades in och tolkades. I KALKYL blir formeln trådad kod, som Forth själv, och översättningen tar så lång tid som det tar att skriva den.

12.7 Ordningen

Varje formel har en mängd med de celler den läser, 64 bitar, en för varje cell. Av mängderna gör KALKYL en ordning där en cell kommer efter alla dem den läser. Därför spelar det ingen roll var i listan en formel står. Kommer inte alla celler med i ordningen finns en cirkel. Då letar KALKYL upp den kortaste vägen runt och skriver den (KALKYL, avsnitt 3.6):

```
SATT A 1  OK
SATT B A+1  OK
SATT C B*2  OK
SATT A C-3 FEL: cirkel: A B C A  OK
```

`A` behåller sitt gamla innehåll. En formel som delar med noll eller ger ett för stort tal får värdet `fel`, och felet sprids längs beroendena till alla celler som läser den.

När ett tal ändras räknas bara de celler om som beror på det. Med 40 celler tar en ändring som inte ger nya beroenden under en tiondels sekund, och en som gör att alla 40 räknas om omkring 0,4 sekunder. `TABORT` och en ny cell mitt i listan flyttar cellernas nummer, och då måste alla formler översättas på nytt. Det tar över fyra sekunder (Programskivan).

12.8 ROM:ens tal

KALKYL räknar inte med egna flyttal. Det anropar BASIC II:s rutiner i ROM:en: `FADD` på `$4F41`, `FSUB` på `$4F39`, `FMUL` på `$4FEA`, `FDIV` på `$505F` och `FCOMP` på `$4C61`, och `TXTNUM` på `$47F3` och `FTODEC` på `$5153` för att läsa och skriva tal. Talen läggs på Z80:ans stack före anropet, och resultatet ligger där efteråt. Längden står på `$FF1E`. KALKYL sätter den till åtta byte, DOUBLE med 16 siffror, och lägger tillbaka BASIC:s värde efteråt. Ett tal i KALKYL är alltså en variabel i BASIC med DOUBLE, och KALKYL räknar och avrundar precis som BASIC (Programskivan).

Mallarna är också BASIC:s. `FORMAT PRIS ####.##` ger cellen en mall som i `PRINT USING`. `USEFLD` på `$4713` läser mallen, och `FMUSE` på `$4575` skriver talet efter den. Får talet inte plats skrivs `%` och talet i fri form, med `PRFREE` på `$4504`, som i BASIC. `SKRIV` skriver kalkylen med mallarna på skrivaren, `PR:`, genom ROM:ens drivrutin. Kalkyl 800 hade sina format som åtta tecken, punkt eller `X` (Programskivan; Kalkyl 800, s. 64).

12.9 Omgångarna

SPRIT kunde skriva ut upp till fem omgångar sida vid sida och markera de värden som hade ändrats. KALKYL gör samma sak med två ord. `OMGANG` sparar alla cellers värden som nästa omgång, högst fyra, och `JAMFOR` visar dem bredvid varandra, en rad per cell, med # efter ett värde som skiljer sig från omgången före. Offerten sparad som den laddades, med rabatten 0.1 och med 20 stolar (KALKYL, avsnitt 2.5):

```
JAMFOR
                        1               2               3
RUBRIK   Offert 1984-03-12
PRIS               1250.00         1250.00         1250.00
ANTAL                12              12              20#
```

```
RABATT          0.05          0.10#         0.10
NETTO       14250.00      13500.00#     22500.00#
MOMS         3562.50       3375.00#      5625.00#
SUMMA       17812.50      16875.00#     28125.00#
 OK
```

En omgång ligger på RAM-skivan från 16 384, nedanför överläggen, med 18 byte per cell: namnet, typen och värdet med ROM:ens åtta byte. `JAMFOR` söker posten på namnet och inte på numret, så det går att lägga till och ta bort celler mellan omgångarna; en cell som saknas i en omgång får ett tomt fält. Ordet finns bara i KALKYL:s fasta del och tar 1 164 byte (Programskivan). `OMGANG NY`, `NY` och `LADDA` tömmer omgångarna, och de sparas inte i filen.

12.10 Minnet och överläggen

KALKYL flyttar gränsen mot BASIC till `$D800`. Då har Forth 2 397 byte fritt för formlernas kod och BASIC omkring 6,5 kilobyte. Det räcker inte till allt på en gång. Det som bara behövs när något ändras eller visas ligger därför i två *överlägg*, som hämtas från RAM-skivan till samma plats i minnet. Överlägg 1 är formelkompilatorn med `SATT`, `TABORT`, cirkelprovet och ordningen, 3 490 byte. Överlägg 2 är `VISA`, `FORMAT`, `SPARA`, `LADDA`, `SKRIV` och tabellen, 4 840 byte; det ligger i en egen fil, `KALKYL.OVL`, som läses till RAM-skivan när KALKYL startar. Det fasta, cellerna, formeltexterna och omräkningen, finns hela tiden i minnet (Programskivan).

Ett överlägg hämtas med `RMOVE` bara när ett annat ligger där, och det tar några hundradels sekunder. För varje överlägg finns platsen på RAM-skivan, längden och en summa av bytena. Stämmer summan inte efter hämtningen har RAM-skivan skrivits över, och KALKYL skriver `FEL: överlägget på RAM-skivan är överskrivet` och måste laddas igen. Det händer om ORD har körts, för ORD:s text ligger på samma plats. ORD och KALKYL får alltså inte plats i maskinen samtidigt, varken i RAM:et eller på RAM-skivan (Programskivan).

Kalkyl 800 hade samma problem från andra hållet. Det ville ha extra minne och hade en konfigureringsmeny för Luxors, Facits och MYAB:s minneskort, och på demoskivan valet `ABC802 32K` (avsnittet Kalkyl 800, s. 71). Båda programmen använder alltså de 32 kilobyten under `$8000` på ABC 802 som det som andra maskiner fick köpa till.

12.11 Filerna

`SPARA OFFERT` skriver kalkylen till `OFFERT.KAL` på skivan, och `LADDA OFFERT` läser den igen. Filen är en vanlig textfil med en cell per rad, namnet, ett mellanslag och innehållet som det skrevs efter `SATT`. Mallarna står efter cellerna, med `%` före namnet, och sist kommer tabellen, en rad text per rad med `%=` först. I tabellen byts varje cellnamn mot cellens värde med dess mall, högerställt där namnet slutade, så att `TABELL` på skärmen och `SKRIV` på skrivaren ger en uppställd offert (KALKYL, avsnitt 6.2):

```
RUBRIK "Offert 1984-03-12
PRIS 1250
ANTAL 12
RABATT 0.05
NETTO PRIS*ANTAL*(1-RABATT)
MOMS NETTO*0.25
SUMMA NETTO+MOMS
%PRIS ######.##
%RABATT #.##
```

Det är nästan samma form som posterna i Kalkyl 800, där en post har rutans namn, ett mellanslag och innehållet, och en post med snedstreck ger formatet (avsnittet Filerna emellan, s. 72). Ett BASIC-program kan läsa filen med `INPUT LINE` och skriva en med `PRINT #`, och på så vis göra en

kalkyl åt KALKYL. `LADDA` lägger in alla rader först och översätter formlerna sedan, så en formel får använda en cell som står längre ned i filen. Programskivan har två kalkyler, en offert och en hushållsbudget (KALKYL, avsnitt 6.2 och 6.3).

KALKYL anropar DOS:en direkt, inte genom BASIC. Forth går in i DOS:ens hopptabell, på `$6015` för `DPREP`, `$6018` för `DOPEN`, `$6021` för `DCLOSE`, `$602A` för `DSETRC`, `$602D` för `DRDREC` och `$6030` för `DWRREC`, med bufferten på `$F600`. Medan anropet pågår pekar DOS:ens felutgång på `$FD33` på ett Forth-ord, som tar tillbaka stacken och ger felet, så att ett skivfel blir `FEL: skivan, fel n` och inte ett BASIC-fel (Programskivan).

Med Luxors program bytte man programskiva mellan kalkylen och texten. Med ORD och KALKYL ligger båda på samma skiva, men de går inte heller att ha i minnet på en gång, och ORD har ännu ingen fil att ta emot en tabell i. Det är det VERK gör: ORD:s skärm och KALKYL:s celler i samma program.

VERK

Med Luxors program skrevs offerten i Ord 800 och räknades i Kalkyl 800, och siffrorna gick från det ena till det andra genom en fil på skivan, med byte av programskiva emellan (se kapitlet Ord 800 och Kalkyl 800, avsnittet Filerna emellan, s. 72). VERK är det tredje programmet på programskivan och gör båda i samma text. Det är ORD:s editor med KALKYL:s celler. Ett ställe i texten kan vara ett *fält*, som visar värdet av en cell, och när en formel ändras räknas texten om (Programskivan).

Förebilden är Jef Raskins Canon Cat från 1987. Den såldes som en ”work processor” och inte som en ordbehandlare, den hade ett enda långt dokument i stället för olika program, och ett uttryck i texten kunde räknas ut på plats. Cat var till stor del byggd i Forth. Raskins företag Information Appliance hade en egen Forth att utveckla i, och den som slog på en dold inställning kunde programmera Cat i Forth (Canon Cat). Namnet VERK kommer därifrån, som ORD kommer från ordbehandlare. VERK går ett steg längre än Cat med KALKYL:s celler: ett fält har ett namn och en formel, och formlerna kan läsa varandra.

13.1 Brevet

`RUN VERK` läser in programmet och visar introduktionen. `CTRL-Q` ger menyn, som har ORD:s fyra val och tre till: spara, läsa och skriva ut. Med 7 och `OFFERT` läser VERK brevet `OFFERT.VRK` från programskivan. Fälten syns inverterade, och den nedersta raden visar var markören står och, när den står på ett fält, cellens namn och mall:

```
Tack för er förfrågan. Vi erbjuder 12 bordslampor av modellen Kontor för
1250.00 kr. Med 5 % rabatt blir nettot 14250.00 kr, momsen 3562.50 kr och att
betala 17812.50 kr.

VERK markören 222 av 490 fritt 15238  ANTAL 12  ##
```

Markören står på fältet `ANTAL`. `CTRL-K` skriver cellens rad på den nedersta raden, `ANTAL 12`, som den står i en fil från KALKYL. Där kan den rättas med samma tangenter som texten. `RETURN` gör det som `SATT` gör i KALKYL. Med `ANTAL 20` blir första raden `Vi erbjuder 20 bordslampor`, och nettot, momsen och summan räknas om till 23 750, 5 937,50 och 29 687,50 kronor. Texten runt fälten står still. En rad som börjar med `%`, som `%PRIS ####.##`, ger cellen en mall, och utan fält vid markören lägger `CTRL-K` in ett nytt (Programskivan).

Ett fel visas på samma rad med KALKYL:s meddelande, och nästa tangent tar bort det, så att raden kan rättas. `ANTAL 20*` ger `FEL: tal eller namn saknas vid tecken 4`. `CTRL-Q` ångrar, och då ändras ingenting (Programskivan).

13.2 Fälten i texten

Ett fält tar lika många byte i texten som det är brett på skärmen. Den första byten är 128 plus cellens nummer, och resten är 255. Då är en plats i texten fortfarande en kolumn på skärmen, och ORD:s radbrytning, markör och rader fungerar som de är. Bredden är mallens, eller 10 utan mall, och högst 40. Den är fast, så att stycket inte bryts om när värdet ändras. Ett tal som inte får plats blir stjärnor (Programskivan).

Det finns ingen tabell över fälten. När en rad ritas hämtas den från RAM-skivan med `RMOVE`, och en liten rutin i maskinkod letar efter byte med bit 7 satt. Över dem skriver VERK cellens värde med bit 7 satt, och på ABC 802 betyder bit 7 i bildminnet att tecknet visas inverterat (se kapitlet Bildskärmen, s. 17). Värdet räknas alltså fram varje gång raden ritas och ligger aldrig i texten. Markören går över ett fält i ett steg, och vänsterpilen och `CTRL-G` tar bort hela fältet. Bildkretsens markör, ett streck under tecknet, syns inte mot ett inverterat fält. Därför är det namnet på den nedersta raden som visar var den står (Programskivan).

Formlerna, värdena och mallarna ligger i KALKYL:s tabell över cellerna, och formlernas texter på RAM-skivan, som i KALKYL. Ett namn gäller i hela texten. Samma cell kan visas på flera ställen, och en cell behöver inget fält alls: `RABATT` räknas, men i brevet syns bara `PROC`, som är `RABATT*100` (Programskivan).

13.3 Filen

En VERK-fil är en textfil, så att BASIC och ORD kan läsa den. Först kommer texten med fälten som `<NAMN>`, sedan en rad `%%` och cellerna som i en fil från KALKYL. Ett stycke är en rad i filen, och VERK bryter raderna först på skärmen. Ur `OFFERT.VRK`:

```
Tack för er förfrågan. Vi erbjuder <ANTAL> bordslampor av modellen Kontor för ...
%%
PRIS 1250
ANTAL 12
RABATT 0.05
PROC RABATT*100
NETTO PRIS*ANTAL*(1-RABATT)
...
%PRIS ####.##
```

Klamrar hade varit det naturliga, men i ABC 802:s teckenuppsättning är { och } bokstäverna ä och å, och [och] Ä och Å (se kapitlet Tangentbordet, s. 28). < och > är inga bokstäver. Vid läsningen blir bara ett `<NAMN>` med en cell efter `%%` ett fält, och ett annat < i texten förblir text. Utskriften går till PR: med värdena i mallens bredd i stället för fälten och med raderna brutna som på skärmen (Programskivan).

13.4 Minnet

VERK är det största av de tre programmen. Det är Forth, ORD:s editor, KALKYL:s celler och omräkning och fälten, och `VERK.ABS` är 26 758 byte. Gränsen mot BASIC ligger på `$ED00`, högre än för KALKYL, och när VERK går är 1 881 byte fria för formlernas kod. Det som bara behövs ibland ligger i fyra överlägg, som hämtas till samma plats från RAM-skivan: formelkompilatorn (3 490 byte, samma som i KALKYL), att spara och skriva ut (2 051), statusraden med tangenterna och menyn (1 950) och att läsa en fil (2 523). Det sista får inte plats i `VERK.ABS` och är en egen fil, `VERK.OVL`, som läses vid starten. Introduktionen är också en fil, `VERK.HLP`, som läses när den visas (Programskivan).

RAM-skivan delas mellan texten och överläggen. Texten får 15 728 tecken, omkring fem maskin-

skrivna sidor, mot 27 392 i ORD. Det räcker till ett brev eller en offert men inte till en rapport, och det är priset för att text och kalkyl ska rymmas i en ABC 802 på en gång (Programskivan).

Returstacken blev för grund. Under BASIC:s stackpekare har Forth bara 64 byte, och `CTRL-K` som kompilerar om alla formler går genom ett tiotal ord i varandra, från editorn genom `SATT` till formelkompilatorn. Det gav `STACK ?`. VERK flyttar därför datastackens botten 160 byte under BASIC:s stackpekare och tömmer båda stackarna, när det startar. Det går eftersom VERK aldrig går tillbaka till det som anropade det, bara till BASIC eller menyn (Programskivan).

13.5 Varifrån VERK kommer

VERK är skrivet nu, men idén är äldre. Omkring 1981–82 skrev jag ett program för deklarationsblanketten på ABC80. Det skulle likna blanketten så långt det gick, med ledtexter och fält att fylla i, och delarna lästes in från skivan när de behövdes, eftersom allt inte fick plats i minnet på en gång. Förebilden var Isaksson och Kärrsgårds bok om programmering på ABC80, där formulären har fälten mellan < och >, en meddelanderad längst ned och felen som texter i en tabell (Isaksson och Kärrsgård 1980). Räkningen i blanketten blev aldrig klar, och skolarbetet tog tiden just då. Programmet finns inte kvar. Blanketten och vad ABC80 klarade på kontoret står i *ABC80 inifrån. Utanför boken*, kapitlet Deklarationen på ABC80.

Likheterna med VERK är många. Meddelanderaden är statusraden, felens texter är en tabell, introduktionen ligger på skivan, formulären är texten med fälten, och delarna läses in från skivan som överlägg. Skillnaden är att fälten i VERK räknas.

Den andra tråden är fönstren. På en mässa i Sollentuna stod en Xerox Star, och en bit bort, i ett annat bås, en IBM PC. Det var Star som intresserade mig. Det var mot slutet av 1984, kanske i början av 1985. PC:n hade då sålts i Sverige sedan den 18 januari 1983 (Sjöström 1996, s. 129), och Star såldes fram till 1985, då Xerox 6085 tog över. Star hade fönster och dokument med tabeller i, men den kostade för mycket för ett kontor. Fönsteridén på ABC80 står i *Utanför boken*, kapitlet Fönster på ABC80. Olle Wolkert och Christer Ekman träffade jag vid ett mässbesök runt 1984/85, och Lisan hos OWOCO står i förordet.

Canon Cat fick jag veta om senare, och bakom den finns en historia om Apple. Jef Raskin startade Macintosh-projektet på Apple 1979. Han ville ha en billig och enkel dator för vanliga människor, en maskin som gick att använda utan att man först lärde sig hur den var byggd. Den skulle i stort sett styras från tangentbordet, och hans tidiga Macintosh visade mest text. Någon mus och några ikoner fanns inte i planerna (Macintosh).

Steve Jobs ville något annat. Han hade sett det grafiska gränssnittet hos Xerox PARC, samma arbete som blev Star, och när han tog över projektet 1981 förde han det mot fönster, ikoner och mus, i en mer påkostad och sammanhållen produkt. Den Macintosh som kom 1984 var därför till stor del en annan dator än Raskins. Namnet, några av de tidiga idéerna och en del av grundtankarna kom från honom, men inte mycket mer (Macintosh). Raskin lämnade Apple 1982 och startade Information Appliance. Hans idé blev först SwyftCard, ett ROM-kort till Apple IIe, och sedan Cat, som Canon sålde (Canon Cat).

Fönstren från Xerox, som jag såg i Star, och tanken på en enkel maskin med en enda text möttes alltså i ett och samma projekt på Apple, och där skildes de. Om Jobs och Raskin hade arbetat vidare tillsammans i stället för att gå åt var sitt håll, kunde det ha blivit en maskin med både Macintoshs fönster och Cats text som räknar, de två idéer jag hade haft på det tidiga 1980-talet. Varken ABC80 eller ABC 802 hade räckt till båda. VERK får plats bara genom att delarna läses in från skivan, och utan fönster.

13.6 Vad som saknas

Cat hade två tangenter, LEAP, för att söka framåt och bakåt medan man skrev, och det var så man flyttade sig i texten (Raskin 2000). VERK flyttar sig med CTRL-tangenterna som ORD och kan inte söka. Det har inga fönster, ingen fetstil och ingen avstavning, och texten är kort. Men brevet räknar om sig självt när priset ändras, och det gjorde inte Ord 800 och Kalkyl 800 tillsammans, hur många disketter man än bytte.

Källor

Här står varifrån uppgifterna i texten kommer, så att den som vill kan läsa vidare eller pröva dem. I texten citeras en källa med nyckeln inom parentes, till exempel (Falke 1983, s. 127); nyckeln står i fetstil först i posten. Kapitlen om maskinen, bildskärmen, tangentbordet och minnet bygger mest på Luxors servicemanual, och där avser ett ensamt sidnummer, (s. 84), servicemanualen. Under posten står vad texten hämtar ur källan och var källan finns inskannad eller på nätet. Posterna står i fyra grupper: Luxors handböcker och ritningar; prislistor och trycksaker; datablad; böcker, tidskrifter och artiklar.

Luxors handböcker och ritningar

ABC 802 Servicemanual Luxor Datorer: *Dator ABC 802. Servicemanual.* Luxor Datorer AB, Motala, art. nr 66 78088-40, 104 s., utan årtal.
Huvudkällan för kapitlen om maskinen, bildskärmen, tangentbordet och minnet. Maskinen som helhet (s. 4–10, 18–21, 29–30, 42–46, 54), minnet och uppfriskningen (s. 46–50), bildskärmen, 6845:an, teckengeneratorn och bildminnet (s. 66, 76–83), tangentbordet, dess processor, kommandona och kodtabellen (s. 84–88). Tryckt sida är PDF-sida.
Inskannad: `https://www.abc80.net/archive/luxor/ABC80x/ABC802-dator-servicemanual.pdf`

ABC 802 BASIC II Luxor Datorer: *Manual BASIC II. ABC 802.* Luxor Datorer AB, Motala 1982, 120 s.
Kallas i texten också BASIC II-manualen. `WIDTH 40` och `WIDTH 80` (s. 61), grafiken (s. 78–82), `SYS` (s. 76), funktionstangenterna (s. 83, 85) och tangentkoderna i bilaga 3 (s. 104–105). Tryckt sida är PDF-sida minus 4 i början och minus 6 längre fram.
Inskannad: `https://www.abc80.net/archive/luxor/ABC80x/ABC802-dator-manual-BASIC-II.pdf`

Bruksanvisningen Luxor Datorer: *Bruksanvisning ABC 802.* Luxor Datorer AB, Motala 1984, art. nr 66 78088-10, 57 s.
Tangenterna och styrtecknen (s. 13–15), tangentbordets mått (s. 36) och tangentkoderna (s. 40–41). Förordet är daterat 1984-03; artikelnumret är prislistans (Luxor, cirkaprislista 1-83). Tryckt sida är PDF-sida minus 4.
Inskannad: `https://www.abc80.net/archive/luxor/ABC80x/ABC802-dator-bruksanvisning.pdf`

Bruksanvisningen för ABC 806 Luxor Datorer: *Bruksanvisning ABC 806.* Luxor Datorer AB, Motala 1983.
Samma tangentbord som ABC 802: lågprofil enligt DIN 66345 och de tre utförandena ABC 22, ABC 55 och ABC 77.
Inskannad: `https://www.abc80.net/archive/luxor/ABC80x/ABC806-dator-bruksanvisning.pdf`

Bit för bit med ABC 800 (Luxor 1982) Luxor: *Bit för bit med ABC 800.* Luxor Datorer AB, Motala 1982 (framtagen av NANCO Elektronik).
Avancerad programmering på ABC 800. Bildkretsens register från BASIC, R11 för markören och R2 och R5 för att flytta bilden (s. 54), och den 25:e raden på skärmen med `OUT 56` (s. 55). Tryckt sida är PDF-sida minus 4.
Inskannad: `https://www.devili.iki.fi/pub/Luxor/docs/books/`

Ritningen Luxor Datorer: *ABC 802, datorkortet*, blad 1–5: I/O (blad 1), CRTC (blad 2), CPU och bussdrivare (blad 3), Memory part (blad 4, 66 80892-01, 1983) och videominnet (blad 5);

kommunikationsmodulen.
Bildkretsen, klockkedjan, PAL:en och skiftregistret (blad 2 och 5), bildminnet (blad 2, 3 och 5), minnet och grindarna kring M1* (blad 4), tangentbordets ledningar och summern (blad 1 och 3, kommunikationsmodulen).
Inskannad: https://www.devili.iki.fi/pub/Luxor/docs/schematics/abc802/

Ritningen 66 88055-01 Luxor Datorer: *ABC 55/ABC 77*, ritning 66 88055-01, 1983.
Tangentbordets processor, matrisen och ABC 22, ritad som tillvalet ”numerical keyboard ABC22” med kabeln mellan J4 och J2.
Inskannad: https://www.devili.iki.fi/pub/Luxor/docs/schematics/abc55_abc77.gif

LUXNET 1984 Luxor Datorer: *Lokalt datornät LUXNET. Bruksanvisning*. Luxor Datorer AB, Motala 1984, 21 s.
Nätet som helhet, CSMA och HDLC, 500 kbit/s, upp till 32 noder (s. 4–5, 19), centralen ABC 850 med sina kort (s. 6–7, 12), nodkortet och kontakterna (s. 8–10, 19–20), biblioteken och inloggningen (s. 11), skrivarna (s. 13) och PROM:arna till varje dator (s. 14, tabell 1). Tryckt sida är PDF-sida minus 3.

Anvisning UFD-DOS Luxor Datorer: *Anvisning UFD-DOS.* Luxor Datorer AB, Motala 1983, 77 s.
UFD: som en fil med eget bibliotek (”Allmänt om UFD”), filernas skydd med `REPROT` (avsnitt 3.22) och inställningen ”central” för ABC-net i `SYS` (avsnitt 3.23).
Inskannad: https://www.abc80.net/archive/luxor/sw/doc/Anvisning-UFD-DOS.pdf

Luxor serviceinformation 1984 Luxor Datorer: *Serviceinformation. ABC 850, förfarande vid utbyte av winchesterdrivenhet*, 1984-06-14, 1 s., som textfilen MARKHD.INF på systemskivan.
Serienumret som skrivs magnetiskt på winchesterdrivenheten och står på etiketten på baksidan av ABC 850, programmen MARKHD.BAC och GETID.BAC, och att märkningen kan göras före eller efter formateringen och dosgenereringen.
Arkivet: https://www.devili.iki.fi/pub/Luxor/software/ABC-klubben/abc800/systempr/

DataSweden, ABC-net DataSweden: *ABC-net användarmanual* och *ABC-net mjukvarumanual*, utan årtal (LIB i handboken är version 6.03 från 1983-02-02).
Centralkortet i det centrala skivminnet, datorerna Facit DTC, ABC 80 och ABC 800, kabeln upp till 300 meter, MFD och UFD, enhetsnamnen (bilaga A) och filernas skydd W och D.
Inskannad: https://www.abc80.net/archive/luxor/sw/doc/abc-dtc-net-användarmanual.pdf

Ord 800 Luxor: *ORD 800. Ordbehandling på ABC 800.* Luxor AB, Motala 1981, art. nr 66 22003-14. Programmet är utvecklat av RTB-program, Lund.
Sidnumren är handbokens tryckta. Principerna (s. 3), starten och skärmen (s. 5), förkortningarna (s. 20), `summa` och `byt` (s. 29) och skillnaderna mellan ABC 800 M och ABC 800 C (s. 46).
Inskannad: https://www.abc80.net/archive/luxor/sw/doc/ABC800-ord800.pdf

Kalkyl 800 Luxor Datorer: *KALKYL 800. Avancerat kalkyleringsprogram för ABC 800.* Luxor Datorer AB, Motala, december 1982, art. nr 66 22201-14, 100 s. Programmet är gjort av Watchdog Computer Assistance, Täby, och handboken skriven av Idé-Data, Täby.
Sidnumren är handbokens tryckta. Matrisen och fönstret (s. 1–2), utrustningen (s. 3), funktionerna (s. 46), utbytet med andra program (kapitel 6, s. 57–64) och konfigureringen (s. 67–68). DTC KALKYL för Facit DTC 2 (Ericsson Information Systems 1984, version 2.22A, 92 s.) är samma program: handbokens bilagor beskriver `WSYSPROT` och `WSYSCONV`, som också finns på demoskivan för Kalkyl 800.
Inskannad: https://www.abc80.net/archive/luxor/sw/doc/DTC-kalkyl.pdf
Inskannad: https://www.abc80.net/archive/luxor/sw/doc/kalkyl800.pdf

Prislistor och trycksaker

Luxor, cirkaprislista 1-83 Luxor Datorer: *Cirkaprislista Nr 1-83.* Luxor Datorer AB, 2 s.
ABC 802 utan tangentbord 7 600 kr, ABC 806 10 700 kr, ABC 800 M 11 600 kr, NRZI-modulen för ABC 802 840 kr, tangentborden ABC 22, ABC 55 och ABC 77 med artikelnummer och pris.
Inskannad: https://www.abc80.net/archive/luxor/marknad/prislistor/Luxor_Cirkaprislista_Nr_1-83.pdf

Luxors prislistor 1982–1987 Luxor Datorer: cirkaprislistor och konsultprislistor från 2-82 till 1-1987, bland dem *Cirkaprislista Nr 12-84* (s. 36), *Cirkapris ABC 800 I/II 1985* (1985-11-01), *Cirkapris 3-85* och *Prislista 800-serien* 1986 och 1-1987, och skolprislistorna 86-02-01 och 1987-01-01, samlade i en skanning om 136 s. med både daterade och odaterade listor.
Priserna på tangentborden ABC 22, ABC 55, ABC 77 och ABC 99 (230 8450-15); i 12-84 står alla fyra, ABC 77 för 2 860 kr. I Cirkapris 3-85 står ABC 22 och ABC 55 som en post, 2 860 kr. Kontorsprogrammen står i avsnittet om programvara för ABC 800-familjen: Ord 800 (68 20031-), Kalkyl 800 (68 22011-), Diagram II 800, Brevrutin 800 och Bas-Ord-Brev, från 1985 med ett pris för LUX-NET och anmärkningen att CP/M säljs av MYAB. Luxors egen CP/M 160 K (230 8004-01) för 1 800 kr i 2-83. Skolprislistorna har CP/M 2.2 och 3.0 med licenser och kravet på CP/M för statsbidraget (86-02-01) och CP/M 3.0 från MYAB (1987-01-01). Skolprislistan 86-02-01 har ett baspaket med Kalkyl 800, Diagram II 800 och Kontek Datalön 800 och ordbehandlaren ORD III från P-data.
Inskannad: https://www.abc80.net/archive/luxor/marknad/prislistor/

MyAB:s prislista 1985 MYAB Mikrokonsult AB, Sundbyberg: produktblad för UNI802, brev om UNI 802 och prislista, 1 juni 1985.
UNI802 med 64–512 kilobyte, kontrollporten på I/O-adress 6 och dess bitar, mapmoderna och priserna.
Inskannad: https://www.abc80.net/archive/luxor/marknad/prislistor/Prislista_Myab.pdf

MyAB, UNI800 MYAB Mikrokonsult AB, Sundbyberg: *Användning av UNI800* och *Hårdvarubeskrivning av UNI800 rev. 2*, utan årtal, 6 s.
UNI800 med 64–256 kilobyte i fyra banker, kontrollporten på I/O-adress 6, mapmoderna, PROM:en för CP/M och kortet som RAM-disk under UFD-DOS.
Inskannad: https://www.abc80.net/archive/luxor/sw/doc/användning-av-UNI800.pdf

Programbörsen 1-84 Luxor Datorer: *Svenska programbörsen. Programvara för Luxors datorer* nr 1-84, 187 s. Luxors katalog över program till ABC 80, ABC 800 och HX 20, både Luxors egna och andras.
Sidnumren är katalogens tryckta. Kalkylprogram och modeller för Kalkyl 800 (s. 38–43), ordbehandling och dokument skrivna med Ord 800 (s. 68–71), EDIT 800 (s. 17), FOTOSÄTT 800 (s. 24) och Prosofts program under CP/M (s. 71–73).
Inskannad: https://www.abc80.net/archive/luxor/sw/doc/svenska-programbörsen-programvara-för-luxors-datorer-nr-1-84.pdf

P-datas annons P-data AB, Lund: annons för ORD III och MULTIBAS till ABC 802, ABC 806 och Facit DTC 2, utan årtal. Annonsen kallar programmen nyheter och ger priserna 5 500 kr för ORD III och 4 800 kr för MULTIBAS. Eftersom ORD III redan står i skolprislistan 86-02-01 är annonsen från före 1986.
Inskannad: https://www.abc80.net/archive/luxor/marknad/annonser/pdata_abc_ordiii_multibas_annons.pdf

Luxor, produktblad ABC 802 Luxor Datorer: *ABC 802.* Produktblad, 2 s., Luxor Datorer AB, Motala, utan årtal.
”Lilla ABC 800”, ”perfekt som beslutsfattardator och terminaldator”; teledata och terminalrutiner i datorn, hårdvaran för ABC NET inbyggd och programmet ett tillval; tangentbordet ABC 55.

Nokia-Luxors broschyr om ABC 802 Nokia-Luxor: *ABC 802.* Reklambroschyr, 4 s., utan årtal.
Tangentbordet ”värt ett eget kapitel” och bland marknadens bästa enligt Statskontorets ut-

värdering; sladden med sjupolig kontakt.
Inskannad: https://www.abc80.net/archive/luxor/marknad/Nokia-Luxor-ABC-802.pdf

Tekniska museet, ABC 1600 Tekniska museet: *Persondator, Luxor ABC 1600*, TEKS0047538. DigitaltMuseum, läst 2026-11-12.
Datorn levererades till Uppsala universitet 1985 och har bildskärmen ABC 1615 och tangentbordet ABC 99.
https://digitaltmuseum.se/021026360748/persondator

Program

Luxors systemskiva Luxor Datorer: systemprogrammen till ABC 800 och ABC 802, filerna BASICINI.SYS, CMDINT.SYS, LIB.ABS, LIB.BAS, DOSGEN.ABS, MARKHD.BAC och GETID.BAC, ur ABC-klubbens arkiv.
Feltexterna på svenska och autostarten i BASICINI.SYS, och kommandotolken CMDINT.SYS som `BYE` laddar (”Ver 6.02, 1983-04-21, Copyright 1982 Dataindustrier AB”). DOSGEN.ABS (”Disk generation”, ”Ver 6.01, 1983-04-21”) med sina frågor och utskrifter, kommandona `$10` och `$80` till kortet och den ordning det lämnar på en hårddisk; MARKHD.BAC, ”Numrera winchester”, ver. 1.03.
Arkivet: https://www.devili.iki.fi/pub/Luxor/software/ABC-klubben/abc800/systempr/

PROM-listan på abc80.net abc80.net: förteckningen över Luxors EPROM:ar, *eproms.html*, med raden ”Mica802.ith”, ”DOS /Option till ABC802 med MICA winchester”, och kretsens innehåll. Utan årtal; kopian i Internet Archive är från 2010-08-14.
MICA-PROM:en jämförd byte för byte med Luxors DOS v.19: fyra byte, `$612B`–`$612C` och `$613E`–`$613F`, är ändrade från `$28` `$0D` till nollor.
På nätet: http://www.abc80.net/old/ver1/luxor/eproms.html

ABC-klubben, LUXNET-programmen Dataindustrier AB: hjälpprogrammen till LUXNET, ett trettiotal program i BASIC (`L.BAC`, `LOGIN.BAC`, `CD.BAC`, `CDLFD.BAC`, `CDPFD.BAC`, `CENID.BAC`, `DF.BAC` med flera), skrivna 1984–85 av Benny Löfgren och Göran Nordenborg, ur ABC-klubbens arkiv. Där finns också `PRATA.BAC` av Kristoffer Eriksson (1985, insänt 1986), som använder `FILESTAT`.
`NETCALL` (`$6027`) och meddelandet på `$FDC1`–`$FDC6`, underfunktionerna till funktion 14, pekarna till UFD, LFD och PFD, nätets felkoder och hur `L.BAC` läser biblioteket och filernas RIB.
Arkivet: https://www.devili.iki.fi/pub/Luxor/software/ABC-klubben/abc800/luxnet/

Nodkortets ROM Luxor: ROM till nodkortet i LUXNET, en 27128 på 16 kilobyte, märkt 55 21141-01 och daterad 85-04-12.
Kortets Z80 och portarna `$7C`–`$7F`, avbilden på 2 516 byte som kopieras till `$6000`, hoppet `JP $684F` på `$6009`, funktionerna 1–`$16` och anropen av BASIC:ens rutiner.

Pascal 800 Luxor Datorer: *Pascal 800*, version 1.04, art. nr 68 29067-43, ”ABC 800 – Pascal System Ver. 1.04, Copyright (C) Dataindustrier AB”, skivavbilden `pascal10.td0` (640 kilobyte, UFD, med Teledisks packning) i devili.iki.fi:s arkiv. Skivan är den finska utgåvan, med DOS:en 6.00 och etiketten ”NOKIA”. Utan årtal. Kompilatorn och länkaren har datum från 1982 (PASCOMP 1982-06-09 och PASLINK 1982-07-15). Systemprogrammen på skivan har datum till och med 1983-05-09 (ABC-DOSGEN, Vers. 6.00), så skivan är gjord tidigast då.
Filerna PASSYS.ABS, PASCAL.ABS, PASCOMP.COD, PASLINK.COD och EDIT.ABS, kompilatorn ”ABC800 – Pascal Compiler Ver. P-1.01”, länkaren ”ABC800 – Pascal P-Code Linker Ver. P-1.02” och editorn ”ABC800 Editor version 1.05”. Tolkarna, kodfilernas format, segmenten, minnet och kommandona är lästa ur programmen själva, som maskinkod och p-kod, och prövade i emulatorn för ABC 802: DOS:en 6.20 (CMDINT.SYS) startar med START.BAC, program kompileras med PASCOMP, länkas med PASLINK och körs med PASSYS och PASCAL, och EDIT läser och ändrar en källfil. PASCAL.ABS saknar kompilatorns sökrutiner (IDSEARCH och TREESEARCH) och kör därför inte PASCOMP. BASICINI.SYS är inte med, ef-

tersom den ställer skärmen för ABC 800, och skyddsprovet i PASSYS.ABS och PASCAL.ABS är förbigånget, eftersom avbilden inte har de trasiga sektorerna. Arkivet har ingen handbok till programmet, och kompilatorns flaggor utom L och B, en del av länkarens kommandon och de flesta av editorns är inte utredda.
Arkivet: https://www.devili.iki.fi/pub/Luxor/software/ABC800/

Ord 800, skivan Luxor: Ord 800, version 2.4, skivavbild om 160 kilobyte i abc80.net:s arkiv, med programmen START, YRSTART, YRINIT, YRLINK och YRBASIC i BASIC och maskinkoden YRASM.ABS.
Startprogrammen, minnets användning och kopieringsskyddet är lästa ur programmen och prövade i emulatorn för ABC 802. Där är skyddsprovet förbigånget med en lapp på maskinkoden (`ordlapp.py`), eftersom emulatorns skivkort inte har kontrollerns kommando `$10`. Avbilden har sidnumret `$42` kvar i huvudena på spår 0. Arkivet har också ORD 800 W V 2.6 (68 20031-46) för skivenheter om 640 kilobyte, som inte är prövad.
Arkivet: https://www.abc80.net/archive/luxor/sw/disk_images/ABC800/

Programmet i ABC 830:s kontroller Luxor: EPROM på 2 kilobyte till skivkontrollern 55 10828 (kortet 55 10900-01) för BASF 6106, `BASF02.bin`, som är MAME:s `basf .02.7c` med samma CRC, 5daba200. Kommandotolken med hopptabellen på `$00C4`, kommando `$10` på `$0340` och READ ADDRESS på `$0385` är lästa ur programmet.
Arkivet: https://www.abc80.net/archive/luxor/Prom/fw/Floppy-controller/

Kalkyl 800, demoskivan Luxor Datorer: KALKYL 800 DEMO, skivavbild om 160 kilobyte i abc80.net:s arkiv. Etiketten säger 2.15 (68 22011-40), och samma nummer står i programfilen `WSYS.ABS`. Talet 2.13 som också finns på skivan ligger i en av de andra filerna. Konfigureringarna är `ABC802 32K` och `ABC806`. Skivans filer heter `WSYS` med olika tillägg, och bland dem finns `WSYSPROT` och `WSYSCONV`. Skivan är inte körd.
Arkivet: https://www.abc80.net/archive/luxor/sw/disk_images/ABC800/

CALC och SPRIT ABC-klubben: kalkylprogrammen CALC (1985, ursprungligen för ABC80) och SPRIT (1986) i klubbens programbank, i BASIC.
Programmen och deras beskrivningar är lästa, inte körda. CALC är ett rutnät där varje ruta har en funktion av sex slag. SPRIT har namngivna variabler och formler som BASIC-satser, som fogas till ett program och kedjas in.
Arkivet: https://www.devili.iki.fi/pub/Luxor/software/ABC-klubben/abc800/

Programskivan Programskivan till den här boken, 2026: Forth för ABC 802 (`FORTH.ABS` och `BASICINI.SYS`), ORD, KALKYL med `KALKYL.ABS` och kalkylerna `OFFERT.KAL` och `BUDGET.KAL`, och VERK med `VERK.ABS`, `VERK.OVL`, introduktionen `VERK.HLP` och brevet `OFFERT.VRK`.
Källkoden ligger i förrådet, i `forth/`: `ord.bas`, `kalkyl.fs`, `kalkyl.bas`, `verk.fs` och `verk.bas`, med anteckningarna `README.md`, `KALKYL.md` och `VERK.md`. Forth är bokens Forth för ABC80 (*ABC80 inifrån*, band 2, exempel 7), flyttat till `$8000`. Tiderna i kapitlet ORD och KALKYL är mätta i emulatorn för ABC 802. `KALKYL.ABS` är Forth med KALKYL:s ord översatta, sparat ur emulatorns minne. Programmen och exemplen i kapitlet är körda i emulatorn med programskivan.

ORD Set Lonnert: *ORD. Ordbehandling på ABC 802. Bruksanvisning*, 2026. Ett häfte i A5 i stil med kontorsprogrammens häften, med skärmbilder ur emulatorn. Källan är `manualer/ord/ord.tex`.

KALKYL Set Lonnert: *KALKYL. Kalkylprogram för ABC 802. Bruksanvisning*, 2026. I samma stil som ORD:s, med felmeddelandena i en tabell och kalkylerna på skivan. Källan är `manualer/kalkyl/kalkyl.tex`.

MyAB:s CP/M Plus för ABC 802 MYAB Mikrokonsult AB, Göteborg: CP/M Plus, version 3.0, med BIOS 1.2C, ”ABC802 C version”, filerna CPM3802A.SYS, CPM3802B.SYS och CPM3802C.SYS och startkoden i BASICINI.SYS, avbilden `disk001.img` i abc80.net:s arkiv. Filerna daterade 1984-08-25 och 1984-10-18. Etiketten: ”CP/M Plus (3.0) för ABC8XX / MYAB

BIOS vers 1.2 C / För ABC-800, 802 och 806 / Format Luxor 832”.
Starten genom BASICINI.SYS och de två rättelserna i emulatorn. Laddarens val mellan CPM3802A, B och C, de bankade med 59 kilobyte för programmen och det obankade med 51, ur GENCPM:s huvuden i filerna; laddarens prov av port 6 och kort `$34` och bokstaven den väljer; bankbytet med kort `$34` i CPM3802B.SYS och CPM3800C.SYS; BIOS:ens klocka på CTC:ns kanal 3; katalogetiketten med datum. Avbilden ligger inte i förrådet.
Arkivet: https://www.abc80.net/archive/luxor/sw/disk_images/ABC800/cpm/640k/

Digital Research, CP/M 2.2 Digital Research: CP/M 2.2, CCP:n och BDOS:en i Clark A. Calkins källa `CPM22.Z80` (`cpm2-asm.zip`) och programmen på Xerox 1800:s distributionsskiva (`cpm22-b.zip`), ur The Unofficial CP/M Web Site, med tillståndet från Bryan Sparks (Lineo 2001, DRDOS, Inc. 2022).
CCP:n, BDOS:en och programmen ASM, DDT, DUMP, ED, LOAD, PIP, STAT, SUBMIT och XSUB; tillståndet att sprida dem.
Arkivet: http://www.cpm.z80.de/license.html

Digital Research, Pascal/MT+ Digital Research: *Pascal/MT+*, version 5.6.1 för 8080 och Z80 (`mtp561.zip`), ur The Unofficial CP/M Web Site. Hämtas när Pascal-skivan byggs och ligger inte i förrådet.

Russell, BBC BASIC (Z80) R. T. Russell: *BBC BASIC (Z80)*, version 5.00, den allmänna utgåvan för CP/M, under zlib-licensen.
Programmet på systemskivan, utom sidan `$0100`–`$01FF`, och licensen. Flyttalen i Pascal på ABC 802 är modulen FPP ur samma källa.

Infocom, Zork I Infocom: *Zork I*, berättelsefilen `zork1.z3` för Z-maskinen version 3, i förrådet historicalsource/zork1 (commit 5e9a9ce, 2019), under MIT-licensen sedan november 2025.
Filen på systemskivan, 86 838 byte, och licensen.
Arkivet: https://github.com/historicalsource/zork1

MAME MAME, emulatorn: drivrutinen för ABC 800, 802 och 806 (`src/mame/luxor/abc80x.cpp`), korten för ABC-bussen (`src/devices/bus/abcbus/`), bildkretsen (`src/mame/luxor/abc80x_v.cpp`), tangentborden (`src/devices/bus/abckb/`) och Z80 SIO/DART (`src/devices/machine/z80sio.cpp`), läst 2026-11-12.
ABC-bussens stift för XM*, XMEMFL* och XMEMWR*, RAM-kortet på 16 kilobyte för ABC80 som använder dem, och knepet med hämtningar på `$7800`–`$7FFF`, som drivrutinen har för ABC 800 och 806. För ABC 802 har den bara ROM:en och 64 kilobyte RAM, inte det särskilda läget, så den säger inget om det.
Tangentbordet ABC 99: två 8035, funktionstangenterna PF1–PF15, piltangenterna och kontakten för musen ABC R8 med tre knappar, och att det kan sitta på ABC 802 och 806. I DART:en står det sista tecknet kvar i mottagarens register när kön är tom. Teckengeneratorn 64 90191-01 som 4 kilobyte (CRC `4d54eed8`), blinkklockan som växlar efter 33 bildpulser och PAL-kretsen 2G, som inte är avläst.
https://github.com/mamedev/mame

Datablad

MC6845, Motorola Motorola: *MC6845 MOS CRT Controller (CRTC)*. Advance Information, Motorola Semiconductors, 19 s.
Adressutgångarna MA0–MA13 och RA0–RA4, DEN och CUR, teckenklockan på högst 2,5 MHz, markörens register R10 och R11 och dess blinkning, och den vertikala synkpulsens fasta längd på 16 linjer.

CD4020B, Texas Instruments Texas Instruments: *CD4020B, CD4024B, CD4040B Types.* Datablad SCHS030D, reviderat december 2003 (efter Harris Semiconductor).
Binärräknaren med fjorton steg, som räknar på den fallande flanken, och benen för utgångarna Q5, Q6 och Q13 (ben 2). I ABC 802 räknar den den vertikala synkpulsen och ger takten

för blinkningen, och i tangentbordet är den vakthund.

Toshiba MOS Memory Products 1983 Toshiba: *MOS Memory Products.* Databok, Toshiba Corporation, 1983.
Det statiska CMOS-minnet TC5514AP, 1024 ord om 4 bitar med samma ben som 2114 och högst 200 nanosekunders åtkomsttid (s. 131), som bildminnet är byggt av.
Inskannad: https://bitsavers.org/components/toshiba/_dataBook/1983_Toshiba_MOS_Memory.pdf

Toshiba C²MOS 1985 Toshiba: *C²MOS Integrated Circuits.* Databok, Toshiba Corporation, 1985.
Förteckningen över familjerna (s. 16), där TC40H000 är den snabba serien, kompatibel med TTL-serien 54/74. Bildminnets låskretsar och register är TC40H373 och TC40H374.
Inskannad: https://bitsavers.org/components/toshiba/_dataBook/1985_Toshiba_C2MOS_Integrated_Circuits.pdf

Z80 CPU User Manual, Zilog Zilog: *Z80 CPU User Manual* (UM0080).
`LDIR`, som backar programräknaren två steg för varje varv och tar emot avbrott mellan varven, och uppfriskningen med registret I på den höga halvan av adressen och R på den låga.

Böcker, tidskrifter och artiklar

ABC-bladet ABC-klubben: *ABC-bladet.* ABC-klubbens medlemstidning, Stockholm 1980–2010.
1983 nr 2, s. 2: Gunnar Tidners ledare, råd att spara programmen i BAS-format inför bytet till ABC 802. S. 13: ”ABC-klubbens årsmöte 83-02-12”, om ”den nya 802 som är den som kallas lilla ABC-800”. 1984 nr 1, s. 5: Herbert Söderström, ”Datorpolitik”, om hemdatorerna och ABC-datorerna i kontor och skola.
Inskannad: https://www.abc80.net/archive/luxor/ABC-bladet/

Alm 1983 Torbjörn Alm: *CP/M på ABC-datorer.* ABC-bladet 1983:4, s. 25.
Luxors CP/M för ABC 800, startad från sektor 16, och att samma skiva gick i ABC 802 och ABC 806 (s. 25). Uppgiften är hämtad ur *ABC80 inifrån*, band 2.

Bray, Dickens och Holmes 1983 Andrew C. Bray, Adrian C. Dickens och Mark A. Holmes: *The Advanced User Guide for the BBC Microcomputer.* Cambridge Microcomputer Centre, Cambridge 1983.
Bildkretsen 6845 i BBC-datorn (s. 359).
Inskannad: https://archive.org/details/manualcomputergame-bbc-cambridgemicrocomputercentre-theadvanceduserguideforthebbcmicro-1983-c048

Canon Cat Canon Cat (1987) och SwyftCard: Information Appliance, Forth i Cat och SwyftCard, priset och den enda texten. Huvudkällan är Ezra Shapiro: *A Spiritual Heir to the Macintosh.* Byte, vol. 12 nr 11, oktober 1987, s. 121 (Product Preview). Om SwyftCard och Information Appliance är uppgifterna ur andrahandskällor.
Inskannad: https://archive.org/details/byte-magazine-1987-10-rescan; https://www.digibarn.com/collections/systems/swyft/

Chung och Yuen 1978 Kin-Man Chung och Herbert Yuen: *A ”Tiny” Pascal Compiler.* Byte, vol. 3 nr 9–11, september–november 1978: del 1, *The P-Code Interpreter* (nr 9, s. 58); del 2, *The P-Compiler* (nr 10, s. 34); del 3, *P-Code to 8080 Conversion* (nr 11, s. 182).
En delmängd av Pascal. Kompilatorn var skriven i North Star BASIC och gav p-kod, som tolkades eller översattes till 8080-kod; tolken var skriven i Pascal. Författarna använde en Altair 8800 med 44 kilobyte och North Star-skivor. Förebild för Pascal på ABC 802.
Inskannad: https://archive.org/details/byte-magazine-1978-09, https://archive.org/details/byte-magazine-1978-10, https://archive.org/details/byte-magazine-1978-11

Commodore 1982 Commodore: *Commodore 64 Programmer's Reference Guide.* Commodore Business Machines och Howard W. Sams, 1982. ISBN 0-672-22056-3.
Bildkretsen VIC-II, som ser 16 kilobyte åt gången (s. 102), och ljudkretsen med tre röster, ADSR-kurvor, filter och brus (s. 184).

Digital Research 1983 Digital Research: *CP/M Plus* (*CP/M Version 3*) *Operating System. User's Guide, Programmer's Guide, System Guide.* Digital Research Inc., Monterey 1983.
Bankat CP/M 3: minst 96 KB, upp till 16 banker, ett gemensamt område överst, normalt 16 KB (Programmer's Guide, s. 1-1–1-5). Det som är nytt mot CP/M 2.2 står i User's Guide: datum och lösenord (avsnitt 2.7, lösenorden bara i det bankade systemet; i Programmer's Guide s. 1, och radredigeringen i det obankade i tabell 3-2, s. 65), sökningen efter program (avsnitt 4.4) och kommandona (avsnitt 5, till exempel `DATE` s. 5-11, `GENCOM` s. 5-40 och `GET` s. 5-44). Manualen har ingen egen lista över nyheterna; jämförelsen med 2.2 är gjord här.

Falke 1983 Bengt Falke: *ABC 802 – den perfekta skoldatorn.* Mikrodatorn nr 4 1983, s. 126–127.
Rubriken går över uppslaget, och i skanningen försvinner a:et i ”perfekta” i mitten.
Test av ett av de första exemplaren: tangentbordet och lådan, skärpan och tecknens storlek, ljudet och grafiken, BASIC II och programmen från ABC80 i BAS-format, priset ”strax under 10 000 kr”, skoldator och terminal.
Inskannad: https://www.aef.se/Amatortidningar/Mikrodatorn/Mikrodatorn_1983-04.pdf

Isaksson och Kärrsgård 1980 *Avancerad programmering på ABC80.* Studentlitteratur, 1980.
Kapitel 5, Interaktion människa–dator: formulären med fälten mellan < och >, meddelanderaden och felen som texter (s. 67, 83–86). Förebilden för deklarationsprogrammet i kapitlet VERK.

Kullmar 1984 Bo Kullmar: *Historik.* ABC-bladet (ABC-klubben) 1984 nr 1, s. 12.
ABC-klubbens och datorernas historia: ABC 802 och ABC 806 våren 1983.
Inskannad: https://www.abc80.net/archive/luxor/ABC-bladet/

Macintosh Macintosh-projektet på Apple: Raskins början 1979, Jobs från 1981 och Macintosh 1984. Uppgifterna är ur andrahandskällor.
https://en.wikipedia.org/wiki/Jef_Raskin, https://en.wikipedia.org/wiki/Macintosh_128K

Markesjö 1978 Gunnar Markesjö: *Mikrodatorns ABC. Elektroniken i ett mikrodatorsystem.* Esselte Studium, 1978. ISBN 91-24-29008-4.
Lärobok som bygger på ABC80. Uppgifterna om ABC80 i jämförelsen med ABC 802: processorn, minnet, bildminnet, tangentbordet, ljudet, kassetten och V24.
Inskannad: https://www.abc80.net/archive/luxor/ABC80/Mikrodatorns-ABC_searchable.pdf

Raskin 2000 Jef Raskin: *The Humane Interface. New Directions for Designing Interactive Systems.* Addison-Wesley, 2000.
Raskins bok om gränssnitt, med Canon Cat som exempel: en enda text i stället för program, LEAP-tangenterna och uträkning i texten.

Sjöström 1996 Roland Sjöström: *Positionering under strategisk osäkerhet. Vol. 2: Luxor Datorer och persondatorbranschen.* Linköping Studies in Management and Economics, Dissertations No. 30, Linköpings universitet, 1996. ISBN 91-7871-699-3.
Doktorsavhandling; band 2 följer Luxor Datorer år för år. Pressvisningen 1983, IBM PC och Ericsson PC, Compis och hemdatorerna (s. 103, 123–126, 129), Jet 80 (s. 132), försäljningen och pressen (s. 135–136), skolmarknaden, 1984 och Nokia (s. 162, 165–166). Sidnumren är bandets tryckta.
Band 1, *En studie av positionering i en ny bransch*, citeras för testerna av ABC 802 och ABC 806 mot de nyare maskinerna (band 1, s. 137).

Williams 1982 Gregg Williams: *A Closer Look at the IBM Personal Computer.* Byte, vol. 7 nr 1, januari 1982, s. 36 ff.
Intel 8088 på 4,77 MHz, 40 KB ROM med BASIC, 16–64 KB RAM, färggrafikkortet med 320 × 200 bildpunkter i fyra färger (s. 36–45); bildkretsen 6845 på portarna 3D4 och 3D5 (s. 39–40).
Inskannad: https://archive.org/details/byte-magazine-1982-01

Wirth 1975 Niklaus Wirth: *Pascal-S: A Subset and its Implementation.* Berichte des Instituts für Informatik 12, ETH Zürich, 1975. DOI 10.3929/ethz-a-000147073.
En delmängd av Pascal för undervisning, med kompilatorn och tolken i ett program. Förebild

för den rekursiva nedstigningen i Pascal på ABC 802. Rapporten är inte läst här; innehållet är taget ur senare beskrivningar av Pascal-S.

Namnregister

Registret har de namn på rutiner, tabeller och variabler som står i texten, i bokstavsordning: först BASIC II och DOS v.20 i ROM:en, sedan det som bara finns i ROM:en med LUXNET-DOS, och sist CP/M:s BIOS och programmen. Varje post ger namnet, adressen, uppgiften i korthet och sidorna där namnet står. Där ROM:en med LUXNET-DOS har rutinen på en annan adress står den efter ”nät”. Fet sida är där namnet står tillsammans med sin adress. Namnen är valda för den här boken och för ABC80 inifrån; ROM:arna själva innehåller inga.

BASIC II och DOS v.20

ADR80 $7E92 plats på skärmen: rad gånger 80 plus kolumn – **21**

BEEP $0241 pipet, CHR$(7) – **33**
BRKPTR $FF85 pekare till BREAK-flaggan – 33

CASDRV $78A9 drivrutinen för CAS:, kassetten – **43**
CFILNR $1C2D läser # och filnummer – 42
CHRMSK $FF61 masken som tar bort bit 7 vid utskrift – **26**
CON $019A drivrutinen för CON:, skärmen och tangentbordet – 42–43, 46
CONINP $0322 läser en rad från tangentbordet – **33**
CONWID $FF54 radlängden för CON: – **42**
CRDRAM $FDE2 korttabellen i RAM – **50**
CRDTAB $6ED3, nät $6BD7 korttabellen: kortnummer och sektorer per kluster – **45**, 46
CRTADR $FF57 bildens början i bildminnet – **21**
CRTHOM $7768 bildens början tillbaka till $7800 – 21
CRTINI $02BB ställer bildkretsen vid start, 40 eller 80 tecken – 24
CRTRST $76A1 vrider bildminnet så att bilden börjar på $7800 – **21**
CRTSTA $776B skriver bildens början i R12–R13 – 21
CRTT60 $0471 bildkretsens register för 60 Hz – **18**
CRTTAB $0463 bildkretsens register för 50 Hz – **18**, 19
CURADR $02FA markörens adress i bildminnet – **24**, 42

DBUFRD $6033 läser en byte i kanalens buffert – 50
DBUFW2 $603C skriver en byte i kanalens buffert – 50
DCLOSE $6021 stänger en fil – **79**
DEVLNK $7021 länkar in enheterna i enhetslistan – **43**
DEVLST $FF7B första posten i enhetslistan – **46**
DFNJP $6048 katalogfunktionerna – 50
DOPEN $6018 öppnar en fil – **79**
DOSDEV $6E2D, nät $6AEF DOS:ens enhetsposter – **45**, **46**, **52**
DOSDRV $6D85, nät $6968 drivrutinen för alla DOS-enheter – **45**, 46, 51–52
DOSINI $6000 DOS:ens första hopp, till DOSSTA – **45**
DOSRST $6CDE, nät $6A39 alla kanaler lediga, felvektorerna nollade – **45**
DOSRUN $6F81, nät $6F20 BYE, KILL och NAME – **45**
DOSSTA $6C64, nät $68E3 DOS:ens start – **45**, **48**, 51
DOSSTH $6F9A, nät $6F4E huvudet till DOS:ens satstabell – **45**
DPREP $6015 skapar en fil (PREPARE) – **79**
DRDREC $602D läser en post – **79**
DRUN $601E läser in och kör ett program – 63, 74
DSELCD $6060 väljer skivkortet – **50**, 51
DSETRC $602A sätter postnumret – **79**
DSYSUN $FFF7 systemenheten – **50**, **52**, 53
DWRREC $6030 skriver en post – 50, **79**

EXTSCN $00E3 letar efter tillägg i $6000–$7700 – 48

FADD $4F41 flyttal, addition – **77**
FCOMP $4C61 flyttal, jämförelse – **77**
FDIV $505F flyttal, division – **77**
FMUL $4FEA flyttal, multiplikation – **77**
FMUSE $4575 skriver ett tal efter en mall – **77**
FNDFIL $10E5 letar upp en öppen fil – 42
FNHDR $0661 nyckelordstabellen för funktionerna – **41**, 43
FSUB $4F39 flyttal, subtraktion – **77**
FTODEC $5153 flyttal till text – **77**

GETDEF $7B11 kopierar standardinställningarna för V24: och PR: – **38**

KEYDN $3B70 SYS(8), om en tangent är nedtryckt – **33**, **42**
KEYFLG $FFE2 bit 7 = en tangent har kommit – **33**
KEYINT $043E avbrottsrutinen för tangentbordet – 32, **33**, 58
KEYSET $0191 sätter KEYFLG – 33
KEYWT $03A0 väntar på en tangent och blinkar markören – **27**, **34**

LASTKY $FFE3 den senaste tangentkoden – 32, **33**
LDBCHL $30A0 BC från (HL), HL ökas två steg – 41
LDDEHL $31E9 DE från (HL), HL ökas två steg – 41
LDIR7F $7FFD LDIR och RET, i ROM:ens sista byte – **37**, **38**, 39, **43**, 44, **75**
LINFD $029D ny rad, rullar skärmen med LDIR – **20**, 21

MEMARE $7894 lägger MEM:s buffert på $FC00 – 39, 48
MEMDRV $7800 drivrutinen för MEM:, RAM-skivan – **39**
MEMOPN $7815 öppnar en fil på MEM: – 39
MEMPAG $787E sidan i RAM för ett block på MEM: – 39
MEMRD $7841 läser ett block från MEM: – 39
MEMTOP $FF0E minnets topp – **48**
MEMWR $7864 skriver ett block till MEM: – 39

OPNINI $6DF2 öppnar BASICINI.SYS – **45**, 57

PRDEF $7FEB standardinställningen för PR: – **38**, **43**
PRFREE $4504 skriver ett tal i fri form – **77**
PRINIT $7003 starten av PR:, skrivaren – **43**
PRINP $7317 läser från skrivar- eller V24-kanalen – **21**

RAMINI $014F startvärdena för variablerna i RAM – 42
RECLFT $6478 delrutin till SETREC i v.19 – 45
RECSEC $62B2 delrutin till SETREC i v.19 – 45

SCRCHR $01E6 skriver ett tecken på skärmen – **26**

SECIO $608F, nät $60C7 läser och skriver en sektor på skivkortet – 47
SETDOT $047D SET DOT – **27**
SETEND $6566, nät $6294 sätter filens slut (KILL) – 51
SETREC $6457, nät $62CE sätter postnumret – 45
SETW5 $0420 högst 80 tecken, 40 eller 80 tecken på skärmen – **24**, 42
STBCHL $31F0 BC till (HL), HL ökas två steg – 41
STDEHL $31F5 DE till (HL), HL ökas två steg – 41
STHDR $088B nyckelordstabellen för satserna – **41**, **42**, 43
TRMIN $7EA8 terminalprogrammets läsning av tangentbordet – **34**
TXTNUM $47F3 text till flyttal – **77**
USEFLD $4713 läser en mall för PRINT USING – **77**
V24DEF $7FE0 standardinställningen för V24: – **38**, **43**
VWRAP $7761 håller bildens början inom bildminnet – **21**
WR5INI $0181 DART:ens Write Register 5 vid starten – **38**

LUXNET-DOS

DNET $6027 anrop till nätet (NETCALL) – **50**, **51**, **53**
DNONE $6DC0 RET, för de tomma platserna i hopptabellen – 50
DOSADR $6069 platser i RAM för programmen på skivan – **50**, 54
MSG.FC $FDC1 meddelandets funktionskod – **53**
MSG.SFC $FDC2 meddelandets underfunktion – **53**
NETBAK $6DDE dit nodkortet lämnar tillbaka – 51–52, 54
NETCALL $6027 programmens namn på DNET – **53**, **87**
NETCRD $6E1E tabellen över nodkorten – **50**, 54
NETFN $6DF4 kör en lokal funktion åt nätet – 51–52
NETHK $6DC1 frågar nätet vid varje DOS-anrop – **50**, 51–52
NETOUT $6009 OUT (C),A: här tar nodkortet över – **50**
NETUNT $6BF8 översätter enheten, UFD:, PFD: och LFD: – 52
RPDRV $6E2C drivrutinen för skrivarna i centralen – 52

CP/M:s BIOS

BOOT $F600 kallstarten – **57**, 58
BREDDEN $F633 skärmens bredd, 80 – 58–60
KONUT $F77C skriver ett tecken på skärmen – 58
SECTRAN $F630 översätter sektornumret – 58

Pascal och BBC BASIC

BAKAT pc.pas: hopp bakåt – 65
FAKTOR pc.pas: en faktor – 64
FRAMAT pc.pas: hopp framåt, målet fylls i senare – 64
FYLL pc.pas: fyller i målet för ett hopp framåt – 64
LADDAD Pascal-tolken: körs efter inläsningen – 63
NASTA pc.pas: läser nästa symbol – 64
SATS pc.pas: en sats – 64–65
TERM pc.pas: en term – 64
TICKS BBC BASIC: väntar 10 millisekunder – 59
UTTRYCK pc.pas: ett uttryck – 64

Sakregister

Fet sida är där ordet förklaras.

25 rader, **21**

40 och 80 tecken, **24**, 42

50 och 60 Hz, 18, 21

6845, *se* bildkretsen 6845

ABC 800, **3**
ABC 800 M
    samma tolk, **41**
ABC 802
    datorkortet, **8**
    datorn i bildskärmen, **7**
    jämförd med ABC80, **15**
ABC 806, 3–4
ABC 850, 47, 49
ABC NET, 48, **49**
ABC-bussen, **13**, 45
ABC80, 1, 15
anslutningar, **14**
attribut, **25**, 26
avbrott, 13, **14**
    från tangentbordet, 30, **33**

bankväxling, *se* minnet
BASIC II, **41**, 45
BASICINI.SYS, **48**, 57
BBC BASIC, **59**
bildkretsen 6845, **18**, 19
    registren, **18**
bildminnet, **19**, 20, 37
BIOS, **58**

CAS:, *se* kassetten
Compis, 4
CP/M, **55**
    CP/M 2.2, **57**
    CP/M Plus, **55**
CTC, 12–13, **14**

DART, 13, **14**, 30
DOS
    v.19 och v.20, **45**, 49
DOSGEN, **46**
drivrutin, **46**, 52

enhet, **46**
enhetslistan, 43, **46**

Ericsson, 4
externt minne, **39**, 56

flyttal, **64**
formatering, **46**
Forth, **74**, 76
FPP, 64
funktionstangenterna, **31**, 72

grafik, **26**

hårddisken, **46**, 47
hämtning eller data, **37**

IBM PC, 4–5
inverterade tecken, **27**

KALKYL, 74, **76**, 77
Kalkyl 800, 70, **71**, 72
kassetten (CAS:), **43**
klockan (CP/M Plus), **57**
klockor
    kristallen, **13**
kluster, **46**, 47
kommunikationsmodulen, **12**, 14
kompilatorn, **64**, 65

`LDIR`, **37**, 39
`LIB`, **53**
ljud, 31, **32**
LUXNET, **49**
LUXNET-DOS, **49**
Luxor, 3, 5, 70

Macintosh, 82
markören, 19, **27**
MEM:, *se* RAM-skivan
MICA, **47**
minneskarta, **13**
minnet
    64 kilobyte RAM, **38**, 55
    tre minnen på samma adresser, **35**
MyAB, **55**, 56

NMI, **14**
nodkortet, 50, **51**
nätet, *se* ABC NET

ORD, **74**, 75

Ord 800, **70**, 72
ordbehandling, 70, 80
OWOCO, 1

p-kod, 61, **62**, 63, 66
PAL-kretsen, 23, **25**
Pascal, **61**
Pascal 800, **66**, 68
Pascal/MT+, **60**
portkarta, **14**
PR:, *se* skrivaren
PRIMTAL, **60**
priser, **3**, 28, 70

RAM, dynamiskt, **36**
RAM-skivan (MEM:), **39**, 48, 75, 78, 81
Raskin, Jef, 82
ROM:en, **36**
rullning, **20**, 21
Russell, R. T., 59

serienumret, **47**
`SET DOT`, **26**, 42
skivformat
    CP/M, **58**
    DOS, **47**
skivminnet, **45**
SKRIV, **59**
skrivaren (PR:), **43**
skärmsläckaren, **27**
Smartaid, 1
synkpulsen, **19**
`SYS`, 33, **42**

tangentbordet
    de fyra varianterna, **28**
    i datorn, **33**
    koderna, **31**, 32
    processorn, **29**
    sladden, **30**
    styrkoderna, **31**
teckengeneratorn, 19, **23**
terminalprogrammet, **43**
tillägget på `$7000`, **43**
tolken (Pascal), **63**

UFD:, PFD: och LFD:, **52**, 54

V24:, 12, **43**
VERK, **80**, 81

`WIDTH`, **42**
Wirth, Niklaus, 61
Wolkert, Olle, 1

Z80, **15**
Zork I, **60**

återställning, **14**

överlägg, **78**, 81